

Proving Authenticated Key Exchange via Memory-Efficient Reductions

Jiaxin Pan[✉] and Runzhi Zeng[✉]

University of Kassel, Kassel, Germany
{jiaxin.pan, runzhi.zeng}@uni-kassel.de

Abstract. We initiate the study of memory efficiency in proving the security of authenticated key exchange (AKE) protocols: We first revise the security model for AKE protocols in order to prove their security in a memory-efficient manner without comprising its capability of capturing usual attacks. We formally show that security in our model implies previous ones, and thus our model captures the same security as before.

After that we propose a generic construction of AKE from key encapsulation mechanisms (KEMs) and digital signature schemes, motivated by the signed Diffie-Hellman protocol. Under the multi-user security of the signature scheme and (relatively weak) oneway-security against plaintext checking attacks of the KEM, our generic construction is proven to be tightly secure (in terms of success probability) via memory-efficient reductions in the random oracle model. We refer to our reductions as memory-efficient rather than memory-tight, since their memory requirements grow proportionally with the number of users. This growth is not an artifact of our analysis, but rather stems from the necessity of solving the dictionary problem within our proof. By the result of Pagh (SIAM J. Computing, 2002), such user-dependent memory consumption is unavoidable. Nevertheless, our proof is more memory-efficient than the previous reductions for AKE, including even those that are non-tight. Given that most post-quantum assumptions (e.g., the Learning-With-Errors and Short-Integer-Solution assumptions) are memory-sensitive, our work holds significant value for post-quantum AKE protocols.

1 Introduction

Reductions are crucial for proving security of cryptographic protocols, where a reduction is constructed to show that if an adversary $\mathcal{A}$ can break the security of a protocol Π then the reduction $\mathcal{R}$ can use $\mathcal{A}$ to solve some underlying hard problem P . By the hardness of these well-established problems, we can conclude the security of Π .

CONCRETE SECURITY AND TIME-ADVANTAGE-TIGHTNESS. The analysis of concrete security is to quantify security by measuring how much resources the reduction $\mathcal{R}$ needs in addition to that of $\mathcal{A}$. We often call this tightness between a security reduction and an adversary. Previous work (e.g. [7,6,28,37]) typically consider the Time-Advantage (TA) tightness between adversary $\mathcal{A}$ and the corresponding reduction $\mathcal{R}$. More precisely, let $(t_{\mathcal{A}}, \varepsilon_{\mathcal{A}})$ and $(t_{\mathcal{R}}, \varepsilon_{\mathcal{R}})$ be the running time and advantage of $\mathcal{A}$ and $\mathcal{R}$, respectively, and they show that $t_{\mathcal{A}} \approx t_{\mathcal{R}}$ and $\varepsilon_{\mathcal{A}} \leq L \cdot \varepsilon_{\mathcal{R}}$, where L is called the security loss of a reduction. A smaller L gives a tighter reduction and stronger security guarantees for a cryptographic protocol. When L is a small constant, the security reduction is called tight in the early literature (such as [7,6,28,13,37]), and we write $\varepsilon_{\mathcal{R}} \approx \varepsilon_{\mathcal{A}}$.

Achieving tightness is highly desirable, since it does not only provide stronger security guarantees as mentioned earlier, but also allow one to instantiate the scheme with shorter security parameters. In the past 10 years, it has been an active direction to study tightly secure cryptographic primitives, such as signature schemes [28,13,39], public-key encryption schemes [18,24], and authenticated key exchange (AKE) protocols [4,23,34,31,37,40].

MEMORY-TIGHT / MEMORY-EFFICIENT REDUCTIONS. Memory efficiency is a crucial aspect to consider when analyzing algorithms, yet it has often been neglected in security reductions. Memory inefficiency can undermine the effectiveness of tight security reductions, especially since several problems commonly used in cryptography are sensitive to memory capacity. In essence, these problems can be resolved more swiftly with access to larger memory resources. According to the best known cryptanalytic results [33,42,14,10,11,2,26,5,25], prominent examples in the public-key setting include factoring, Discrete

Logarithms (DL) in finite fields, Learning Parities with Noise (LPN), Learning With Errors (LWE), approximate Shortest Vector Problem, and Short Integer Solution (SIS).

Surprisingly, many known tightly secure cryptosystems are based on memory-sensitive problems, but their security reductions are not memory-efficient, for instance, the tight LWE-based AKE protocol of Pan, Wagner, and Zeng [40]. These schemes often need to maintain some lists in order to keep track of adversaries’ queries or to program the random oracles (ROs). In 2017, Auerbach, Cash, Fersch, and Kiltz [3] initiated the analysis of memory efficiency for security reductions and proposed the notion of memory-tight reductions, meaning $m_{\mathcal{R}} \approx m_{\mathcal{A}}$ where $m_{\mathcal{A}}$ and $m_{\mathcal{R}}$ are the amount of memory used by $\mathcal{A}$ and $\mathcal{R}$. Following [30], we distinguish the two different tightness notions, Time-Advantage (or TA for short) tightness and Time-Advantage-Memory (or TAM) tightness, in this section.

Since its introduction in 2017, memory tightness is a well-established concept that attracts broad interests, but only limited types of primitives can be constructed via TAM-tight reductions. These are the RSA- and Diffie-Hellman-based (DH-based) signature schemes [12], the DH-based public-key encryption schemes [9,30,20], and the “Encrypt-then-PRF” authenticated encryption scheme [20]. This indicates that constructing TAM-tight schemes is rather challenging, since a TAM-tight reduction has to simulate the security game with limited memory and time. In some cases [3,45,21,22] it is even proven to be impossible. Hence, it is interesting not only in practice, but also in theory, to investigate the memory efficiency of proving primitives beyond signature and encryption schemes.

OUR GOAL: PROVING AKE VIA MEMORY-EFFICIENT REDUCTIONS. Our paper focuses on the memory efficiency of proving security of AKE protocols. This is particularly important for post-quantum protocols, as major post-quantum assumptions (e.g. LWE and SIS) are memory-sensitive.

An AKE protocol allows two parties to agree on a session key for a secure channel, and AKE protocols are crucial for secure communication over the internet. TA-tight AKE protocols are often constructed from the DH assumption [4,23,31,37] or the LWE assumption [40]. The security reductions of these protocols consume a large amount of memory and are highly memory-inefficient. This may not pose a significant problem for DH-based protocols, since the DL problem over elliptic curves is not memory-sensitive. However, for lattice, the large memory consumption lowers the security guarantee of the LWE-based AKE protocol [40], even though it is already TA-tight, since the LWE problem is memory-sensitive.

We stress that the memory-tightness is interesting, in particular, for non-TA-tight AKE protocols (e.g. [29,41]) from memory-sensitive assumptions, since these protocols suffer not only from security loss over time and advantage, but also from memory-inefficient reductions that further diminish their security guarantees. Currently, there is no lattice-based AKE protocol (regardless of its TA-tightness) with memory-efficient reductions.

DIFFICULTIES IN ACHIEVING OUR GOAL. Constructing memory-tight (or memory-efficient) AKE protocols is more challenging than encryption or signatures, since AKE-attacks are more complex and dynamic. Adversaries against AKE have full control on the communication channel, and they can reveal the shared session keys and corrupt some of the honest users’ long-term secret. As a result, the AKE security model needs to keep track of the protocol transcripts that have been transferred, the shared session keys, and the honest users’ long-term key. This requires a large amount of memory. In addition, for protocols in the random oracle (RO) model, one will need to keep track of the RO history. Hence, the security proof of AKE protocols is rather memory-inefficient. Concretely, the memory consumption of proving TA-tight AKE protocols in [23,31,37,40] is roughly

$$m_{\mathcal{A}} + (O(N_{\text{user}}) + O(N_{\text{ses}}) + O(Q_h)) \cdot m', \quad (1)$$

where N_{user} , N_{ses} , and Q_h are numbers of users, sessions, and RO queries, respectively, and m' is the bit-length of representing either a group element for the DH-based protocols or a lattice vector for the lattice-based one. For today’s social media applications, N_{ses} can easily be a billion ($\approx 2^{30}$). For 128-bit security, $Q_h \leq 2^{64}$. Such a large memory consumption as in Equation (1) lowers the security guarantees provided by their TA-tight proofs and restricts the interest of TA-tight AKE.

Not only the TA-tight AKE protocols suffer from memory-inefficient reductions, but also the non-TA-tight ones. We consider the non-TA-tight ones in [16,29,41], since they can be implemented with post-quantum assumptions. The memory consumption of their security reductions is at least

$$m_{\mathcal{A}} + (O(N_{\text{user}}) + O(N_{\text{ses}})) \cdot m'. \quad (2)$$

AKE protocols in the quantum RO model (QROM) [29,41] are proven secure with reductions that do not reprogram the RO, since a quantum adversary can query a RO with a superposition, and it is hard to program this. Hence, it removes the $O(Q_h)$ -term in Equation (1). Similarly, it is the case for the protocol in the standard model [16]. However, N_{ses} remains a dominant factor.

1.1 Our Contributions

We improve the memory efficiency for proving AKE protocols. Specifically,

- We propose *a new AKE security model* that allows us to record adversaries’ history in a memory-efficient manner, and
- We construct *a generic AKE protocol* that preserves the TA-tightness of the underlying primitives with memory-efficient proofs. Our construction requires signatures and key encapsulation mechanisms (KEMs). Different to an ad-hoc protocol, ours is flexible and can be efficiently implemented with various memory-sensitive assumptions (e.g. LWE or DH over integers).

Technical Overview. We detail our contributions as follows:

A NEW MODEL. The main motivation of proposing a new model is that the current AKE models may not be suitable for memory-tight reductions. For instance, the models in [31,40,19,41] need to record information about whether a session exists, and this session has either concluded a session key or been rejected. As a result, security reductions in these models have to create extra memory and store this information. Our new model modifies the way of maintaining session information and, at the same time, captures the same security properties as in other AKE models [31,40,19,41]. Formally, we show (cf. Theorem 2 in Supp. Mat. C) that if a protocol is secure in our new model, then it is also secure according to the previous definition. Our proof is rather straightforward. For a fair comparison, our reduction is not memory-tight, but it shows that our new model does not capture less security than the previous ones.

We consider two-message AKE models because our memory-efficient protocol is two-message. Our model is presented in Section 3.

A GENERIC AKE WITH MEMORY-EFFICIENT PROOFS. We propose an AKE protocol *generically* from digital signatures and KEMs. Under our new security model, we show that our protocol preserves the TA-tightness of the signature and KEM schemes with *memory-efficient* reductions. We call them memory-efficient, as their memory consumption is only dependent of N_{user} , but independent of N_{ses} and Q_h . Compared with existing proofs for (either TA-tight or not) AKE as described in Equation (1), our protocol’s proof is much more memory-efficient. Our proof is in the random oracle model and requires the multi-user security of the signature scheme and the multi-user, oneway-security against plaintext checking attacks (OW-PCA) of the KEM.

In a nutshell, our generic protocol can be viewed as a memory-efficient generalization of the two-message Signed Diffie-Hellman protocol (as in [37]) with a subtle modification. The modification is to expand a random nonce in each protocol message, and the nonce enables our reduction to simulate the security game memory-efficiently. Although we add the nonce into the protocol transcript, this overhead is rather small, in particular, when we implement it with lattices. Compared with Kyber ciphertexts (≈ 1 KB) and Dilithium signatures (≈ 2.4 KB), our nonce is about 64 Bytes, and this is very marginal.

We briefly sketch some technical details here: We recall that the signed DH protocol is to use a digital signature scheme to sign the unauthenticated DH protocol. In 2022, Pan et al. [37] showed that the signed DH protocol is TA-tightly secure in the random oracle model, but their proof is highly memory-loose as in Equation (1). Our approach is to keep the TA-tightness and make it memory-tighter. We further generalize it with a suitable abstraction that can be implemented with post-quantum assumptions, such as LWE.

The proof in [37] has two major steps: Firstly, they eliminate active attacks with a TA-tight reduction to the signature security. It requires the signature scheme to be secure in the multi-user setting against adaptive signing key corruptions. We note that existing memory-tight signatures [12,46] do not consider adaptive corruptions, and we will discuss how to construct a new scheme for it with memory-efficient reductions.

Now, since the active attacks have been eliminated, their reduction can randomize the computational DH (CDH) challenge and embed it into every session TA-tightly by the random self-reducibility of CDH. Although their reduction cannot compute the session key for any session, it can use the strong DH

predicate [1] to carefully patch the random oracle so that the AKE adversary $\mathcal{A}$ does not detect that. After doing so, $\mathcal{A}$ has to solve the CDH challenge. This second step requires the use of strong DH assumption [1]. Because of patching the random oracle (RO), their reduction needs to use its memory to keep track of the RO history. Moreover, it has to store the protocol transcripts for each session and the user secret keys to answer corruption COR queries.

To remove the storage for RO queries, an idea is to replace the RO with a pseudo-random function (PRF) as in [3]. However, this is not straightforward because the reduction itself cannot compute the session keys for sessions where the DH challenges are embedded, and it needs to carefully patch the RO to “hide” this fact. Also, storing the protocol transcripts seems necessary for the reduction in [37] to check if the adversary submits a valid attack or not.

To overcome these two fundamental obstacles, we slightly modify the signed DH protocol by introducing a random nonce in each protocol transcript. We develop a message encoding technique that embeds critical information into these nonces. This allows our reduction avoid storing the protocol transcripts. Instead, we extract the transcripts from the adversary’s queries only when needed to simulate RO queries or the security game. As a result, we remove the memory consumption of storing RO queries and protocol transcripts. This largely improves the memory tightness of proving AKE protocols.

TAM-TIGHT KEMS IN THE MULTI-USER, MULTI-CHALLENGE SETTING. It is unclear to us whether there is a suitable strong-DH-like assumption in the lattice setting. We abstract the strong DH assumption used in [37] as a KEM that is oneway-secure against plaintext checking attacks (OW-PCA) in the multi-user, multi-challenge setting, where an adversary can see multiple public keys, check whether an encapsulated key is valid wrt a ciphertext and some user, and obtain multiple challenge ciphertexts wrt to different users. Unlike the signature scheme, we do not require security against adaptive corruptions for the KEM. We show that a multi-user, multi-challenge IND-CPA-secure PKE can TAM-tightly imply our multi-user, multi-challenge OW-PCA-secure KEM. The IND-CPA-secure PKE can be implemented with LWE. Our technique is derived from a combination of the QROM techniques in [32,43] and methods from memory-tight PKE constructions in [30].

We note that Jaeger et al. [30] turns an IND-CPA PKE into an OW-PCA PKE, but we suppose that their result is insufficient for our AKE proof. To be more technical, their advantage bound includes an additive term $|\mathcal{U}| \cdot \delta_{\text{KEM}}$ where $\mathcal{U}$ is the set of user identifiers and δ_{KEM} is the correctness error. For Crystals-Kyber512 [44], δ_{KEM} is about 2^{-139} (cf. [44, Table 1]). For a PKE $\mathcal{U}$ is relatively small, and thus the additive term will not be a problem. However, when it is used for our AKE protocol, a PKE user will correspond to a *possible* AKE session, and thus $|\mathcal{U}|$ is a parameter related to the number of all possible session and can even be 2^{128} or even larger (cf. our comparison in Supp. Mat. D.1). This makes the advantage bound using their result not meaningful for our application. In our KEM construction, we tighten this additive loss.

MEMORY-TIGHT SIGNATURES WITH ADAPTIVE CORRUPTIONS. Our AKE construction requires a multi-user-secure signature scheme, where an adversary $\mathcal{A}$ receives multiple users’ public keys and adaptively corrupts some user signing keys, and it can furthermore ask signature for some message m under user u . To win, $\mathcal{A}$ has to submit a signature σ^* on some message m^* under user u^* . We require strong existential unforgeability, namely, an adversary can forge on a previous message-user pair (m^*, u^*) with a fresh signature σ^* . Moreover, our AKE proofs only require the weaker one-per-message security as in [15], where the signing oracle only issues one signature per message-user pair. We name our security notion as multi-user strong unforgeability against adaptive corruptions and chosen-one-per-message attack (aka. $\text{MU}^c\text{-sEUF-1CMA}$).

We show that the single-user variant of our notion (sEUF-1CMA) implies $\text{MU}^c\text{-sEUF-1CMA}$ with a memory-tight reduction. The reduction loses a factor of the number of public keys in terms of advantage. Combining with the previous work in [12,46], this gives us a memory-tight, TA-non-tight signature schemes against adaptive corruptions from various assumptions, including lattices. While the previous memory-tight schemes [12,46] do not consider adaptive corruptions, our scheme is the first achieving memory-tightness in this setting, although it is TA-non-tight. We leave constructing TAM-tight signatures against adaptive corruptions as an open problem.

INSTANTIATIONS AND COMPARISON WITH THE RELATED PROTOCOLS. Putting things together, we can instantiate our generic AKE construction with integer lattices (using the LWE and SIS assumptions) or module lattices (using the module-LWE (MLWE) and module-SIS (MSIS) assumptions). Both instantia-

tions have memory-efficient reductions whose memory consumption is only dependent of the number of users N_{user} . For integer lattices, we manage to achieve “almost” TA-tight, where the advantage loses a factor of $O(N_{\text{user}}\lambda)$ where λ is the security parameter. This loss is due to the $\text{MU}^c\text{-sEUF-1CMA}$ signature scheme. The work of Pan, Wagner, and Zeng [41] has the same TA-tightness as ours, but ours are more memory-efficient, as it does not depend of N_{ses} (cf. the memory complexity of [41] as in Equation (2)).

For module lattices, we instantiate our scheme with Kyber [44] and Dilithium [35]. Our instantiation has security loss $O(N_{\text{user}}N_{\text{ses}})$ to the advantage of Kyber and $O(N_{\text{user}})$ to that of Dilithium. A natural module lattice variant of the Signed DH protocol will also be replacing the DH protocol with Kyber and signature with Dilithium. Our reductions have the same TA-tightness, but much less memory consumption, since the security proof of this natural protocol will have memory consumption as in Equation (1). The cost for a memory-efficient proof is 64 Bytes in each protocol message. We note that the natural module-lattice-based protocol already transfers several KB data and several MB for integer lattices. Adding another 128 Bytes is rather inexpensive. More details of our instantiations can be found in Supp. Mat. A.

LIMITATIONS ON OUR WORK. In our AKE proofs, the memory consumption is still dependent of the number of users. This memory loss is due to the fact that, after eliminating active attacks with the signature security, we still need to keep track of user’s secret keys to simulate the security game and construct a reduction to the KEM security. We note that this memory loss seems inherent because it is essentially the dictionary problem, and [36] shows that solving it requires such memory loss. We refer Remark 1 for detailed discussion.

2 Preliminaries

For an integer n , $[n] := \{1, \dots, n\}$. Let $\mathcal{X}$ and $\mathcal{Y}$ be two finite sets. We define $x \leftarrow \mathcal{X}$ to denote sampling an element x from $\mathcal{X}$ uniformly at random. $\text{Func}(\mathcal{X}, \mathcal{Y})$ is the set of all functions with domain $\mathcal{X}$ and codomain $\mathcal{Y}$.

Let $\mathcal{A}$ be an algorithm. We write $y \leftarrow \mathcal{A}(x)$ to denote that $\mathcal{A}$ outputs y on input x . If $\mathcal{A}$ is deterministic, then we may write $y := \mathcal{A}(x)$. We also write $y \leftarrow$ (or $:=$) $\mathcal{A}^f(x)$ if $\mathcal{A}$ has black-box access to some function (or oracle) f .

We use code-based games [8] to define and prove security. For all security games, we assume that all Boolean flags are initialized to false and sets and (ordered) lists are initialized to $\emptyset$. Let $\mathcal{L}$ be an ordered list. If $\mathcal{L}[i]$ has not been assigned a value, then we define $\mathcal{L}[i] = \perp$. The notation $\Pr[\mathbf{G}^{\mathcal{A}} \Rightarrow 1]$ denotes the probability that the final output of game $\mathbf{G}$ running an adversary $\mathcal{A}$ is 1.

2.1 Signature Schemes

We define multi-user strong unforgeability against adaptive corruptions and chosen-one-per-message attack (aka. $\text{MU}^c\text{-sEUF-1CMA}$) in Definition 1 that is needed for our memory-efficient AKE. We also define its weaker, single-user variant in Definition 1.

DEFINITIONS. A digital signature scheme with message space $\mathcal{M}$ and randomness space $\mathcal{R}$ consists of four algorithms:

- **Setup** takes the security parameter λ as input and outputs a scheme parameter par_{sig} for the signature scheme.
- **KG** takes par_{sig} as input and outputs a key pair (pk, sk) .
- **Sign** takes a secret key sk and a $m \in \mathcal{M}$ as input and outputs a signature σ .
- **Vrfy** takes a public key pk and a pair (m, σ) as input, outputs 0 or 1. We say σ is a valid signature of m under pk if $\text{Vrfy}(\text{pk}, m, \sigma) = 1$.

Since the security parameter and scheme parameter are fixed after setup, we ignore them and write $\text{par}_{\text{sig}} \leftarrow \text{Setup}$ and $(\text{pk}, \text{sk}) \leftarrow \text{KG}$. We specify the randomness in key generation and signing by writing $(\text{pk}, \text{sk}) := \text{KG}(\cdot; r)$ and $\sigma := \text{Sign}(\text{sk}, m; r)$.

Let $\mathcal{U}$ be a set of user identifiers, and let $(\text{pk}_u, \text{sk}_u)$ be the key pair of user $u \in \mathcal{U}$. $\text{MU}^c\text{-sEUF-1CMA}$ and sEUF-1CMA are defined in Definition 1. Discussion on how to define $\mathcal{U}$ can be found in Section 3.2 and [30, Section 2.6].

Definition 1 (Security for Signatures). Let $\mathcal{A}$ be an adversary against SIG. We define two security games $\text{MU}^c\text{-sEUF-1CMA}$ and sEUF-1CMA and say SIG is $\text{MU}^c\text{-sEUF-1CMA}$ secure, if, for all PPT adversaries $\mathcal{A}$,

$$\text{Adv}_{\text{SIG}}^{\text{MU}^c\text{-sEUF-1CMA}}(\mathcal{A}) := \Pr \left[\text{MU}^c\text{-sEUF-1CMA}_{\text{SIG}}^{\mathcal{A}} \Rightarrow \text{true} \right]$$

is negligible on λ . Similarly, SIG is sEUF-1CMA secure, if, for all PPT adversaries $\mathcal{A}$,

$$\text{Adv}_{\text{SIG}}^{\text{sEUF-1CMA}}(\mathcal{A}) := \Pr \left[\text{sEUF-1CMA}_{\text{SIG}}^{\mathcal{A}} \Rightarrow \text{true} \right]$$

is negligible on λ .

Game $\text{MU}^c\text{-sEUF-1CMA}_{\text{SIG}}^{\mathcal{A}}$	Oracle $\text{PK}(u)$	Game $\text{sEUF-1CMA}_{\text{SIG}}^{\mathcal{A}}$
00 $\text{par}_{\text{sig}} \leftarrow \text{Setup}$	15 if $\text{pk}[u] = \perp$:	22 $\text{par}_{\text{sig}} \leftarrow \text{Setup}$
01 $\text{win} := \text{false}, \mathcal{S}_{\text{sign}} := \emptyset$	16 $(\text{pk}[u], \text{sk}[u]) \leftarrow \text{KG}$	23 $(\text{pk}^*, \text{sk}^*) \leftarrow \text{KG}$
02 $O := \{\text{PK}, \text{COR}, \text{SIGN}, \text{FORGE}\}$	17 return $\text{pk}[u]$	24 $\text{win} := \text{false}, \mathcal{S}_{\text{sign}} := \emptyset$
03 $\text{run } \mathcal{A}^O(\text{par}_{\text{sig}})$	Oracle $\text{COR}(u)$	25 $\text{run } \mathcal{A}^{\text{SIGN}', \text{FORGE}'}(\text{par}_{\text{sig}}, \text{pk}^*)$
04 return win	18 $\mathcal{S}_{\text{cor}} := \mathcal{S}_{\text{cor}} \cup \{u\}$	26 return win
Oracle $\text{SIGN}(u, m)$	19 if $\text{pk}[u] = \perp$	Oracle $\text{SIGN}'(m)$
05 if $\exists \sigma$ s.t. $(u, m, \sigma) \in \mathcal{S}_{\text{sign}}$	20 $(\text{pk}[u], \text{sk}[u]) \leftarrow \text{KG}$	27 if $\exists \sigma$ s.t. $(m, \sigma) \in \mathcal{S}_{\text{sign}}$
06 return σ	21 return $(\text{pk}[u], \text{sk}[u])$	28 return σ
07 $\text{pk}[u] \leftarrow \text{PK}(u)$		29 $\sigma \leftarrow \text{SIG}(\text{sk}^*, m)$
08 $\sigma \leftarrow \text{SIG}(\text{sk}[u], m)$		30 $\mathcal{S}_{\text{sign}} := \mathcal{S}_{\text{sign}} \cup \{(m, \sigma)\}$
09 $\mathcal{S}_{\text{sign}} := \mathcal{S}_{\text{sign}} \cup \{(u, m, \sigma)\}$		31 return σ
10 return σ		Oracle $\text{FORGE}'(m^*, \sigma^*)$
Oracle $\text{FORGE}(u, m^*, \sigma^*)$		32 if $(m^*, \sigma^*) \notin \mathcal{S}_{\text{sign}}$
11 $\text{pk}_u := \text{PK}(u)$		33 $\wedge \text{Vrfy}(\text{pk}^*, m^*, \sigma^*) = 1$
12 if $u \notin \mathcal{S}_{\text{cor}} \wedge (u, m^*, \sigma^*) \notin \mathcal{S}_{\text{sign}}$		34 win := true
13 $\wedge \text{Vrfy}(\text{pk}_u, m^*, \sigma^*) = 1$		
14 win := true		

In $\text{MU}^c\text{-sEUF-1CMA}$, $\mathcal{A}$ can obtain multiple public keys by querying PK.

MEMORY-TIGHT IMPLICATION. In Lemma 1, we show that sEUF-1CMA implies $\text{MU}^c\text{-sEUF-1CMA}$. Our reduction is not TA-tight, but preserves the memory tightness.

Lemma 1 (sEUF-1CMA implies $\text{MU}^c\text{-sEUF-1CMA}$, memory-tightly). Let $\text{SIG} = (\text{Setup}, \text{KG}, \text{Sign}, \text{Vrfy})$ be a signature scheme where the randomness spaces of KG and Sign are $\mathcal{R}_k$ and $\mathcal{R}_s$, respectively. For any $\mathcal{A}$ against $\text{MU}^c\text{-sEUF-1CMA}$ security of SIG, if $\mathcal{A}$ issues Q_{pk} queries to the oracle PK, then there exists an $\mathcal{F}(\mathcal{U}, \mathcal{R}_k) \times \mathcal{F}(\mathcal{U} \times \mathcal{M}, \mathcal{R}_s) \times \mathcal{F}(\mathcal{U}, [Q_{\text{pk}}])$ -adversary $\mathcal{B}$ s.t.

$$\begin{aligned} \text{Adv}_{\text{SIG}}^{\text{MU}^c\text{-sEUF-1CMA}}(\mathcal{A}) &\leq e \cdot N' \cdot \text{Adv}_{\text{SIG}}^{\text{sEUF-1CMA}}(\mathcal{B}), \\ \text{Time}^*(\mathcal{B}) &= \text{Time}(\mathcal{A}) + O(N' + Q_{\text{forge}} + Q_{\text{sign}} + Q_{\text{cor}}) \cdot \text{Time}(\text{SIG}), \\ \text{Mem}^*(\mathcal{B}) &= \text{Mem}(\mathcal{A}) + \text{Mem}(\text{SIG}) + \log |\mathcal{U}| + \log N' + 1, \end{aligned}$$

where Q_{pk} , Q_{forge} , Q_{sign} , and Q_{cor} are the numbers of $\mathcal{A}$'s queries to PK (including direct and indirect queries), FORGE, SIGN, and COR, and the numbers of $\mathcal{B}$'s queries to SIGN' and FORGE' are the same as $\mathcal{A}$'s. N' denote the number of users involved in the game and we have $N' \leq Q_{\text{pk}}$.

Proof. We construct a reduction $\mathcal{B}$ in Figure 1. In the construction of $\mathcal{B}$, we use two random functions f_1 and f_2 to generate the randomnesses for KG and Sign, respectively. Here we use the memory-tight guessing technique proposed in [3, Section 3.4]. Let N' denote the number of distinct executions of GetKey. To randomly embed the challenge public key pk^* , we use a random function $f_3 : \mathcal{U} \rightarrow [N']$ and if $f_3(u^*) = 0$, then we let the public key of u^* as pk^* .

Note that repeated (regardless of direct or indirect) queries to PK does not help the adversary, because by the reduction in Figure 1, the same user identifiers always produce the same key pairs.

Suppose that $\mathcal{A}$'s FORGE query raises the flag win. Let such a query as (u^*, m^*, σ^*) . By the winning condition, $u^* \notin \mathcal{S}_{\text{cor}}$. Since f_3 is a random function, the probability that $f_3(u^*) = i^*$ is $1/N'$. This means that the probability that $\mathcal{B}$ embed pk^* into the user u^* is $1/N'$. However, we also need to consider the case

Reduction $\mathcal{B}^{\text{PK}', \text{FORGE}'}$ (par, pk*)	Alg GetKey(u)
00 $f_1 \leftarrow \mathcal{F}(\mathcal{U}, \mathcal{R}_k)$	13 if $f_3(u) = i^*$: return (pk*, $\perp$)
01 $f_2 \leftarrow \mathcal{F}(\mathcal{U} \times \mathcal{M}, \mathcal{R}_s)$	14 (pk _{u} , sk _{u}) := KG($f_1(u)$)
02 $f_3 \leftarrow \mathcal{F}(\mathcal{U}, [N'])$	15 return (pk _{u} , sk _{u})
03 win = false	Oracle SIGN(u, m)
04 $\mathcal{S}_{\text{cor}} := \emptyset$	16 (pk, sk) $\leftarrow$ GetKey(u)
05 run $\mathcal{A}^{\text{PK}, \text{COR}, \text{SIGN}, \text{FORGE}}$ (par)	17 if $f_3(u) = i^*$
06 abort	18 return $\sigma \leftarrow \text{SIGN}'(m)$
Oracle PK(u)	19 return Sign(sk, m ; $f_2(u, m)$)
07 (pk, sk) $\leftarrow$ GetKey(u)	Oracle FORGE(u, m, σ)
08 return pk _{u}	20 (pk, sk) $\leftarrow$ GetKey(u)
Oracle COR(u)	21 if Vrfy(pk, m, σ) = 1
09 $\mathcal{S}_{\text{cor}} := \mathcal{S}_{\text{cor}} \cup \{u\}$	22 win := true
10 (pk, sk) $\leftarrow$ GetKey(u)	23 if $f_3(u) = i^*$
11 if $f_3(u) = i^*$: abort	24 $\wedge \sigma \neq \text{Sign}(\text{sk}, m; f_2(u, m))$
12 return (pk, sk)	25 FORGE' (m, σ)

Fig. 1. Reduction $\mathcal{B}$ for Lemma 1.

that there exists $u' \neq u^*$ and $f_3(u') = i^*$, which is within the following probability $(1 - \frac{1}{N'})^{(N'-1)} \geq 1/e$. Therefore, we have

$$\begin{aligned}
& \Pr [\text{sEUF-1CMA}_{\text{SIG}}^{\mathcal{B}} \Rightarrow \text{true}] \\
&= (1 - \frac{1}{N'})^{(N'-1)} \frac{1}{N'} \cdot \Pr [\text{MU}^c\text{-sEUF-1CMA}_{\text{SIG}}^{\mathcal{A}} \Rightarrow \text{true}] \\
&\geq \frac{1}{e \cdot N'} \Pr [\text{MU}^c\text{-sEUF-1CMA}_{\text{SIG}}^{\mathcal{A}} \Rightarrow \text{true}] \\
&\implies \text{Adv}_{\text{SIG}}^{\text{MU}^c\text{-sEUF-1CMA}}(\mathcal{A}) \leq e \cdot N' \cdot \text{Adv}_{\text{SIG}}^{\text{sEUF-1CMA}}(\mathcal{B}),
\end{aligned}$$

and the running time and memory usage of $\mathcal{B}$ are

$$\begin{aligned}
\text{Time}^*(\mathcal{B}) &= \text{Time}(\mathcal{A}) + O(N' + Q_{\text{forge}} + Q_{\text{sign}} + Q_{\text{cor}}) \cdot \text{Time}(\text{SIG}) \\
\text{Mem}^*(\mathcal{B}) &= \text{Mem}(\mathcal{A}) + \text{Mem}(\text{SIG}) + \log |\mathcal{U}| + \log N' + 1
\end{aligned}$$

When constructing reduction in our AKE proof, the number N' will be set as the number of users involved in the AKE game.

2.2 Key Encapsulation Mechanisms

A KEM scheme KEM consists of four algorithms (Setup, KG, Encaps, Decaps) and a ciphertext space $\mathcal{C}$, a randomness space $\mathcal{R}$, and a KEM key space $\mathcal{K}$:

- On input security parameter λ , Setup outputs a system parameter par. The key generation algorithm KG(par) outputs a public-secret key pair (pk, sk). Since the security parameter and system parameter are public, for simplicity, we do not write them in algorithm's input explicitly.
- The encapsulation algorithm Encaps, on input pk, outputs a ciphertext $c \in \mathcal{C}$.
- The decapsulation algorithm Decaps, on input sk and a ciphertext c , outputs a KEM key $k \in \mathcal{K}$ or a rejection symbol $\perp \notin \mathcal{K}$.

Some KEM schemes (such as lattice-based schemes) may not have perfect decryption correctness. To capture this, we define the correctness definition for KEM schemes in Definition 2.

Definition 2 (KEM Correctness). We say KEM is ρ -correct if

$$\Pr_{(c, k) \leftarrow \text{Encaps}(\text{pk})} [k = \text{Decaps}(\text{sk}, c)] \geq \rho,$$

where the probability is taken over $\text{par} \leftarrow \text{Setup}$, $(\text{pk}, \text{sk}) \leftarrow \text{KG}$ and the randomness of Encaps. When using this definition in security proof, we may let $\delta := 1 - \rho$ and use δ to bound the probability of decryption error.

Definition 3 (OW-PCA security of KEM). KEM is OW-PCA secure if, for all PPT $\mathcal{A}$, $\text{Adv}_{\text{KEM}}^{\text{OW-PCA}}(\mathcal{A}) := \Pr \left[\text{OW-PCA}_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1 \right]$ is negligible.

Game $\text{OW-PCA}_{\text{KEM}}$ <pre> 00 $\text{par} \leftarrow \text{Setup}$ 01 $O := (\text{CHALPK}, \text{CHALCT}, \text{PCO})$ 02 $(u, i, k) \leftarrow \mathcal{A}^O(\text{par})$ 03 return $\text{PCO}(u, \text{CHALCT}(u, i), k)$ </pre> Oracle $\text{CHALPK}(u)$ <pre> 04 if $\text{pk}[u] = \perp$: 05 $(\text{pk}[u], \text{sk}[u]) \leftarrow \text{KG}$ 06 return $\text{pk}[u]$ </pre>	Oracle $\text{PCO}(u, c, k)$ <pre> 07 $\text{CHALPK}(u)$ 08 $k' := \text{Decaps}(\text{sk}[u], c)$ 09 if $k' = k$: return 1 10 else return 0 </pre> Oracle $\text{CHALCT}(u, i)$ <pre> 11 if $\text{c}[u, i] \neq \perp$: return $\text{c}[u, i]$ 12 $(c, k) \leftarrow \text{Encaps}(\text{CHALPK}(u))$ 13 $\text{c}[u, i] := c$ 14 return $\text{c}[u, i]$ </pre>
---	---

2.3 Memory-tight Reductions

$\mathcal{F}$ -ORACLE ADVERSARY. In this paper, we use the formalism of memory-tight reductions in [20,30]. All reduction adversaries in this paper have the form that can be divided into two parts: a part that simulates random functions, and a remaining part. The latter part cannot control the simulation of random functions in the former part. More precisely, let $\mathcal{B}$ be a reduction algorithm. We can write $\mathcal{B}$ in the form shown below for some set(s) of functions $\mathcal{F}$ and algorithm $\mathcal{B}^*$:

Reduction $\mathcal{B}^O(inp)$ <pre> 15 $f \leftarrow \mathcal{F}$ 16 $out \leftarrow \mathcal{B}^{*O,f}(inp)$ 17 return out </pre>
--

The time and memory complexity of $\mathcal{B}$ includes the complexity from $\mathcal{B}^*$ and complexity from the sampling, storing, and computing $\mathcal{F}$. More specifically, the (full) running time and memory usage of $\mathcal{B}$ are written as

$$\text{Time}(\mathcal{B}) := \text{Time}(\mathcal{B}^*) + \text{Time}(f) \text{ and } \text{Mem}(\mathcal{B}) := \text{Mem}(\mathcal{B}^*) + \text{Mem}(f).$$

As in [20], it is sufficient to use the reduced time complexity $\text{Time}^*(\mathcal{B})$ and reduced memory complexity $\text{Mem}^*(\mathcal{B})$ are defined as

$$\text{Time}^*(\mathcal{B}) := \text{Time}(\mathcal{B}^*) \text{ and } \text{Mem}^*(\mathcal{B}) := \text{Mem}(\mathcal{B}^*).$$

The argument in favor of using the reduced notions is as follow. Let $\mathcal{B}$ be an adversary against a computational hard problem. If $\mathcal{F}$ is independent of the hard problem and is black-box accessed by $\mathcal{B}$, then it is very unlikely that the independent time and memory usage for simulating $\mathcal{F}$ helps $\mathcal{B}$ to break the hard problem. To see this, a conservative way to measure how much the oracle $\mathcal{F}$ helps $\mathcal{B}$ is to replace $\mathcal{F}$ with a pseudorandom version of it. For example, when $\mathcal{F}$ is a random oracle, then we can replace $\mathcal{F}$ with a pseudorandom function PRF so that we do not do (memory-inefficient) lazy sampling. Such an approach was used by [3,9,12]. After the replacement, we can write the advantage of $\mathcal{B}$ as $\text{Adv}(\mathcal{B}) = \text{Adv}(\mathcal{B}^*) + \text{Adv}(\text{PRF})$, where $\text{Adv}(\text{PRF})$ denotes the advantage bound of PRF. PRF can be chosen independently, so we may choose PRF such that $\text{Adv}(\text{PRF})$ is negligible compared to $\text{Adv}(\mathcal{B}^*)$, namely, PRF itself does not significant aid $\mathcal{B}^*$ to break its challenge. This observation motivates the definition of *reduced complexity* which simplifies the notations of theorems and helps us focus on the memory usage of the core adversary (i.e., $\mathcal{B}^*$) without taking care of the running time and memory usage of black-box-access oracles. We refer [20] for more discussion on it.

In this paper, we use reduced complexity to measure the running time and memory usage of reductions. In our proofs, $\mathcal{F}$ is normally a set of random functions or set of random injective functions with specific domains and codomains. One can instantiate random functions by using pseudorandom functions. For random injective functions, one can choose appropriate encryption schemes as instantiations. We refer

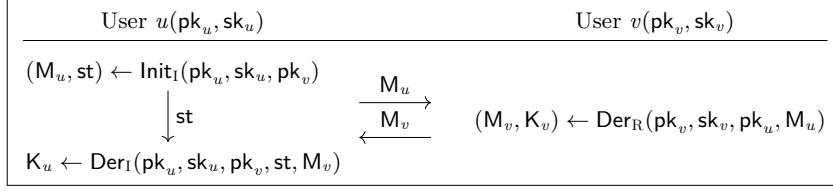


Fig. 2. Illustration for a two-message AKE protocol execution between users u and v .

readers to [20, Lemma 2] and [30, Section 2.2] which discussed how much advantage of the adversary will increase when random objects are replaced by pseudorandom counterparts.

For an $\mathcal{F}$ -oracle adversary $\mathcal{B}$, if the context of is clear, then we will ignore $\mathcal{F}$ and simply write it as adversary $\mathcal{B}$.

MESSAGE ENCODING TECHNIQUES. Let $\mathcal{X}$ and $\mathcal{Y}$ be two sets. If $|\mathcal{Y}| \geq |\mathcal{X}|$, then $\text{InjF}(\mathcal{X}, \mathcal{Y})$ denotes the set of all injections from $\mathcal{X}$ to $\mathcal{Y}$. For $f \in \text{InjF}(\mathcal{X}, \mathcal{Y})$, we write f^{-1} as its inverse. If $y \in \mathcal{Y}$ has no preimage, then $f^{-1}(y) = \perp$ where $\perp$ is a special symbol indicating “rejection” or “not exists”. If $y = (y_1, y_2) \in \mathcal{Y}_1 \times \mathcal{Y}_2$ is a tuple, then in this paper, we define that $y \neq \perp \iff y_1 \neq \perp \text{ and } y_2 \neq \perp$.

We use the message encoding technique proposed in [20,30] that helps reductions to hide state information in some randomly generated object. For example, let (a, b) be a random pair from $\mathcal{Y}^2$ and $x \in \mathcal{X}$ be some element that is coupled with (a, b) , e.g., x is the index of (a, b) . In reduction, we can generate (a, b) by picking two random injections $f_1, f_2 : \mathcal{X} \rightarrow \mathcal{Y}$ and computing $a := f_1(x)$ and $b := f_2(x)$. Later, when the adversary gives a back to the reduction, then the reduction can retrieve x (and then retrieve b) by computing $x := f_1^{-1}(a)$. This technique allows the reduction to “forget” some information and thus is very useful for memory-tight reductions.

3 Our New Model for Authenticated Key Exchange

We propose a new security model for two-message AKE that captures the same security goals as in other models [31,40,41], but in a memory-efficient manner. Formally, we prove in Supp. Mat. C that a protocol secure in our model is also secure in the previous ones.

The current AKE models usually store a large amount of information to simulate the security game with adversaries. Some of them are not meaningful in the sense that they cannot give any additional advantage for the adversary to attack the protocol. As a consequence, it makes the security reduction unnecessarily memory-inefficient. Hence, we revisit the security model in a memory-efficient manner. Our syntax is defined (in Section 3.1) in a similar way as the previous work, and our new security definitions (in Section 3.2) are the major contribution.

3.1 Syntax

A two-message AKE protocol $\text{AKE} := (\text{Setup}, \text{KG}, \text{Init}_I, \text{Der}_R, \text{Der}_I)$ consists of five algorithms as illustrated in Figure 2. The setup algorithm Setup takes system security parameter 1^λ as input and outputs an AKE protocol parameter par . On input par , the key generation algorithm KG outputs an AKE long-term key pair (pk, sk) . Since 1^λ and par are public and fixed after setting up, for simplicity, we ignore the inputs of Setup and KG and write $\text{par} \leftarrow \text{Setup}$ and $(\text{pk}, \text{sk}) \leftarrow \text{KG}(\text{par})$.

Let u, v be two users with long-term key pairs $(\text{pk}_u, \text{sk}_u), (\text{pk}_v, \text{sk}_v)$, respectively. To establish a shared session key, u, v perform the following steps:

- To initiate a session, u firstly runs the initialization algorithm Init_I which takes $(\text{pk}_u, \text{sk}_u, \text{pk}_v)$ as inputs and outputs an initiator message M_u and session state st . Then u sends M_u to v . The state st is stored locally by u and will be used to compute a shared key later.
- On receiving M_u from the user u , v runs the responder derivation algorithm Der_R . On input $(\text{pk}_v, \text{sk}_v, \text{pk}_u, M_u)$, this algorithm generates and returns a responder message M_v and a session key K_v . To continue the session, v sends M_v to u . K_v can be $\perp$ (rejection symbol) which means that the received message M_u is invalid.
- Finally, on receiving M_v , u restores the state st generated before and runs the client derivation algorithm Der_I which inputs $(\text{pk}_u, \text{sk}_u, \text{pk}_v, \text{st}, M_v)$ and outputs a session key K_u . Similarly, K_u can be $\perp$.

Game ind-wFS-KCI_{II}^A 00 $b \leftarrow \{0, 1\}$, $\text{par} \leftarrow \text{Setup}$ 01 $O_1 := (\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R)$ 02 $O_2 := (\text{PK}, \text{COR}, \text{REV}, \text{TEST})$ 03 $b' \leftarrow \mathcal{A}^{O_1, O_2}(\text{par})$ 04 for $\text{sID}^* \in \mathcal{S}_{\text{test}}$: 05 if $\text{Fresh}(\text{sID}^*) = 0$: return b 06 if $\text{Valid}(\text{sID}^*) = 0$: return b 07 return $\llbracket b' = b \rrbracket$ Oracle SESSION_I(u, v) 08 $(\text{pk}_u, \text{sk}_u) \leftarrow \text{GetKey}(u)$ 09 $\text{pk}_v \leftarrow \text{PK}(v)$ 10 $(M_u, \text{st}) \leftarrow \text{Init}_I(\text{pk}_u, \text{sk}_u, \text{pk}_v)$ 11 $\text{seq} \leftarrow \text{SEQ} \setminus \mathcal{S}_{\text{seq}}$ 12 $\mathcal{S}_{\text{seq}} := \mathcal{S}_{\text{seq}} \cup \{\text{seq}\}$ 13 $\mathcal{L}_{\text{st}}[\text{seq}] := (u, v, M_u, \text{st})$ 14 $\mathcal{S}_{\text{msg}} := \mathcal{S}_{\text{msg}} \cup \{(u, M_u)\}$ 15 return (seq, M_u) Oracle SESSION_R(u, v, M_u) 16 $(\text{pk}_u, \text{sk}_u) \leftarrow \text{GetKey}(u)$ 17 $\text{pk}_v \leftarrow \text{PK}(v)$ 18 $(M_v, K) \leftarrow \text{Der}_R(\text{pk}_u, \text{sk}_v, \text{pk}_u, M_u)$ 19 if $K = \perp$: return $\perp$ 20 if $(u, M_u) \in \mathcal{S}_{\text{msg}}$ 21 $\mathcal{S}_{\text{msg}} := \mathcal{S}_{\text{msg}} \cup \{(u, v, M_u, M_v)\}$ 22 $\text{seq} \leftarrow \text{SEQ} \setminus \mathcal{S}_{\text{seq}}$ 23 $\mathcal{S}_{\text{seq}} := \mathcal{S}_{\text{seq}} \cup \{\text{seq}\}$ 24 $\text{sID} := (\text{seq}, u, v, M_u, M_v)$ 25 $\mathcal{S}_{\text{sid}} := \mathcal{S}_{\text{sid}} \cup \{\text{sID}\}$ 26 $\mathcal{L}_{\text{SK}}[\text{sID}] := K$ 27 return sID	Oracle DER_I(u, v, seq_1, M_v) 28 if $\text{seq}_1 \notin \mathcal{S}_{\text{seq}}$: return $\perp$ 29 $(\text{pk}_u, \text{sk}_u) \leftarrow \text{GetKey}(u)$ 30 $\text{pk}_v \leftarrow \text{PK}(v)$ 31 $(u', v', \text{st}, M_u) := \mathcal{L}_{\text{st}}[\text{seq}_1]$ 32 if $(u', v') \neq (u, v) \vee \text{st} = \perp$ 33 return $\perp$ 34 $K := \text{Der}_I(\text{pk}_u, \text{sk}_u, \text{pk}_v, \text{st}, M_v)$ 35 if $K = \perp$: return $\perp$ 36 if $\mathcal{L}_{\text{seq}}[\text{seq}_1, u, v, M_v] = \perp$ 37 $\text{seq}_2 \leftarrow \text{SEQ} \setminus \mathcal{S}_{\text{seq}}$ 38 $\mathcal{S}_{\text{seq}} := \mathcal{S}_{\text{seq}} \cup \{\text{seq}_2\}$ 39 $\mathcal{L}_{\text{seq}}[\text{seq}_1, u, v, M_v] := \text{seq}_2$ 40 $\text{seq}_2 := \mathcal{L}_{\text{seq}}[\text{seq}_1, u, v, M_v]$ 41 $\text{sID} := (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$ 42 $\mathcal{S}_{\text{sid}} := \mathcal{S}_{\text{sid}} \cup \{\text{sID}\}$ 43 $\mathcal{L}_{\text{SK}}[\text{sID}] := K$ 44 return sID Oracle PK($u \in \mathcal{U}$) 45 $(\text{pk}_u, \text{sk}_u) \leftarrow \text{GetKey}(u)$ 46 return pk_u Oracle COR_I(u) 47 $\mathcal{S}_{\text{cor}} := \mathcal{S}_{\text{cor}} \cup \{u\}$ 48 $(\text{pk}_u, \text{sk}_u) \leftarrow \text{GetKey}(u)$ 49 return sk_u Oracle REV(sID) 50 $\mathcal{S}_{\text{rev}} := \mathcal{S}_{\text{rev}} \cup \{\text{sID}\}$ 51 return $\mathcal{L}_{\text{SK}}[\text{sID}]$ Alg GetKey(u) 52 if $\mathcal{L}_{\text{key}}[u] = \perp$ 53 $\mathcal{L}_{\text{key}}[u] := (\text{pk}, \text{sk}) \leftarrow \text{KG}$ 54 return $\mathcal{L}_{\text{key}}[u]$	Oracle TEST(sID) 55 if $\mathcal{L}_{\text{SK}}[\text{sID}] = \perp$: return $\perp$ 56 $\mathcal{S}_{\text{test}} := \mathcal{S}_{\text{test}} \cup \{\text{sID}\}$ 57 parse $(*, u, v, M_u, M_v) := \text{sID}$ 58 if $\mathcal{L}_{\text{test}}[u, v, M_u, M_v] = \perp$ 59 $\text{SK}_0 := \mathcal{L}_{\text{SK}}[\text{sID}]$, $\text{SK}_1 \leftarrow \mathcal{SK}$ 60 $\mathcal{L}_{\text{test}}[u, v, M_u, M_v] := \text{SK}_0$ 61 return $\mathcal{L}_{\text{test}}[u, v, M_u, M_v]$ Alg ComputeMS(sID) 62 parse $(*, u, v, M_u, M_v) := \text{sID}$ 63 for $\text{sID}' \in \mathcal{S}_{\text{sid}} \setminus \{\text{sID}\}$ 64 parse $(*, u', v', M'_u, M'_v) := \text{sID}'$ 65 if $(u, v, M_u, M_v) = (u', v', M'_u, M'_v)$ 66 $\mathfrak{M}(\text{sID}) := \mathfrak{M}(\text{sID}) \cup \{\text{sID}'\}$ 67 return $\mathfrak{M}(\text{sID})$ Alg Fresh(sID) 68 $\mathfrak{M}(\text{sID}) := \text{ComputeMS}(\text{sID})$ 69 if $ \mathfrak{M}(\text{sID}) > 1$: return 1 70 if $\text{sID} \in \mathcal{S}_{\text{rev}}$: return 0 71 for $\text{sID}' \in \mathfrak{M}(\text{sID})$: 72 if $\text{sID}' \in \mathcal{S}_{\text{rev}}$: return 0 73 return 1 Alg Valid(sID) 74 $\mathfrak{M}(\text{sID}) := \text{ComputeMS}(\text{sID})$ 75 if $ \mathfrak{M}(\text{sID}) > 1$: return 1 76 if $\text{sID} = (\text{seq}, u, v, M_u, M_v)$ 77 // Responder session 78 if $u \notin \mathcal{S}_{\text{cor}} \vee (u, M_u) \in \mathcal{S}_{\text{msg}}$ 79 return 1 80 if $\text{sID} = (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$ 81 // Initiator session 82 if $v \notin \mathcal{S}_{\text{cor}}$ 83 $\vee (u, v, M_u, M_v) \in \mathcal{S}_{\text{msg}}$ 84 return 1 85 return 0
--	---	--

Fig. 3. Game ind-wFS-KCI. For simplicity, we use procedure GetKey to generate and store a user's long-term key pair, and use procedures ComputeMS, Fresh, and Valid to check if a session is fresh or is valid (cf. Definition 5). The adversary cannot access these procedures.

We define the correctness of AKE protocols in Definition 4 which states that an honestly execution between users u and v as in Figure 2 should produce the same session key $K_u = K_v$.

Definition 4 (AKE Correctness). *Let $\text{AKE} := (\text{Setup}, \text{KG}, \text{Init}_I, \text{Der}_R, \text{Der}_I)$ be an AKE protocol and u and v be two users. We say AKE is ρ -correct if*

$$\Pr \left[\begin{array}{l} \text{par} \leftarrow \text{Setup} \\ (\text{pk}_u, \text{sk}_u) \leftarrow \text{KG}, (\text{pk}_v, \text{sk}_v) \leftarrow \text{KG} \\ (M_u, \text{st}) \leftarrow \text{Init}_I(\text{pk}_u, \text{sk}_u, \text{pk}_v) \\ (M_v, K_v) \leftarrow \text{Der}_R(\text{pk}_v, \text{sk}_v, \text{pk}_u, M_u) \\ K_u \leftarrow \text{Der}_I(\text{pk}_u, \text{sk}_u, \text{pk}_v, \text{st}, M_v) \end{array} \right] \geq \rho,$$

where the probability is taken over the randomness of Setup, KG, Init_I, Der_R, and Der_I. ρ is some function related to the security parameter λ (e.g., $\rho = 1 - \text{negl}(\lambda)$).

3.2 Security for AKE

Our security game is in Figure 3. We explain how the useful variables are defined.

NAMING USERS AND GENERATING PK. Our model considers a set of user identifiers, $\mathcal{U}$. Each user in our model has a unique identifier from $\mathcal{U}$. For simplicity, we use u to indicate a user with identifier $u \in \mathcal{U}$. The definition of $\mathcal{U}$ depends on the concrete real-world implementation of AKE protocols. Usually, we model $\mathcal{U}$ as a set of fixed-length bit-string (namely, $\mathcal{U} = \{0, 1\}^c$ for some fixed constant c). For more general case $\mathcal{U} := \{0, 1\}^*$, we will handle it as in [30, Section 2.6].

Let $\Pi = (\text{Setup}, \text{KG}, \text{Init}_I, \text{Der}_R, \text{Der}_I)$ be an AKE protocol and $\text{par} \leftarrow \text{Setup}$ be the protocol parameter. The long-term key pair of a user u is denoted as $(\text{pk}_u, \text{sk}_u)$. In this model, the adversary $\mathcal{A}$ can access an oracle PK to learn the long-term public key of any user. When $\mathcal{A}$ queries $\text{PK}(u \in \mathcal{U})$, the oracle returns

pk_u . If $\mathcal{A}$ queries PK repeatedly on the same user identifier, then the same public key is returned. This method of modelling public key generation is proposed by [20,30]. It allows security reductions to simulate all public key in a memory-tight manner and does not change the adversary's capabilities.

MODELING SESSIONS. $\mathcal{A}$ has oracle access to SESSION_I , SESSION_R , and DER_I . This is to capture the adversary's ability to control the transmission of all protocol messages and launch active attacks. Every query to SESSION_I and SESSION_R will produce a unique sequence number from a sequence number pool SEQ . A query to DER_I will produce a unique sequence number which is determined by the query, namely, the same queries to DER_I will produce the same sequence numbers. More precisely,

- SESSION_I allows $\mathcal{A}$ to initiate a session between u (as an initiator and owner) and v (as a responder). When $\mathcal{A}$ queries $\text{SESSION}_I(u, v)$, the oracle returns (seq, M_u) where M_u is u 's initiator protocol message (generated by running Init_I) and $\text{seq} \in \text{SEQ}$ is a uniformly random sequence number.
- SESSION_R allows $\mathcal{A}$ to send an arbitrary message M to v , pretending u . This creates a session owned by v . When $\mathcal{A}$ queries $\text{SESSION}_I(u, v, M)$, the oracle runs Der_R to generate a responder message M_v and a session key K . If $K = \perp$, then the oracle simply returns $\perp$; Otherwise, the oracle generates a unique sequence number seq uniformly at random and define a session identifier sID for this session. Finally, SESSION_R returns (M_v, sID) . $\mathcal{A}$ can use the identifier sID to test or reveal this session later.
- DER_I allows $\mathcal{A}$ to complete an initiator session by sending a message. When $\mathcal{A}$ queries $\text{DER}_I(u, v, \text{seq}_1, v, M)$, the oracle restores session state information st related to seq_1 . If such seq_1 is not generated from previous SESSION_I queries, then the oracle simply returns $\perp$; Otherwise, it runs Der_I to generate a session key K . Similar to SESSION_R , if K is $\perp$, then the oracle returns $\perp$. Otherwise, the oracle generates a sequence number seq_2 related to this query, defines a session identifier sID for this session, and returns sID .

For the above oracles, we use the following lists and sets to keep track of some necessary internal information (which an adversary cannot directly access). All lists and sets are initially set as empty:

- Lists $\mathcal{L}_{\text{key}}$ and $\mathcal{L}_{\text{SK}}$ are used to store user long-term keys and session keys, respectively.
 - List $\mathcal{L}_{\text{st}}$ is used to store session states of initiator sessions and the identifiers of intended responders.
 - List $\mathcal{L}_{\text{seq}}$ is used to store the sequence numbers generated by Der_I . By using this list, the model makes sure that the same query to Der_I will produce the same session final sequence number seq_2 (cf. lines 36 to 39).
 - Sets $\mathcal{S}_{\text{seq}}$ and $\mathcal{S}_{\text{msg}}$ are used to store session sequence numbers and protocol messages (with the identifier of the user that generates this message) generated by oracles, respectively.
- More specifically, $(u, M_u) \in \mathcal{S}_{\text{msg}}$ means that $\mathcal{A}$ has queried $\text{SESSION}_I(u, v)$ before and the query produce the protocol message M_u . If $\mathcal{A}$ later sends this (u, M_u) to some v (via querying SESSION_R), then we record $(u, v, M_u, M_v) \in \mathcal{S}_{\text{msg}}$ to help us determine if there exists a matching session, where M_v is the responding message produced by the party v that responds M_u .
- Set $\mathcal{S}_{\text{sID}}$ stores all session identifiers. This set is used when we compute the number of matching sessions. We have $\text{sID} \in \mathcal{S}_{\text{sID}} \iff \mathcal{L}_{\text{SK}}[\text{sID}] \neq \perp$.

DEFINING SESSION IDENTIFIERS. For an initiator session owned by a party u , if its intended peer is v , sent message is M_u , and received message is M_v , then its identifier is of the format $\text{sID} := (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$, where seq_1 and seq_2 are the initial and final sequence numbers generated by SESSION_I and DER_I . Similarly, the identifier of a responder session is defined as $\text{sID} := (\text{seq}, u, v, M_u, M_v)$, where seq is the sequence number generated by SESSION_R .

Let sID and sID' be an initiator and responder session, respectively. We say sID and sID' are matching sessions, if their user identifiers of initiator and responder and the protocol messages sent and received are the same. Namely, $\text{sID} = (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$ matches $\text{sID}' = (\text{seq}', u', v', M'_u, M'_v)$ if $(u, v, M_u, M_v) = (u', v', M'_u, M'_v)$.

MODELLING ATTACKS AND SECURITY. We define *freshness* and *validity* to exclude all trivial attacks. In this model, we use auxiliary procedurs **Fresh** and **Valid** (cf. Figure 3) to check whether a session is fresh and valid, respectively.

For a full understanding of the attacks defined in our model, please refer to Supp. Mat. B. In short, our model captures same attacks defined in [31,40,41], including replay attacks, man-in-the-middle (MiM) attacks, key-compromise impersonation (KCI) attacks, and weak forward-secrecy (wFS). A difference is that we do not consider partially matching session in this model (cf. Remark 2).

Alg Setup <pre> 00 $\text{par}_{\text{kem}} \leftarrow \text{KEM.Setup}$ 01 $\text{par}_{\text{sig}} \leftarrow \text{Sign.Setup}$ 02 return $\text{par} := (\text{par}_{\text{kem}}, \text{par}_{\text{sig}})$ Alg KG($\text{par}_{\text{kem}}, \text{par}_{\text{sig}}$) 03 $(\text{pk}, \text{sk}) \leftarrow \text{SIG.KG}(\text{par}_{\text{sig}})$ 04 return (pk, sk) Alg Init_I($\text{pk}_u, \text{sk}_u, \text{pk}_v$) 05 $\alpha \leftarrow \{0, 1\}^\ell$, $(\text{epk}, \text{esk}) \leftarrow \text{KEM.KG}$ 06 $\sigma_1 \leftarrow \text{Sign}(\text{sk}_u, (\alpha, \text{epk}))$ 07 $M_u := (\alpha, (\text{epk}, \sigma_1))$, $\text{st} := (\text{esk}, M_u)$ 08 return (st, M_u) </pre>	Alg Der_R($\text{pk}_v, \text{sk}_v, \text{pk}_u, (\alpha, \text{epk}, \sigma_1)$) <pre> 09 if $\text{Vrfy}(\text{pk}_u, (\alpha, \text{epk}), \sigma_1) = 0$ 10 return $\perp$ 11 $\beta \leftarrow \{0, 1\}^\ell$, $(k, c) \leftarrow \text{Encaps}(\text{epk})$ 12 $\sigma_2 \leftarrow \text{Sign}(\text{sk}_v, (\alpha, \beta, \text{epk}, c))$ 13 $K := \text{H}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$ 14 $M_v := (\beta, c, \sigma_2)$ 15 return (M_v, K) Alg Der_I($\text{pk}_u, \text{sk}_u, \text{pk}_v, \text{st}, (\beta, c, \sigma_2)$) 16 $(\text{esk}, \alpha, \text{epk}, \sigma_1) := \text{st}$ 17 if $\text{Vrfy}(\text{pk}_v, (\alpha, \beta, \text{epk}, c), \sigma_2) = 0$ 18 return $\perp$ 19 $k := \text{Decaps}(\text{esk}, c)$ 20 if $k = \perp$: return $\perp$ 21 $K := \text{H}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$ 22 return K </pre>
--	---

Fig. 4. Our AKE protocol aSigKEM.

Definition 5 (Freshness and Validity). A session sID is fresh if (1) $\mathcal{L}_{\text{SK}}[\text{sID}] \neq \perp$, namely, sID is complete and has derived a session key, (2) $\text{sID} \notin \mathcal{S}_{\text{rev}}$, and (3) for any $\text{sID}' \in \mathfrak{M}(\text{sID})$, we have $\text{sID}' \notin \mathcal{S}_{\text{rev}}$, where $\mathfrak{M}(\text{sID})$ stores all the matching sessions of sID .

Furthermore, a session is valid if at least one of the following conditions holds true: (1) The session has more than one matching sessions. (2) If the session is a responder session, then either the initiator in the session is uncorrupted or the initiator message of this session is generated from SESSION_I . (3) If this session is an initiator session, then either the responder in this session is uncorrupted or this session has a matching session.

Following the previous indistinguishability(IND)-based AKE model [31,40], our model is the multi-TEST model with a single random bit b . Let sID be an arbitrary established session, $\text{TEST}(\text{sID})$ returns the real session key of sID if $b = 0$ and returns an independently and uniformly random key if $b = 1$. To deal with the case that $\mathcal{A}$ issues repeating queries to TEST or tests both a session and its matching session, we use a list $\mathcal{L}_{\text{test}}$ to record the returning keys of TEST (cf. lines 57 to 60). The formal security definition of our AKE model is given below.

Definition 6 (Key Indistinguishability of AKE). Let $\Pi := (\text{Setup}, \text{KG}, \text{Init}_I, \text{Der}_R, \text{Der}_I)$ be an AKE protocol and consider the game ind-wFS-KCI defined in Figure 3. We say AKE is secure if, for all PPT adversaries $\mathcal{A}$, the following advantage is negligible on the security parameter λ :

$$\text{Adv}_{\Pi}^{\text{ind-wFS-KCI}}(\mathcal{A}) := \left| \Pr \left[\text{ind-wFS-KCI}_{\Pi}^{\mathcal{A}} \Rightarrow 1 \right] - 1/2 \right|$$

4 Our Generic AKE from KEM and Signature

Our new AKE protocol is an abstraction of the well-known signed Diffie-Hellman key exchange protocol. We abstract the DH part with a KEM. Motivated by the argumented-random-string technique in [30], our protocol includes a random nonce in each move, and thus we call it argumented Signed KEM (aSigKEM) protocol.

CONSTRUCTION. Let $\text{KEM} := (\text{Setup}, \text{KG}, \text{Encaps}, \text{Decaps})$ be a KEM scheme and $\text{SIG} := (\text{Setup}, \text{KG}, \text{Sign}, \text{Vrfy})$ be a signature scheme. Let $\{0, 1\}^\ell$ be a nonce space, $\mathcal{SK}$ be a session key space, and $\text{H} : \{0, 1\}^* \rightarrow \mathcal{SK}$ be a hash function. Our protocol aSigKEM is defined in Figure 4.

Theorem 1. For any adversary $\mathcal{A}$ against aSigKEM as in our model ind-wFS-KCI with user identifier set $\mathcal{U}$ and session sequence number set SEQ (cf. Figure 3), if $\mathcal{A}$ issues Q_{pk} , Q_{rev} , Q_{cor} , Q_{test} , Q_{H} , and Q_{send} queries to oracles PK (including direct and indirect queries, i.e., queries from reductions), REV, COR, TEST, H, and $(\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R)$ then there exist adversaries $\mathcal{B}_1$ and $\mathcal{B}_2$ such that

$$\begin{aligned} \text{Adv}_{\text{aSigKEM}}^{\text{ind-wFS-KCI}}(\mathcal{A}) &\leq \Pr[\text{OW-PCA}_{\text{KEM}}^{\mathcal{B}_2} \Rightarrow 1] + \Pr[\text{MU}^c\text{-sEUF-1CMA}_{\text{SIG}}^{\mathcal{B}_1} \Rightarrow 1] \\ &\quad + N_{\text{user}}^2 \eta_{\text{pk}} + N_{\text{ses}}^2 \left(\frac{1}{2^{\gamma_{\text{pk}}-3}} + \frac{1}{2^{\gamma_{\text{ct}}-3}} + \frac{1}{2^{\ell-2}} \right) + 3N_{\text{ses}} \cdot \delta_{\text{KEM}} + \frac{2(Q_{\text{H}} + N_{\text{ses}})^2}{|\mathcal{SK}|} \end{aligned}$$

$$\begin{aligned}
& + \frac{3(Q_{\text{rev}} + Q_{\text{test}} + N_{\text{ses}} + N_{\text{ses}}^2)}{|\text{SEQ}|} + \frac{3(Q_{\text{H}} + N_{\text{ses}})^2}{|\mathcal{K}|} + \frac{14N_{\text{ses}}(N_{\text{ses}} + 1)^2}{|\mathcal{U}|N}, \\
\mathbf{Mem}^*(\mathcal{B}_2) & \approx \mathbf{Mem}^*(\mathcal{B}_1) = \mathbf{Mem}(\mathcal{A}) + O(N_{\text{user}} \log |\mathcal{U}|) + O(\ell) \\
& + O(\log |\text{SEQ}|) + O(\log N_{\text{ses}}) + \mathbf{Mem}(\text{aSigKEM}), \\
\mathbf{Time}^*(\mathcal{B}_2) & \approx \mathbf{Time}^*(\mathcal{B}_1) = \mathbf{Time}(\mathcal{A}) + O(N_{\text{ses}} + Q_{\text{H}}) \cdot \mathbf{Time}(\text{aSigKEM}),
\end{aligned}$$

where η_{pk} is the probability that two honestly-generated signing key pairs collide, ℓ is the length of nonces, δ_{KEM} is the decryption error bound of KEM, γ_{pk} and γ_{ct} are the min-entropy of honestly-generated public key and ciphertext of KEM, respectively, $N_{\text{user}} (\leq Q_{pk})$ and $N_{\text{ses}} (\leq Q_{\text{send}})$ are the numbers of different long-term key pairs and sessions established by $\mathcal{A}$, respectively, and N is an integer with $|\mathcal{U}|^2 \cdot N \leq 2^\ell$. $\mathbf{Time}(\text{aSigKEM})$ and $\mathbf{Mem}(\text{aSigKEM})$ are the required running time and memory usage of running a complete local execution of aSigKEM, respectively. We assume that $2^{\gamma_{pk}} > 2N_{\text{ses}}$, $|\mathcal{U}| \geq N_{\text{ses}} + 1$, $N \geq N_{\text{ses}} + 1$, and $|\mathcal{K}| \geq Q_{\text{H}} + N_{\text{ses}}$.

For a large-scale application, we can set up that the set of user identifiers is $\mathcal{U} = \{0, 1\}^{128}$ and the set of session sequence number is $\text{SEQ} = \{0, 1\}^{512}$. If we consider a setting with $(N_{\text{user}}, N_{\text{ses}}) \approx (2^{32}, 2^{64})$, then we can choose $\ell = 384$ (which means that the nonces α and β are 48-byte strings) and $N = 2^{128}$.

INTERPRETING BOUNDS IN THEOREM 1. Essentially, our bounds are TA-tight wrt the advantage of the KEM and signature schemes, but with an additive term $O(N_{\text{user}})$ as memory loss. This is significantly better than other AKE protocols [23, 31, 37, 40, 16, 29, 41] in terms of memory-tightness, as we remove the memory-loss factors $O(N_{\text{ses}})$ and $O(Q_{\text{H}})$ in Equations (1) and (2). For more details, we say $\mathbf{Time}(\text{aSigKEM}) = \text{poly}(\lambda)$ and $\mathbf{Mem}(\text{aSigKEM}) = O(\lambda)$ and have our memory bound as

$$\mathbf{Mem}^*(\mathcal{B}_1) \approx \mathbf{Mem}^*(\mathcal{B}_2) \leq \mathbf{Mem}(\mathcal{A}) + O(N_{\text{user}} \cdot \lambda)$$

Remark 1 (On the linear memory loss in terms of the number of users.) Our reduction achieves session-tightness (i.e., the memory usage only depends logarithmically on the number of sessions), but its memory bound has an additive term proportional to the number of users, and thus it is not memory-tight w.r.t. the number of users. This term comes from the need of recording corrupted users when constructing reductions from the unforgeability of SIG and the OW-PCA security of KEM.

It seems unavoidable to record these corrupted users. In most AKE models (including ours), checking whether a user has been corrupted is essential to capture security as forward secrecy, key compromise impersonation attacks, and check session validity. If such information is not recorded, then we cannot determine the validity of a given session, and thus we cannot determine whether an adversary wins by trivial attacks.

To provide more details, we sketch a simple argument on why this is unavoidable. Deciding whether a user has been corrupted can be understood as the following *dictionary* problem: Given a set $S \subset \{1, \dots, M\}$ and $|S| = N$, decide if a given x is in S , where S can be adversarial, namely, given by an adversary and not known to the algorithm in advance. Any algorithm solving this problem without error needs to store the whole set S , otherwise one can ask an element that contained in S but not recorded to make the algorithm output a wrong answer. Since there are $\binom{M}{N}$ options for S , Pagh [36] shows that storing such a set requires at least $\log(\binom{M}{N})$ bits and

$$\log\left(\binom{M}{N}\right) = N \log\left(e \frac{M}{N}\right) - \Theta\left(\frac{N^2}{M}\right) - O(\log N).$$

In our AKE proof, $M = |\mathcal{U}| = 2^{O(\lambda)}$ corresponds to the size of the space of user identifiers and $N = N_{\text{user}}$ the number of users. Hence, M is much larger than N , and we have

$$\Omega(\log(\binom{M}{N})) \approx \Omega(N \log M) \approx \Omega(N_{\text{user}} \cdot \lambda),$$

which is about the memory bound we get in Theorem 1.

An alternative approach is rewinding, namely, running the adversary with the same randomness twice to check if a user has been corrupted, but then it leads to non-tightness on the running time.

Game G_0-G_3		Oracle $\text{TEST}(\text{slD})$	
23 $\text{cnt} := 0$	// G_2 - G_3	63 if $\mathcal{L}_{\text{SK}}[\text{slD}] = \perp$: return $\perp$	
24 $f_{\text{skg}} \leftarrow \text{Func}(\mathcal{U}, \mathcal{R}_k)$	// G_2 - G_3	64 $\mathcal{S}_{\text{test}} := \mathcal{S}_{\text{test}} \cup \{\text{slD}\}$	
25 $f_{\text{sl}} \leftarrow \text{Func}(\{0, 1\}^\ell \times \mathcal{PK}, \mathcal{R}_s)$	// G_2 - G_3	65 parse $(*, u, v, M_u, M_v) := \text{slD}$	
26 $f_{\text{sr}} \leftarrow \text{Func}(\{0, 1\}^\ell \times \mathcal{PK} \times \mathcal{C}, \mathcal{R}_s)$	// G_2 - G_3	66 $\text{SK}_0 := \mathcal{L}_{\text{SK}}[\text{slD}], \text{SK}_1 := f_{\text{SK}}(u, v, M_u, M_v)$	
27 $f_{\text{seq}} \leftarrow \text{Func}(\{0, 1\}^*, \mathcal{SEQ})$	// G_2 - G_3	67 return SK_b	
28 $f_{\text{ekg}} \leftarrow \text{Func}(N, \mathcal{R}_{\text{ekg}})$	// G_2 - G_3	Oracle $\text{SESSION}_R(u, v, M_u = (\alpha, \text{epk}, \sigma_1))$	
29 $f_{\text{enc}} \leftarrow \text{Func}(N \times \mathcal{PK}, \mathcal{R}_{\text{enc}})$	// G_2 - G_3	68 if $\text{Vrfy}(\text{pk}_u, (\alpha, \text{epk}), \sigma_1) = 0$: return $\perp$	
30 $h_{\text{ncc}} \leftarrow \text{InjF}(\mathcal{U}^2 \times [N], \{0, 1\}^\ell)$	// G_3	69 $(c, k) \leftarrow \text{KEM}.\text{Encaps}(\text{epk})$	// G_0 - G_1
31 $h_{\text{seq}} \leftarrow \text{InjF}(\mathcal{U}^2 \times [N], \{0, 1\}^\ell)$	// G_3	70 $\text{cnt} := \text{cnt} + 1$	// G_2 - G_3
32 $f_{\text{H}}, f_{\text{SK}} \leftarrow \text{Func}(\{0, 1\}^*, \mathcal{SK})$		71 $y := f_{\text{enc}}(\text{epk}, \text{cnt})$	// G_2 - G_3
33 $b \leftarrow \{0, 1\}$, $\text{par} \leftarrow \text{Setup}$		72 $(c, k) := \text{KEM}.\text{Encaps}(\text{epk}; y)$	// G_2 - G_3
34 $O_1 := (\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R)$		73 $\beta \leftarrow \{0, 1\}^\ell$	// G_0 - G_2
35 $O_2 := (\text{PK}, \text{COR}, \text{REV}, \text{TEST})$		74 $\beta := h_{\text{ncc}}(u, v, \text{cnt})$	// G_3
36 $b' \leftarrow \mathcal{A}^{O_1, O_2, H}(\text{par})$		75 $\sigma_2 \leftarrow \text{Sign}(\text{sk}_v, (\alpha, \beta, \text{epk}, c))$	// G_0 - G_1
37 for $\text{slD}^* \in \mathcal{S}_{\text{test}}$:		76 $r_{\text{sign}} := f_{\text{sr}}(\alpha, \beta, \text{epk}, c)$	// G_2 - G_3
38 if $\text{Fresh}(\text{slD}^*) = 0 \vee \text{Valid}(\text{slD}^*) = 0$		77 $\sigma_2 := \text{Sign}(\text{sk}_v, (\alpha, \beta, \text{epk}, c); r_{\text{sign}})$	// G_2 - G_3
39 return b		78 $K := H(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$	
40 return $\llbracket b' = b \rrbracket$		79 $M_v := (\beta, c, \sigma_2)$	
Alg $\text{GetKey}(u)$		80 if $(u, M_u) \in \mathcal{S}_{\text{msg}}$	
41 if $\mathcal{L}_{\text{key}}[u] = \perp$	// G_0 - G_3	81 $\mathcal{S}_{\text{msg}} := \mathcal{S}_{\text{msg}} \cup \{(u, v, M_u, M_v)\}$	
42 $\mathcal{L}_{\text{key}}[u] := (\text{pk}_u, \text{sk}_u) \leftarrow \text{SIG}.\text{KG}$	// G_0 - G_3	82 $\text{seq} \leftarrow \mathcal{SEQ} \setminus \mathcal{S}_{\text{seq}}$	// G_0
43 return $\mathcal{L}_{\text{key}}[u]$	// G_0 - G_3	83 $\mathcal{S}_{\text{seq}} := \mathcal{S}_{\text{seq}} \cup \{\text{seq}\}$	// G_0
44 $(\text{pk}_u, \text{sk}_u) := \text{SIG}.\text{KG}(f_{\text{skg}}(u))$	// G_2 - G_3	84 $\text{seq} \leftarrow \mathcal{SEQ}$	// G_1
45 return $(\text{pk}_u, \text{sk}_u)$	// G_2 - G_3	85 $\text{seq} := f_{\text{seq}}(u, v, M_u, M_v)$	// G_2 - G_3
Oracle $\text{SESSION}_I(u, v)$		86 $\text{slD} := (\text{seq}, u, v, M_u, M_v)$	
46 $\text{cnt} := \text{cnt} + 1$	// G_2 - G_3	87 $\mathcal{L}_{\text{SK}}[\text{slD}] := K$	
47 $(\text{epk}, \text{esk}) \leftarrow \text{KEM}.\text{KG}$	// G_0 - G_1	88 return (M_v, slD)	
48 $(\text{epk}, \text{esk}) := \text{KEM}.\text{KG}(: f_{\text{ekg}}(\text{cnt}))$	// G_2 - G_3	Oracle $\text{DER}_I(u, v, \text{seq}_1, M_v = (\beta, c, \sigma_2))$	
49 $\alpha \leftarrow \{0, 1\}^\ell$	// G_0 - G_2	89 $(u', v', \text{st}) := \mathcal{L}_{\text{st}}[\text{seq}_1]$	
50 $\alpha := h_{\text{ncc}}(u, v, \text{cnt})$	// G_3	90 if $(u', v') \neq (u, v) \vee \text{st} = \perp$: return $\perp$	
51 $\sigma_1 \leftarrow \text{Sign}(\text{sk}_u, (\alpha, \text{epk}))$	// G_0 - G_1	91 $(\text{esk}, \alpha, \text{epk}, \sigma_1) := \text{st}$	
52 $r_{\text{sign}} := f_{\text{sr}}(\alpha, \text{epk})$	// G_2 - G_3	92 if $\text{Vrfy}(\text{pk}_v, (\alpha, \beta, \text{epk}, c), \sigma_2) = 0$	
53 $\sigma_1 := \text{Sign}(\text{sk}_u, (\alpha, \text{epk}); r_{\text{sign}})$	// G_2 - G_3	93 return $\perp$	
54 $\text{seq}_1 \leftarrow \mathcal{SEQ} \setminus \mathcal{S}_{\text{seq}}$	// G_0	94 $k := \text{Decaps}(\text{esk}, c)$	
55 $\mathcal{S}_{\text{seq}} := \mathcal{S}_{\text{seq}} \cup \{\text{seq}_1\}$	// G_0	95 $K := H(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$	
56 $\text{seq}_1 \leftarrow \mathcal{SEQ}$	// G_1 - G_2	96 if $\mathcal{L}_{\text{seq}}[\text{seq}_1, u, v, M_v] = \perp$	// G_0 - G_1
57 $\text{seq}_1 := h_{\text{seq}}(u, v, \text{cnt})$	// G_3	97 $\text{seq}_2 \leftarrow \mathcal{SEQ} \setminus \mathcal{S}_{\text{seq}}$	// G_0
58 $M_u := (\alpha, (\text{epk}, \sigma_1))$		98 $\mathcal{S}_{\text{seq}} := \mathcal{S}_{\text{seq}} \cup \{\text{seq}_2\}$	// G_0
59 $\text{st} := (\text{esk}, M_u)$, $\mathcal{L}_{\text{st}}[\text{seq}_1] := (u, v, \text{st})$		99 $\text{seq}_2 \leftarrow \mathcal{SEQ}$	// G_1
60 $\mathcal{S}_{\text{msg}} := \mathcal{S}_{\text{msg}} \cup \{(u, M_u)\}$		100 $\mathcal{L}_{\text{seq}}[\text{seq}_1, u, v, M_v] := \text{seq}_2$	// G_0 - G_1
61 return (seq_1, M_u)		101 $\text{seq}_2 := \mathcal{L}_{\text{seq}}[\text{seq}_1, u, v, M_v]$	// G_0 - G_1
Oracle $H(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$		102 $\text{seq}_2 := f_{\text{seq}}(\text{seq}_1, u, v, M_v)$	// G_2 - G_3
62 return $f_{\text{H}}(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$		103 $\text{slD} := (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$	
		104 $\mathcal{L}_{\text{SK}}[\text{slD}] := K$	
		105 return slD	

Fig. 5. Games G_0 - G_3 for proving Theorem 1. For simplicity, we assume that in $\text{SESSION}_I, \text{SESSION}_R, \text{DER}_I$, for any party U , we have $(\text{pk}_U, \text{sk}_U) = \text{GetKey}(U)$. All omitted oracles and algorithms are the same as those in Figure 3.

Proof (Theorem 1). Let $\mathcal{A}$ be an adversary against aSigKEM in the ind-wFS-KCI game. Our proof consists of a sequence of games as in Figures 5 to 7.

In our games, we exclude collisions in the outputs of $\text{SIG}.\text{KG}$. Namely, every user has a unique long-term key pair. If such a collision happens at any time, then we abort the game. For readability, we do not explicitly define such collision events in the codes of games sequences. Adding such a collision check introduce an additive term $N_{\text{user}}^2 \cdot \eta_{pk}$ to our final bound since there are N_{user} users in the game and the probability of key collision in SIG is η_{pk} .

Moreover, we also exclude collisions in the outputs of random oracle H and epk 's and c 's from SESSION_I and SESSION_R . This implies that different sessions have different protocol messages, and thus each session has at most one matching session (and also at most one partially matching session). Since the hash function H take the protocol messages as input, different sessions will also have different session keys. Excluding collisions add $\frac{(Q_H + N_{\text{ses}})^2}{2|\mathcal{SK}|} + \frac{N_{\text{ses}}^2}{2^{\gamma_{pk}-1}} + \frac{N_{\text{ses}}^2}{2^{\gamma_{ct}-1}}$ to our final bound, where γ_{pk} and γ_{ct} are the min-entropy of epk and c generated by $\text{KEM}.\text{KG}$ and $\text{KEM}.\text{Encaps}$, respectively.

Game G_0 . G_0 is shown in Figure 5. Compared to the original security game ind-wFS-KCI , the RO outputs of H in G_0 is generated by random function f_{H} (cf. line 62), and the random keys of TEST is generated by

random function f_{rsk} (cf. line 66). Since the distributions of $\mathbf{H}$ and TEST remain unchanged, we have

$$\Pr [\text{ind-wFS-KCI}_{\text{asigKEM}}^{\mathcal{A}} \Rightarrow 1] = \Pr [\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1]$$

Game $\mathbf{G}_1$ (Random sequence numbers). When simulating SESSION_I and SESSION_R , we sample seq from SEQ uniformly at random without recording $\mathcal{S}_{\text{seq}}$. Now a freshly generated sequence number may collide with some previously generated number. The adversary cannot detect this change as long as the sequence number set SEQ is large enough, namely,

$$|\Pr [\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr [\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]| \leq N_{\text{ses}}^2 / |\text{SEQ}|$$

Game $\mathbf{G}_2$ (Use RFs to derive randomness). In this game, we use random functions to generate randomness. Concretely, the sequence numbers in DER_I and SESSION_R are generated by random function f_{seq} (cf. lines 85 and 102), the randomnesses for the key generation are generated by f_{skg} , and the randomness for signing algorithm of SIG are generated by f_{si} and f_{sr} (cf. lines 44, 53 and 77). Moreover, we use a counter $\text{cnt} \leq N$ to compute the ephemeral KEM key pairs and encapsulated ciphertexts using random functions f_{ekg} and f_{enc} , respectively (cf. line 48 and lines 71 to 72). The counter increments by 1 every time $\mathcal{A}$ issues SESSION_I or SESSION_R queries (cf. lines 46 and 70). We will discuss how to choose the upper bound N later.

These modifications allow the game simulator to “forget” the lists $\mathcal{L}_{\text{key}}$ and $\mathcal{L}_{\text{seq}}$ and are conceptual, so we have $\Pr [\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] = \Pr [\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1]$.

Game $\mathbf{G}_3$ (Invertible nonces and sequence numbers). We use a random injective function h_{ncc} to generate the random nonces α 's and β 's (cf. lines 50 and 74) and use h_{seq} to generate sequence numbers of SESSION_I (cf. line 57). Since there are at most N_{ses} queries to SESSION_I and SESSION_R , by the switching lemma, we have

$$|\Pr [\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] - \Pr [\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1]| \leq N_{\text{ses}}^2 \cdot 2^{-\ell}$$

By the changes made in game sequence $\mathbf{G}_0$ - $\mathbf{G}_3$, the game simulator now can forget the lists $\mathcal{L}_{\text{seq}}$ and $\mathcal{L}_{\text{key}}$ and the set $\mathcal{S}_{\text{seq}}$. Moreover, by using injective random functions, we hide state information in all nonces and all initiator first sequence numbers (i.e., seq_1 's) in the game. This allows the game simulator to restore such state information and forget the lists $\mathcal{L}_{\text{st}}$ and $\mathcal{L}_{\text{sk}}$ and the set $\mathcal{S}_{\text{msg}}$ in the future games. For readability, we present the codes of $\mathbf{G}_3$ - $\mathbf{G}_6$ in Figure 6.

Game $\mathbf{G}_4$ (Simulate initiator states without recording). In this game, we restore initiator states $(\text{esk}, \alpha, \text{epk}, \sigma)$ by inverting the state information hidden in seq instead of using the list $\mathcal{L}_{\text{st}}$ (cf. lines 74 to 78).

More specifically, when $\mathcal{A}$ queries $\text{DER}_I(u, v, \text{seq}_1, \text{M}_v)$, we invert seq_1 and get (u', v', cnt') , and check if $u = u'$, $v = v'$, and $\text{cnt} < \text{cnt}'$. If so, then we re-compute esk and $(\alpha, \text{epk}, \sigma_1)$ (by following the lines 25 to 28).

Due to space limitations, we summarize the analysis of $\mathbf{G}_4$ as a lemma and postpone its proof to Supp. Mat. D.2.

Lemma 2. *With the notations and assumptions as stated in Theorem 1, we have*

$$|\Pr [\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] - \Pr [\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1]| \leq 2N_{\text{ses}}^2 \cdot (|\mathcal{U}|^2 N)^{-1}$$

We may choose N as large as possible to make $2N_{\text{ses}}^2 / |\mathcal{U}|^2 N$ negligible as long as the condition $|\mathcal{U}|^2 \cdot N \leq 2^\ell$ always holds true (so that h_{seq} is injective).

Game $\mathbf{G}_5$ (Check if an sID exists without recording). We define an algorithm CheckSid (in Figure 6) to check if a session identifier sID exists. This game is a preparatory step for forgetting the list $\mathcal{L}_{\text{sk}}$. Concretely, when $\mathcal{A}$ queries $\text{TEST}(\text{sID})$ or $\text{REV}(\text{sID})$, we use CheckSid to check if the session identifier sID exists, namely, to check if sID is generated by DER_I or SESSION_R and possesses a valid session key.

To distinguish $\mathbf{G}_5$ from $\mathbf{G}_4$, $\mathcal{A}$ needs to forge a session identifier sID that does not exist, namely, sID is not output from SESSION_R or DER_I , but it makes CheckSid return 1. And to forge such sID , $\mathcal{A}$ has to compute the correct sequence numbers seq (for responder sessions) or $(\text{seq}_1, \text{seq}_2)$ (for initiator sessions) of sID .

Game G_3-G_6		Oracle $H(pk_1, pk_2, \alpha, \beta, epk, c, \sigma_1, \sigma_2, k)$	
00 $cnt := 0, b \leftarrow \{0, 1\}, par \leftarrow \text{Setup}$		49 $k^* := k$	
01 Initialize $f_{seq}, f_H, f_{skg}, f_{sI}, f_{sR}, f_{ekg}, f_{enc}, f_{rsk}, h_{nce}$, and h_{seq} as in G_3 (cf. Figure 5)		50 $(u_1, v_1, cnt_1) := h_{nce}^{-1}(\alpha)$	// G_6
02 $f_k \leftarrow \text{Func}(\{0, 1\}^*, G)$	// G_6	51 $(u_2, v_2, cnt_2) := h_{nce}^{-1}(\beta)$	// G_6
03 $O_1 := (\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R)$		52 if $(u_1, v_1, cnt_1) \neq \perp$	// G_6
04 $O_2 := (PK, COR, REV, TEST)$		53 $(epk', esk') := \text{KEM.KG}(\cdot; f_{ekg}(cnt_1))$	// G_6
05 $b' \leftarrow \mathcal{A}^{O_1, O_2, H}(par)$		54 $k' := \text{KEM.Decaps}(esk', c)$	// G_6
06 for $sID^* \in \mathcal{S}_{test}$:		55 if $epk = epk' \wedge k = k'$: $k^* := f_k(epk, c)$	// G_6
07 if $\text{Fresh}(sID^*) = 0 \vee \text{Valid}(sID^*) = 0$		56 else if $(u_2, v_2, cnt_2) \neq \perp$	// G_6
08 return b		57 $(c', k') := \text{KEM.Encaps}(epk; f_{enc}(epk, cnt_2))$	// G_6
09 return $\llbracket b' = b \rrbracket$		58 if $c = c' \wedge k = k'$: $k^* := f_k(epk, c)$	// G_6
		59 return $f_H(pk_1, pk_2, \alpha, \beta, epk, c, \sigma_1, \sigma_2, k^*)$	
Oracle $TEST(sID)$		Oracle $SESSION_R(u, v, M_u = (\alpha, epk, \sigma_1))$	
10 if $\mathcal{L}_{SK}[sID] = \perp$: return $\perp$	// G_3 - G_4	60 if $\text{Vrfy}(pk_u, (\alpha, epk), \sigma_1) = 0$: return $\perp$	
11 if $\text{CheckSid}(sID) = 0$	// G_5 - G_6	61 $cnt := cnt + 1, \beta := h_{nce}(u, v, cnt)$	
12 return $\perp$	// G_5 - G_6	62 $(c, k) := \text{KEM.Encaps}(epk; f_{enc}(epk, cnt))$	
13 $\mathcal{S}_{test} := \mathcal{S}_{test} \cup \{sID\}$		63 $\sigma_2 := \text{Sign}(sk_v, (\alpha, \beta, epk, c); f_{sR}(\alpha, \beta, epk, c))$	
14 parse $(*, u, v, M_u, M_v) := sID$		64 $K := H(pk_u, pk_v, \alpha, \beta, epk, c, \sigma_1, \sigma_2, k)$	
15 $SK_0 := \mathcal{L}_{SK}[sID]$	// G_3 - G_5	65 $K := f_H(pk_u, pk_v, \alpha, \beta, epk, c, \sigma_1, \sigma_2, f_k(epk, c))$	// G_6
16 $SK_0 = \text{ComputeSK}(sID)$	// G_6	66 $M_v := (\beta, c, \sigma_2)$	
17 $SK_1 := f_{rsk}(u, v, M_u, M_v)$		67 if $(u, M_u) \in \mathcal{S}_{msg}$: $\mathcal{S}_{msg} := \mathcal{S}_{msg} \cup \{(u, v, M_u, M_v)\}$	
18 return SK_b		68 $seq := f_{seq}(u, v, M_u, M_v), sID := (seq, u, v, M_u, M_v)$	
		69 $\mathcal{L}_{SK}[sID] := K$	// G_3 - G_5
Oracle $REV(sID)$		70 return (M_v, sID)	
19 $\mathcal{S}_{rev} := \mathcal{S}_{rev} \cup \{sID\}$		Oracle $DER_I(u, v, seq_1, M_v = (\beta, c, \sigma_2))$	
20 if $\text{CheckSid}(sID) = 0$	// G_5 - G_6	71 if $\mathcal{L}_{st}[seq_1] = \perp$: return $\perp$	// G_3
21 return $\perp$	// G_5 - G_6	72 $(u', v', (esk, \alpha, epk, \sigma_1)) := \mathcal{L}_{st}[seq_1]$	// G_3
22 return $\mathcal{L}_{SK}[sID]$	// G_3 - G_5	73 if $(u', v') \neq (u, v)$: return $\perp$	// G_3
23 return $\text{ComputeSK}(sID)$	// G_6	74 $(u', v', cnt') := h_{nce}^{-1}(seq_1)$	// G_4 - G_6
Oracle $SESSION_I(u, v)$		75 if $(u', v') \neq (u, v) \vee cnt < cnt'$: return $\perp$	// G_4 - G_6
24 $cnt := cnt + 1$		76 $\alpha := h_{nce}(u, v, cnt')$	// G_4 - G_6
25 $seq_1 := h_{seq}(u, v, cnt)$		77 $x := f_{ekg}(cnt'), (epk, esk) := \text{KEM.KG}(\cdot; x)$	// G_4 - G_6
26 $\alpha := h_{nce}(u, v, cnt)$		78 $\sigma_1 := \text{Sign}(sk_u, (\alpha, epk); f_{sI}(\alpha, epk))$	// G_4 - G_6
27 $(epk, esk) := \text{KEM.KG}(\cdot; f_{ekg}(cnt))$		79 if $\text{Vrfy}(pk_v, (\alpha, \beta, epk, c), \sigma_2) = 0$: return $\perp$	
28 $\sigma := \text{Sign}(sk_u, (\alpha, epk); f_{sI}(\alpha, epk))$		80 $k := \text{Decaps}(esk, c)$	
29 $M_u := (\alpha, epk, \sigma)$		81 $K := H(pk_u, pk_v, \alpha, \beta, epk, c, \sigma_1, \sigma_2, k)$	// G_3 - G_5
30 $\mathcal{L}_{st}[seq_1] := (u, v, (esk, M_u))$	// G_3	82 $K := f_H(pk_u, pk_v, \alpha, \beta, epk, c, \sigma_1, \sigma_2, f_k(epk, c))$	// G_6
31 $\mathcal{S}_{msg} := \mathcal{S}_{msg} \cup \{(u, M_u)\}$		83 $seq_2 := f_{seq}(seq_1, u, v, M_u)$	
32 return (seq_1, M_u)		84 $sID := (seq_1, seq_2, u, v, M_u, M_v)$	
Alg $\text{CheckMsgI}(u, \alpha, epk, \sigma_1)$		85 $\mathcal{L}_{SK}[sID] := K$	// G_3 - G_5
33 $(u', v, cnt) := h_{nce}^{-1}(\alpha)$		86 return sID	
34 if $(u, v, cnt) \neq \perp \wedge u = u'$		Alg $\text{CheckSid}(sID)$	
35 $(epk', esk') := \text{KEM.KG}(\cdot; f_{ekg}(cnt))$		87 //Case: Responder session	
36 $\sigma'_1 := \text{Sign}(sk_u, (\alpha, epk); f_{sI}(\alpha, epk))$		88 if $sID = (seq, u, v, (\alpha, epk, \sigma_1), (\beta, c, \sigma_2))$	
37 if $epk' = epk \wedge \sigma'_1 = \sigma_1$: return 1		89 $seq' := f_{seq}(u, v, (\alpha, epk, \sigma_1), (\beta, c, \sigma_2))$	
38 return 0		90 if $seq' \neq seq$: return 0	
Alg $\text{CheckMsgR}(v, cnt_1, \beta, c, \sigma_2)$		91 else return 1	
39 $(u, v', cnt_2) := h_{nce}^{-1}(\beta)$		92 //Case: Initiator session	
40 if $(u, v, cnt_2) \neq \perp \wedge v = v'$		93 if $sID = (seq_1, seq_2, u, v, (\alpha, epk, \sigma_1), (\beta, c, \sigma_2))$	
41 $\alpha := h_{nce}(u, v, cnt_1)$		94 $seq'_2 := f_{seq}(seq_1, u, v, (\beta, c, \sigma_2))$	
42 $(epk, esk) := \text{KEM.KG}(\cdot; f_{ekg}(cnt_1))$		95 if $seq'_2 \neq seq_2$: return 0	
43 $y := f_{enc}(epk, cnt_2)$		96 else return 1	
44 $(c', k') := \text{KEM.Encaps}(epk; y)$		97 return 0	
45 $r_{sign} := f_{sR}(\alpha, \beta, epk, c)$		Alg $\text{ComputeSK}(sID)$	
46 $\sigma'_2 := \text{Sign}(sk_v, (\alpha, \beta, epk, c); r_{sign})$		98 $(*, u, v, (\alpha, epk, \sigma_1), (\beta, c, \sigma_2)) := sID$	
47 if $c' = c \wedge \sigma'_2 = \sigma_2$: return 1		99 return $f_H(pk_u, pk_v, \alpha, \beta, epk, c, \sigma_1, \sigma_2, f_k(epk, c))$	
48 return 0			

Fig. 6. Games G_3 - G_6 for proving Theorem 1. For any party u , we have $(pk_u, sk_u) = \text{GetKey}(u)$. Oracles COR and PK and algorithm GetKey are the same as in Figure 5.

For both responder sessions and initiator sessions, the final sequence number (e.g., seq or seq_2) are generated from the random function f_{seq} (cf. lines 85 and 102). This means that, to construct a valid session identifier sID that does not correspond to any session, $\mathcal{A}$ has to compute the correct corresponding output of f_{seq} . Since f_{seq} is a random function, there are at most N_{ses} session identifiers seen by $\mathcal{A}$, and $\mathcal{A}$ queries TEST or REV at most $Q_{\text{rev}} + Q_{\text{test}}$ times, if $|\text{SEQ}| \geq Q_{\text{rev}} + Q_{\text{test}} + N_{\text{ses}}$, we have

$$|\Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1]| \leq \frac{Q_{\text{rev}} + Q_{\text{test}}}{|\text{SEQ}| - N_{\text{ses}}} \leq \frac{Q_{\text{rev}} + Q_{\text{test}} + N_{\text{ses}}}{|\text{SEQ}|}$$

Game $\mathbf{G}_6$ (Compute session keys without decapsulated keys). We modify $\mathbf{H}$ such that the session keys in Der_I and Der_R can be computed without using decapsulated KEM keys. To make the simulation consistent, we also change how session keys are computed in oracles Der_I and Der_R , and use an auxiliary algorithm ComputeSK (cf. Figure 6) to compute the outputting session key of TEST and REV. More specifically, we make the following changes to $\mathbf{H}$:

- When $\mathcal{A}$ queries $\mathbf{H}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$, where (α, epk) is output by SESSION_I , we can recover esk (the secret key of epk) by using the information hidden in α . In this case, if $k = \text{Decaps}(\text{esk}, c)$, then we return $f_{\mathbf{H}}(u, v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_k(\text{epk}, c))$ (cf. lines 50 to 55).
- When $\mathcal{A}$ queries $\mathbf{H}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$, where (β, c) is output by SESSION_R , we can recover k' (the decapsulated key of c) by using the information hidden in β . In this case, if $k = k'$, then we return $f_{\mathbf{H}}(u, v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_k(\text{epk}, c))$ (cf. lines 56 to 58).

We also change the oracles TEST, Der_I , and SESSION_R so that their outputs are consistent with the outputs of $\mathbf{H}$.

- The outputs of REV and the real keys in TEST are computed by the internal procedure ComputeSK (cf. Figure 6). Oracles REV and TEST first check if the input sID is valid (by using CheckSid), and then use ComputeSK to compute the session key. On input a valid session identifier $\text{sID} = (*, u, v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2)$, ComputeSK computes sID 's session key as $f_{\mathbf{H}}(u, v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_k(\text{epk}, c))$ (cf. lines 16 and 23), where f_k is a random function.
- To keep consistence, we compute the session keys in Der_I and SESSION_R in the same way as ComputeSK (cf. lines 65 and 82).

Due to space limitations, we summarize the analysis of $\mathbf{G}_6$ as a lemma and postpone its proof to Supp. Mat. D.2.

Lemma 3. *With the notations and assumptions as stated in Theorem 1, we have*

$$|\Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_6^{\mathcal{A}} \Rightarrow 1]| \leq (Q_{\mathbf{H}} + N_{\text{ses}})^2 \cdot |\mathcal{K}|^{-1} + N_{\text{ses}} \cdot \delta_{\text{KEM}}.$$

Game $\mathbf{G}_7$ (Identify no-corruption active attacks - 1). In this game, we identify no-corruption active attacks. Namely, we check if the adversary is tampering with a session, namely, attacking a session that does not have matching session and the peer of the session is not corrupted.

Specifically, when $\mathcal{A}$ queries $\text{SESSION}_R(u, v, \text{M}_u)$, if u is corrupted and $(u, \text{M}_u) \notin \mathcal{S}_{\text{msg}}$ (i.e., M_u is not from $\text{SESSION}_I(u, *)$), then we use $\mathcal{S}_{\text{forge}}$ to store this session (cf. lines 01 to 02). Similarly, when $\mathcal{A}$ queries $\text{Der}_I(u, v, \text{seq}, \text{M}_v)$, if v is not corrupted and $(u, v, \text{M}_u, \text{M}_v) \notin \mathcal{S}_{\text{msg}}$ (i.e., M_v is not the responding message of this session, where M_u is the sent protocol message of this session), then we also use $\mathcal{S}_{\text{forge}}$ to store this session (cf. lines 23 to 24).

This game is a preparatory step for the later reduction from SIG. Since the set $\mathcal{S}_{\text{forge}}$ is internal and cannot be accessed by the adversary, the change made in this game is conceptual, so we have $\Pr[\mathbf{G}_6^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_7^{\mathcal{A}} \Rightarrow 1]$.

Game $\mathbf{G}_8$ (Identify no-corruption active attacks - 2). We use two internal auxiliary algorithms CheckMsgR and CheckMsgI (cf. Figure 6) to identify no-corruption active attacks without using $\mathcal{S}_{\text{msg}}$.

We first explain how CheckMsgI works. On input $(u, \alpha, \text{epk}, \sigma_1)$, CheckMsgI tries recovering the session state information of the session that generated α (cf. lines 33 to 38). If $\mathcal{A}$ sends $(u, (\alpha, \text{epk}, \sigma_1))$ to SESSION_R , where u is uncorrupted and $(u, (\alpha, \text{epk}, \sigma_1)) \in \mathcal{S}_{\text{msg}}$ (i.e., generated from oracle SESSION_I), then $\text{CheckMsgI}(u, \alpha, \text{epk}, \sigma_1)$ returns 1. Note that if u is corrupted, then SESSION_R will not run CheckMsgI because of the logic AND condition. When $\mathcal{A}$ sends $(u, (\alpha, \text{epk}, \sigma_1)) \notin \mathcal{S}_{\text{msg}}$ to SESSION_R , we consider two cases:

- Case 1: α is forged by $\mathcal{A}$. Let $(u', v, \text{cnt}) := h_{\text{ncc}}^{-1}(\alpha)$, $x := f_{\text{ekg}}(\text{cnt})$, and $(\text{epk}', \text{esk}') := \text{KEM.KG}(\cdot; x)$. Since h_{ncc} is injective and f_{ekg} is a random function, the probability that $u' = u$ and $\text{epk} = \text{epk}'$ (note

Oracle $\text{SESSION}_R(u, v, M_u = (\alpha, \text{epk}, \sigma_1))$ 00 if $\text{Vrfy}(\text{pk}_u, (\alpha, \text{epk}), \sigma_1) = 0$: return $\perp$ 01 if $u \notin \mathcal{S}_{\text{cor}}$ $\wedge (u, M_u) \notin \mathcal{S}_{\text{msg}}$ $\wedge \text{CheckMsgl}(u, \alpha, \text{epk}, \sigma_1) = 0$ 02 $\mathcal{S}_{\text{forge}} := \mathcal{S}_{\text{forge}} \cup \{\text{sID}\}$ 03 return $\perp$ 04 $\text{cnt} := \text{cnt} + 1, \beta := h_{\text{nce}}^{-1}(u, v, \text{cnt})$ 05 $(c, k) := \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}))$ 06 $\sigma_2 := \text{Sign}(\text{sk}_v, (\alpha, \beta, \text{epk}, c); f_{\text{sr}}(\alpha, \beta, \text{epk}, c))$ 07 $K := f_{\text{H}}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_k(\text{epk}, c))$ 08 $M_v := (\beta, c, \sigma_2)$ 09 $\text{seq} := f_{\text{seq}}(u, v, M_u, M_v)$ 10 $\text{sID} := (\text{seq}, u, v, M_u, M_v)$ 11 if $(u, M_u) \in \mathcal{S}_{\text{msg}}$ 12 $\mathcal{S}_{\text{msg}} := \mathcal{S}_{\text{msg}} \cup \{(u, v, M_u, M_v)\}$ 13 $\mathcal{S}_{\text{sid}} := \mathcal{S}_{\text{sid}} \cup \{\text{sID}\}$ 14 return (M_v, sID)	Oracle $\text{DER}_I(u, v, \text{seq}_1, M_v = (\beta, c, \sigma_2))$ 15 $(u', v', \text{cnt}') := h_{\text{seq}}^{-1}(\text{seq}_1)$ 16 if $(u', v') \neq (u, v) \vee \text{cnt} < \text{cnt}'$: return $\perp$ 17 $\alpha := h_{\text{nce}}(u, v, \text{cnt}')$ 18 $(\text{epk}, \text{esk}) := \text{KEM.KG}(\cdot; f_{\text{ekg}}(\text{cnt}'))$ 19 $k := \text{KEM.Decaps}(\text{esk}, c)$ 20 $\sigma_1 := \text{Sign}(\text{sk}_u, (\alpha, \text{epk}); f_{\text{SI}}(\alpha, \text{epk}))$ 21 $M_u := (\alpha, \text{epk}, \sigma_1)$ 22 if $\text{Vrfy}(\text{pk}_v, (\alpha, \beta, \text{epk}, c), \sigma_2) = 0$: return $\perp$ 23 if $v \notin \mathcal{S}_{\text{cor}}$ $\wedge (u, v, M_u, M_v) \notin \mathcal{S}_{\text{msg}}$ $\wedge \text{CheckMsgR}(v, \text{cnt}', \beta, c, \sigma_2) = 0$ 24 $\mathcal{S}_{\text{forge}} := \mathcal{S}_{\text{forge}} \cup \{\text{sID}\}$ 25 return $\perp$ 26 $K := f_{\text{H}}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_k(\text{epk}, c))$ 27 $\text{seq}_2 := f_{\text{seq}}(\text{seq}_1, u, v, M_u, M_v)$ 28 $\text{sID} := (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$ 29 $\mathcal{S}_{\text{sid}} := \mathcal{S}_{\text{sid}} \cup \{\text{sID}\}$ 30 return sID	Oracle $\text{H}(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$ 31 $k^* := k, (u_1, v_1, \text{cnt}_1) := h_{\text{nce}}^{-1}(\alpha), (u_2, v_2, \text{cnt}_2) := h_{\text{nce}}^{-1}(\beta)$ 32 if $(u_1, v_1, \text{cnt}_1) \neq \perp \wedge (u_2, v_2, \text{cnt}_2) \neq \perp$ 33 $(\text{epk}', \text{esk}') := \text{KEM.KG}(\cdot; f_{\text{ekg}}(\text{cnt}_1))$ 34 $(c', k') := \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}_2))$ 35 if $\text{epk} = \text{epk}' \wedge c = c' \wedge k = k'$: abort and output b 36 Compute k^* as in lines 52 to 58. 37 return $f_{\text{H}}(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k^*)$
--	--	---

Fig. 7. Oracles DER_I , SESSION_R , and H of $\mathbf{G}_7$ - $\mathbf{G}_{10}$. For readability, we only present changed oracles of games sequence $\mathbf{G}_7$ - $\mathbf{G}_{10}$. Other oracles are the same as in Figure 6.

- that epk is derived from cnt) hold is $\frac{1}{|\mathcal{U}| - N_{\text{ses}}} \cdot \frac{1}{(N - N_{\text{ses}})}$ (as the adversary learns at most N_{ses} points of each image), which is less than $\frac{(N_{\text{ses}} + 1)^2}{|\mathcal{U}|N}$ if $|\mathcal{U}| \geq N_{\text{ses}} + 1$ and $N \geq N_{\text{ses}} + 1$. Therefore, the probability that $\mathcal{A}$ successfully forges such α is bounded by $\frac{N_{\text{ses}}(N_{\text{ses}} + 1)^2}{|\mathcal{U}|N}$ if $|\mathcal{U}| \geq N_{\text{ses}} + 1$ and $N \geq N_{\text{ses}} + 1$.
- **Case 2:** α is generated from SESSION_I . Let $(\alpha, \text{epk}', \sigma'_1) \neq (\alpha, \text{epk}, \sigma_1)$ be the initiator protocol message generated by u that corresponds to α (namely, $(u, (\alpha, \text{epk}', \sigma'_1)) \in \mathcal{S}_{\text{msg}}$). Let $(u, v, \text{cnt}) := h_{\text{nce}}^{-1}(\alpha)$, $x := f_{\text{ekg}}(\text{cnt})$, and $(\text{epk}', \text{esk}') := \text{KEM.KG}(\cdot; x)$. If $\text{epk} \neq \text{epk}'$, then CheckMsgl returns 0. So in this case, we must have $\text{epk} = \text{epk}'$, which means that $\sigma_1 \neq \sigma'_1$. However, since $\sigma'_1 = \text{Sign}(\text{sk}_u, (\alpha, \text{epk}'); f_{\text{SI}}(\alpha, \text{epk}'))$ (by line 53), if $\sigma_1 \neq \sigma'_1$, then $\text{CheckMsgl}(u, \alpha, \text{epk}, \sigma_1)$ still returns 0. Therefore, in this case, we have $\text{CheckMsgl}(u, \alpha, \text{epk}, \sigma_1)$ always outputs 0, which means that $\mathcal{A}$ cannot get any advantage from such queries.

Therefore, when u is uncorrupted, $\text{CheckMsgl}(u, \alpha, \text{epk}, \sigma_1)$ returns 1 if and only if $(u, (\alpha, \text{epk}, \sigma_1)) \in \mathcal{S}_{\text{msg}}$ except with probability at most $N_{\text{ses}}(N_{\text{ses}} + 1)^2 \cdot (|\mathcal{U}|N)^{-1}$.

Similarly, we use CheckMsgR to check if a message is from a matching responder session that its owner is uncorrupted. More specifically, when $\mathcal{A}$ query $\text{DER}_I(\text{seq}_1, v, (\beta, c, \sigma_2))$, if v is uncorrupted, then we run $\text{CheckMsgR}(v, \text{cnt}, \beta, c, \sigma_2)$ to check if $(u, v, (\alpha, \text{epk}, \sigma_1), (\beta, c, \sigma_2)) \in \mathcal{S}_{\text{msg}}$, where $(u, v, \text{cnt}) = h_{\text{seq}}^{-1}(\text{seq}_1)$ and cnt is used to generate α, epk , and σ_1 . Since CheckMsgl is symmetric to CheckMsgR , similar arguments of CheckMsgl apply to CheckMsgR , so CheckMsgR correctly simulates the check $(u, v, M_u, M_v) \in? \mathcal{S}_{\text{msg}}$ except with probability at most $N_{\text{ses}}(N_{\text{ses}} + 1)^2 \cdot (|\mathcal{U}|N)^{-1}$. Therefore, we have

$$|\Pr[\mathbf{G}_7^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_8^{\mathcal{A}} \Rightarrow 1]| \leq \frac{2N_{\text{ses}}(N_{\text{ses}} + 1)^2}{|\mathcal{U}|N}$$

Game $\mathbf{G}_9$ (Exclude no-corruption active attacks). When $\mathcal{A}$ launches a successful no-corruption active attacks, we no longer record this session in $\mathcal{S}_{\text{forge}}$. Instead, we simply reject this session by returning $\perp$ (cf. lines 03 and 25). This game is for ensuring that $\mathcal{A}$ cannot impersonate any uncorrupted user.

Intuitively, the adversary notices this change if it can forge a valid signature of some uncorrupted user. We define an event **Forge** to capture this. Specifically, we say **Forge** happens if at least one of the following cases happens: (1) $\mathcal{A}$ queries $\text{SESSION}_R(u, v, (\alpha, \text{epk}, \sigma_1))$ for some u and v that u is uncorrupted, $\text{CheckMsgl}(u, \alpha, \text{epk}, \sigma_1) = 1$, and $\text{Vrfy}(\text{pk}_u, (\alpha, \text{epk}), \sigma_1) = 1$. (2) $\mathcal{A}$ queries $\text{DER}_I(\text{seq}_1, u, (\beta, c, \sigma_2))$ for some u and v that v is uncorrupted, $\text{CheckMsgR}(v, \text{cnt}', \beta, c, \sigma_2) = 1$, and $\text{Vrfy}(\text{pk}_v, (\alpha, \beta, c), \sigma_2) = 1$,

Reduction $\mathcal{B}_1^{\text{PK}', \text{SIGN}', \text{COR}'(\text{par}_{\text{sig}})}$ <pre> 00 cnt := 0, b ← {0, 1}, Forge := false 01 Initialize $f_{\text{seq}}, f_{\text{H}}, f_{\text{SK}}, f_{\text{SI}}, f_{\text{SR}}, f_{\text{EK}},$ $f_{\text{enc}}, f_{\text{rSK}}, f_{\text{K}}, h_{\text{ncc}},$ and h_{seq} as in $\mathbf{G}_6$ 02 $\text{par}_{\text{KEM}} \leftarrow \text{KEM.Setup}$ 03 $\text{par} := (\text{par}_{\text{KEM}}, \text{par}_{\text{sig}})$ 04 $b' \leftarrow \mathcal{A}^{O, H}(\text{par})$ 05 abort and return $\perp$ Oracle PK(u) 06 return $\text{PK}'(u)$ Oracle TEST(sID) 07 if $\text{CheckSid}(\text{sID}) = 0$: return $\perp$ 08 $\text{parse}(*, u, v, M_u, M_v) := \text{sID}$ 09 $\text{SK}_0 = \text{ComputeSK}(\text{sID})$ 10 $\text{SK}_1 := f_{\text{rSK}}(u, v, M_u, M_v)$ 11 return SK_b Oracle REV(sID) 12 if $\text{CheckSid}(\text{sID}) = 0$: return $\perp$ 13 return $\text{ComputeSK}(\text{sID})$ Alg CheckMsgI(u, α, epk, σ_1) 14 $(u, v, \text{cnt}) := h_{\text{ncc}}^{-1}(\alpha)$ 15 if $(u, v, \text{cnt}) \neq \perp$ 16 $(\text{epk}', \text{esk}') := \text{KEM.KG}(:; f_{\text{ekg}}(\text{cnt}))$ 17 $\sigma'_1 := \text{SIGN}'(u, (\alpha, \text{epk}'))$ 18 if $\text{epk}' = \text{epk} \wedge \sigma'_1 = \sigma_1$: return 1 19 return 0 Alg CheckMsgR(v, cnt₁, β, c, σ_2) 20 $(u, v, \text{cnt}_2) := h_{\text{ncc}}^{-1}(\beta)$ 21 $\alpha := h_{\text{ncc}}(u, v, \text{cnt}_1)$ 22 $(\text{epk}, \text{esk}) := \text{KEM.KG}(:; f_{\text{ekg}}(\text{cnt}_1))$ 23 if $(u, v, \text{cnt}_2) \neq \perp$ 24 $y := f_{\text{enc}}(\text{epk}, \text{cnt}_2)$ 25 $(c', k') := \text{KEM.Encaps}(\text{epk}; y)$ 26 $\sigma'_2 := \text{SIGN}'(v, (\alpha, \beta, \text{epk}, c))$ 27 if $c' = c \wedge \sigma'_2 = \sigma_2$: return 1 28 return 0 </pre>	Oracle COR(u) <pre> 29 $\mathcal{S}_{\text{cor}} := \mathcal{S}_{\text{cor}} \cup \{u\}$ 30 return $\text{COR}'(u)$ Oracle SESSION_I(u, v) 31 cnt := cnt + 1 32 $\text{seq}_1 := h_{\text{seq}}(u, v, \text{cnt}), \alpha := h_{\text{ncc}}(u, v, \text{cnt})$ 33 $(\text{epk}, \text{esk}) := \text{KEM.KG}(:; f_{\text{ekg}}(\text{cnt}))$ 34 $\sigma := \text{SIGN}'(u, (\alpha, \text{epk}))$ 35 $M_u := (\alpha, \text{epk}, \sigma)$ 36 return (seq_1, M_u) Oracle SESSION_R(u, v, $M_u = (\alpha, \text{epk}, \sigma_1)$) 37 if $\text{Vrfy}(\text{pk}_u, (\alpha, \text{epk}), \sigma_1) = 0$: return $\perp$ 38 if $u \notin \mathcal{S}_{\text{cor}} \wedge \text{CheckMsgI}(u, \alpha, \text{epk}, \sigma_1) = 0$ 39 Forge := true 40 $\text{FORGE}'(u, (\alpha, \text{epk}), \sigma_1)$ and abort 41 cnt := cnt + 1, $\beta := h_{\text{ncc}}(u, v, \text{cnt})$ 42 $(c, k) := \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}))$ 43 $\sigma_2 := \text{SIGN}'(v, (\alpha, \beta, \text{epk}, c))$ 44 $K := f_{\text{H}}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_{\text{K}}(\text{epk}, c))$ 45 $M_v := (\beta, c, \sigma_2)$ 46 $\text{seq} := f_{\text{seq}}(u, v, M_u, M_v)$ 47 $\text{sID} := (\text{seq}, u, v, M_u, M_v)$ 48 return (M_v, sID) Oracle DER_I(u, v, $\text{seq}_1, M_v = (\beta, c, \sigma_2)$) 49 $(u', v, \text{cnt}') := h_{\text{seq}}^{-1}(\text{seq}_1)$ 50 if $(u', v') \neq (u, v) \vee \text{cnt} < \text{cnt}'$: return $\perp$ 51 $\alpha := h_{\text{ncc}}(u, v, \text{cnt}')$ 52 $(\text{epk}, \text{esk}) := \text{KEM.KG}(:; f_{\text{ekg}}(\text{cnt}'))$ 53 $\sigma_1 := \text{SIGN}'(u, (\alpha, \text{epk}))$ 54 if $\text{Vrfy}(\text{pk}_u, (\alpha, \beta, \text{epk}, c), \sigma_2) = 0$: return $\perp$ 55 if $v \notin \mathcal{S}_{\text{cor}}$ 56 $\wedge \text{CheckMsgR}(v, \text{cnt}', \beta, c, \sigma_2) = 0$ 57 Forge := true 58 $\text{FORGE}'(v, (\alpha, \beta, \text{epk}, c), \sigma_2)$ and abort 59 $K := f_{\text{H}}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_{\text{K}}(\text{epk}, c))$ 59 $\text{seq}_2 := f_{\text{seq}}(\text{seq}_1, u, v, M_v)$ 60 $\text{sID} := (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$ 61 return sID </pre>
---	---

Fig. 8. $\mathcal{B}_1$ against $\text{MU}^c\text{-sEUF-1CMA}$ for $\mathbf{G}_9$ in Theorem 1. $\mathcal{B}_1$ has access to $\{\text{PK}, \text{COR}, \text{SIGN}\}$. $\mathcal{A}$ has access to $O := \{\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R, \text{PK}, \text{COR}, \text{REV}, \text{TEST}\}$. Highlighted lines show how we use these oracles to simulate $\mathbf{G}_9$ for $\mathcal{A}$. We always let $\text{pk}_u := \text{PK}(u)$ and $\text{pk}_v := \text{PK}(v)$ and H is the same as in $\mathbf{G}_8$ (cf. Figure 6).

where $(u, v, \text{cnt}') = h_{\text{seq}}^{-1}(\text{seq}_1)$ and $\alpha = h_{\text{ncc}}(u, v, \text{cnt}')$. The event **Forge** is equivalent to the event that a session identifiers is added to $\mathcal{S}_{\text{forge}}$ in $\mathbf{G}_8$.

If **Forge** does not happen, then $\mathcal{A}$ cannot distinguish $\mathbf{G}_9$ from $\mathbf{G}_8$ since the distributions of DER_I and SESSION_R do not change unless $\mathcal{A}$ triggers **Forge**. To bound $\Pr[\text{Forge}]$, we construct a reduction $\mathcal{B}_1$ in Figure 8 which is against the $\text{MU}^c\text{-sEUF-1CMA}$ security of SIG . In the construction of $\mathcal{B}_1$ in Figure 8, we use a flag **Forge** to indicate if the event **Forge** happens or not. That is, if **Forge** happens, then we raise **Forge** as **true**. Moreover, $\mathcal{B}_1$ aborts the simulation right after raising **Forge** and returns a forgery to $\text{MU}^c\text{-sEUF-1CMA}$.

Suppose that $\mathcal{B}_1$ correctly simulates $\mathbf{G}_9$. We first show that, if **Forge** is raised as **true**, then $\mathcal{B}_1$ finds a valid forgery and wins the game $\text{MU}^c\text{-sEUF-1CMA}$. By the if-conditions of lines 39 and 56, **Forge** is raised when (1) $\mathcal{A}$ sent a valid protocol message $(\alpha, \text{epk}, \sigma_1)$ on behalf of an uncorrupted user u and $\text{CheckMsgI}(u, (\alpha, \text{epk}, \sigma_1)) = 0$, or (2) $\mathcal{A}$ sent a valid protocol message (β, c, σ_2) to user u on behalf of an uncorrupted user v and $\text{CheckMsgR}(v, \text{cnt}, (\beta, c, \sigma_2)) = 0$, where cnt is the internal counter of the session of u that receives (β, c, σ_2) .

For case (1), $\text{CheckMsgI}(u, (\alpha, \text{epk}, \sigma_1)) = 0$ means that $(\alpha, \text{epk}, \sigma_1)$ is not from SESSION_I and thus not from SIGN . For case (2), $\text{CheckMsgR}(v, \text{cnt}, (\beta, c, \sigma_2)) = 0$ means that (β, c, σ_2) is not from the responder session that matches with the initiator session with internal counter cnt . Both cases correspond the winning condition of $\text{MU}^c\text{-sEUF-1CMA}$ (cf. line 12). That is, these cases happen means that the adversary produces a fresh valid forgery of some uncorrupted users. Therefore, if $\mathcal{B}_1$ perfectly simulates $\mathbf{G}_9$, then $\Pr[\text{Forge}] \leq \Pr[\text{MU}^c\text{-sEUF-1CMA}_{\text{SIG}}^{\mathcal{B}_1} \Rightarrow 1]$.

Then we show that $\mathcal{B}_1$ perfectly simulates $\mathbf{G}_9$. It is straightforward to see that $\mathcal{B}_1$ perfectly simulates COR, PK, and SESSION_I by using the oracles COR, PK, and SIGN provided by the MU^c-sEUF-1CMA game. Since the signing oracle SIGN returns the same signature if $\mathcal{B}_1$ queries the same message wrt the same user (cf. line 05), $\mathcal{B}_1$ also correctly simulate the oracles SESSION_R and DER_I. Therefore, we have

$$|\Pr[\mathbf{G}_8^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_9^{\mathcal{A}} \Rightarrow 1]| \leq \Pr[\mathbf{Forge}] \leq \Pr[\text{MU}^c\text{-sEUF-1CMA}_{\text{SIG}}^{\mathcal{B}_1} \Rightarrow 1]$$

$\mathcal{B}_1$ successfully forges a valid signature only if the game does not abort, so one may argue that we need to count in all probabilities of abort events analyzed in previous games. For simplicity, we do not count these bounds here. Instead, we will add these bounds in our final bound.

The memory usage of $\mathcal{B}_1$ will be analyzed later. In the construction of $\mathcal{B}_1$, if the adversary does not forge a valid signature during the game, we simply return $\perp$ without checking if all test sessions are fresh and valid. This is because, if the adversary can forge a valid signature, then $\mathcal{B}_1$ simply returns such a forgery regardless of the validity and freshness of all test sessions. If the adversary cannot produce a valid forgery, then it is meaningless to simulate since it cannot trigger the event **Forge** and thus cannot distinguish $\mathbf{G}_9$ from $\mathbf{G}_8$.

Game $\mathbf{G}_{10}$ (Exclude passive/wFS attacks). We introduce an abort event in oracle H. If $\mathcal{A}$ queries H on $(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$ where $(\alpha, \text{epk}, \sigma_1)$ and (β, c, σ_2) are from two matching sessions and $k = \text{Decaps}(\text{epk}, c)$ (cf. lines 32 to 35), then we abort the game and return a random bit.

We first prove that, when $\mathcal{A}$ queries H on $(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$, if $\text{epk} = \text{epk}'$ and $c = c'$, where $(u, v, \text{cnt}_1) = h_{\text{ncc}}^{-1}(\alpha)$, $(u, v, \text{cnt}_2) = h_{\text{ncc}}^{-1}(\beta)$, $(\text{epk}', \text{esk}') := \text{KEM.KG}(\cdot; f_{\text{ekg}}(\text{cnt}_1))$, and $(c', k') := \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}_2))$, then (α, epk) is from the outputs of SESSION_I and (β, c) is from the outputs of SESSION_R. Suppose that $\mathcal{A}$ forges a (α, epk) such that it satisfies the if-condition in line 32. Since epk is determined if we fix α , any α output by SESSION_I corresponds to a unique epk (which is output by SESSION_I along with this α), $\mathcal{A}$ has to forges both α and epk . As f_{ekg} is a random function and the min-entropy of epk is γ_{pk} , $\mathcal{A}$ forges such (α, epk) within probability $N_{\text{ses}} \cdot 2^{1-\gamma_{\text{pk}}}$. Similarly, $\mathcal{A}$ forges such (β, c) within probability $N_{\text{ses}} \cdot 2^{1-\gamma_{\text{ct}}}$.

Let **QueryH** be the event that $\mathcal{A}$ triggers the abort conditions in lines 32 to 35. If **QueryH** does not happen, then $\mathcal{A}$ cannot distinguish $\mathbf{G}_{10}$ from $\mathbf{G}_9$. To bound the probability that **QueryH** happens in $\mathbf{G}_{10}$, we construct a reduction $\mathcal{B}_2$ in Figure 9 which is against the OW-PCA security of KEM.

In the construction of $\mathcal{B}_2$, we use the oracles CHALPK and CHALCT to replace the generation and restoring of all epk 's and c 's. The simulation of SESSION_R is slightly different from the one in $\mathbf{G}_{10}$. In lines 36 to 40, we distinguish two cases: If the query corresponds to a trivial attack (i.e., $\mathcal{A}$ not only has corrupted u , but also sends M that not from SESSION_I), then we compute c as in $\mathbf{G}_{10}$; Otherwise, we get c by querying CHALCT. This modification is conceptual since the distribution of c does not changed. We distinguish these two cases because if a session corresponds to a trivial attack (i.e., corrupt a user and then impersonate it), then it is unnecessary to embed a challenge ciphertext into this session.

$\mathcal{B}_2$ uses the PCO oracle to check if any RO query contains a solution of the OW-PCA game. If $\mathcal{A}$ triggers the event **QueryH**, then $\mathcal{B}_2$ correctly recover a OW-PCA solution except that the cnt_1 from α or cnt_2 from β is not correct. In $\mathbf{G}_8$, we have shown that the probability of such error is bounded by $\frac{2N_{\text{ses}}(N_{\text{ses}}+1)^2}{|\mathcal{U}|N}$. Therefore, we have

$$\begin{aligned} & |\Pr[\mathbf{G}_9^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_{10}^{\mathcal{A}} \Rightarrow 1]| \leq \Pr[\mathbf{QueryH}] \\ & \leq |\Pr[\text{OW-PCA}_{\text{KEM}}^{\mathcal{B}_2} \Rightarrow 1] + N_{\text{ses}} \cdot (2^{1-\gamma_{\text{pk}}} + 2^{1-\gamma_{\text{ct}}}) + \frac{2N_{\text{ses}}(N_{\text{ses}} + 1)^2}{|\mathcal{U}|N}|. \end{aligned}$$

Since $\mathcal{B}_2$ successfully breaks the security of KEM only if the game does not abort when playing with $\mathcal{A}$, we will add all probabilities of abort events analyzed in previous games to our final bound.

Now we prove that $\Pr[\mathbf{G}_{10}^{\mathcal{A}} \Rightarrow 1] = 1/2 + \text{negl}$. Let $\text{sID} = (*, \text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2)$ be an arbitrary session identifier in $\mathcal{S}_{\text{test}}$. To win the AKE game, $\mathcal{A}$ cannot reveal sID . Since all session keys are generated by random oracle, to distinguish whether $\text{TEST}(\text{sID})$ is the real key or a random key, the only way for $\mathcal{A}$ is to query the correct RO input of the session key of sID .

In DER_I and SESSION_R, we exclude the event that $\mathcal{A}$ impersonates a user without corrupting it. Here impersonation means that $\mathcal{A}$ makes the DER_I or SESSION_R output a session identifier and produce a session key. Therefore, if sID is a valid session identifier, then $(\alpha, \text{epk}, \sigma_1)$ and (β, c, σ_2) are from SESSION_I(u, v) and SESSION_R($u, v, (\alpha, \text{epk}, \sigma_1)$), respectively, and both σ_1 and σ_2 are valid, where u and v are uncorrupted.

Reduction $\mathcal{B}_2^{\text{CHALPK}, \text{CHALCT}, \text{PCO}}(\text{par}_{\text{kem}})$ <pre> 00 cnt := 0, b ← {0, 1} 01 Initialize $f_{\text{seq}}, f_{\text{H}}, f_{\text{skg}}, f_{\text{SI}}, f_{\text{SR}}, f_{\text{ekg}},$ $f_{\text{enc}}, f_{\text{SK}}, f_{\text{k}}, h_{\text{ncc}},$ and h_{seq} as in $\mathbf{G}_6$ 02 $\text{par}_{\text{sig}} \leftarrow \text{Sign.Setup}$ 03 $\text{par} := (\text{par}_{\text{kem}}, \text{par}_{\text{sig}})$ 04 $O_1 := (\text{SESSION}_1, \text{DER}_1, \text{SESSION}_R)$ 05 $O_2 := (\text{PK}, \text{COR}, \text{REV}, \text{TEST})$ 06 $b' \leftarrow \mathcal{A}^{O, H}(\text{par})$ 07 abort and return $\perp$ Oracle $\text{SESSION}_1(u, v)$ 08 cnt := cnt + 1, $\alpha := h_{\text{ncc}}(u, v, \text{cnt})$ 09 $\text{epk} \leftarrow \text{CHALPK}(\text{cnt})$ 10 $\text{seq}_1 := h_{\text{seq}}(u, v, \text{cnt})$ 11 $\sigma := \text{Sign}(\text{sk}_u, (\alpha, \text{epk}); f_{\text{SI}}(\alpha, \text{epk}))$ 12 $M_u := (\alpha, \text{epk}, \sigma)$ 13 return (seq_1, M_u) Oracle $\text{DER}_1(u, v, \text{seq}_1, M_v = (\beta, c, \sigma_2))$ 14 $(u', v, \text{cnt}') := h_{\text{seq}}^{-1}(\text{seq}_1)$ 15 if $(u', v') \neq (u, v) \vee \text{cnt} < \text{cnt}'$: return $\perp$ 16 $\alpha := h_{\text{ncc}}(u, v, \text{cnt}')$ 17 $\text{epk} \leftarrow \text{CHALPK}(\text{cnt}')$ 18 $\sigma_1 := \text{Sign}(\text{sk}_u, (\alpha, \text{epk}); f_{\text{SI}}(\alpha, \text{epk}))$ 19 if $\text{Vrfy}(\text{pk}_v, (\alpha, \beta, \text{epk}, c), \sigma_2) = 0$: return $\perp$ 20 if $v \notin \mathcal{S}_{\text{cor}} \wedge \text{CheckMsgR}(v, \text{cnt}', \beta, c, \sigma_2) = 0$: 21 return $\perp$ 22 $K := f_{\text{H}}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_{\text{k}}(\text{epk}, c))$ 23 $\text{seq}_2 := f_{\text{seq}}(\text{seq}_1, u, v, M_v)$ 24 $\text{sID} := (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$ 25 return sID Alg $\text{CheckMsgI}(u, \alpha, \text{epk}, \sigma_1)$ 26 $(u, v, \text{cnt}) := h_{\text{ncc}}^{-1}(\alpha)$ 27 if $(u, v, \text{cnt}) \neq \perp$: 28 $\text{epk}' \leftarrow \text{CHALPK}(\text{cnt})$ 29 $\sigma'_1 := \text{Sign}(\text{sk}_u, (\alpha, \text{epk}'); f_{\text{SI}}(\alpha, \text{epk}'))$ 30 if $\text{epk} \neq \text{epk}' \wedge \sigma'_1 = \sigma_1$: return 1 31 return 0 </pre>	Oracle $\text{SESSION}_R(u, v, M_u = (\alpha, \text{epk}, \sigma_1))$ <pre> 32 if $\text{Vrfy}(\text{pk}_u, (\alpha, \text{epk}), \sigma_1) = 0$: return $\perp$ 33 if $u \notin \mathcal{S}_{\text{cor}} \wedge \text{CheckMsgI}(u, \alpha, \text{epk}, \sigma_1) = 0$: 34 return $\perp$ 35 cnt := cnt + 1, $\beta := h_{\text{ncc}}(u, v, \text{cnt})$ 36 if $\text{CheckMsgI}(u, \alpha, \text{epk}, \sigma_1) = 1$: 37 $(u, v, \text{cnt}') := h_{\text{ncc}}^{-1}(\alpha)$ 38 $c \leftarrow \text{CHALCT}(\text{cnt}', \text{cnt})$ 39 else 40 $(c, k) := \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}))$ 41 $\sigma_2 := \text{Sign}(\text{sk}_v, (\alpha, \beta, \text{epk}, c); f_{\text{SR}}(\alpha, \beta, \text{epk}, c))$ 42 $K := f_{\text{H}}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_{\text{k}}(\text{epk}, c))$ 43 $M_v := (\beta, c, \sigma_2)$ 44 $\text{seq} := f_{\text{seq}}(u, v, M_u, M_v)$, $\text{sID} := (\text{seq}, u, v, M_u, M_v)$ 45 return (M_v, sID) Oracle $\text{H}(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$ 46 $k^* := k$ 47 $(u_1, v_1, \text{cnt}_1) := h_{\text{ncc}}^{-1}(\alpha)$, $(u_2, v_2, \text{cnt}_2) := h_{\text{ncc}}^{-1}(\beta)$ 48 if $(u_1, v_1, \text{cnt}_1) \neq \perp \wedge (u_2, v_2, \text{cnt}_2) \neq \perp$: 49 $\text{epk}' \leftarrow \text{CHALPK}(\text{cnt}_1)$ 50 $c' \leftarrow \text{CHALCT}(\text{cnt}_1, \text{cnt}_2)$ 51 if $\text{epk} = \text{epk}' \wedge c = c' \wedge \text{PCO}(\text{cnt}_1, c, k) = 1$: 52 abort and output $(\text{cnt}_1, \text{cnt}_2, k)$ 53 if $(u_1, v_1, \text{cnt}_1) \neq \perp$: 54 $\text{epk}' \leftarrow \text{CHALPK}(\text{cnt}_1)$ 55 if $\text{epk} = \text{epk}' \wedge \text{PCO}(\text{cnt}_1, c, k) = 1$: 56 $k^* := f_{\text{k}}(\text{epk}, c)$ 57 else if $(u_2, v_2, \text{cnt}_2) \neq \perp$: 58 $(c', k') := \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}_2))$ 59 if $c = c' \wedge k = k'$: $k^* := f_{\text{k}}(\text{epk}, c)$ 60 return $f_{\text{H}}(\text{pk}_1, \text{pk}_2, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k^*)$ Alg $\text{CheckMsgR}(v, \text{cnt}_1, \beta, c, \sigma_2)$ 61 $(u, v, \text{cnt}_2) := h_{\text{ncc}}^{-1}(\beta)$, $\alpha := h_{\text{ncc}}(u, v, \text{cnt}_1)$ 62 if $(u, v, \text{cnt}_2) \neq \perp$: 63 $c' \leftarrow \text{CHALCT}(\text{cnt}_1, \text{cnt}_2)$ 64 $\sigma'_2 := \text{Sign}(\text{sk}_v, (\alpha, \beta, \text{epk}, c); f_{\text{SR}}(\alpha, \beta, \text{epk}, c))$ 65 if $c = c' \wedge \sigma'_2 = \sigma_2$: return 1 66 return 0 </pre>
---	---

Fig. 9. Reduction $\mathcal{B}_2$ against the OW-PCA game for $\mathbf{G}_{10}$ in Theorem 1. $\mathcal{B}_2$ has access to $(\text{CHALPK}, \text{CHALCT}, \text{PCO})$. Highlighted lines show how we use these oracles to simulate AKE game for $\mathcal{A}$. PK and COR are the same as in $\mathbf{G}_{10}$ (cf. Figure 6) and REV and TEST are the same as in Figure 5. Similar to $\mathcal{B}_1$, we do not need $\mathcal{S}_{\text{rev}}$, $\mathcal{S}_{\text{msg}}$, and $\mathcal{S}_{\text{test}}$.

By the abort event in H, $\mathcal{A}$ cannot get any information about the session key of such a sID from H's outputs. Therefore, in $\mathbf{G}_{10}$, all outputs of TEST are uniformly distributed in $\mathcal{A}$'s view (except that $\mathcal{A}$ finds an output of f_{k} , which is bounded by $\frac{Q_{\text{H}}}{|\mathcal{SK}| - N_{\text{ses}}})$, and we have $\Pr[\mathbf{G}_{10}^{\mathcal{A}} \Rightarrow 1] = 1/2 + (Q_{\text{H}} + N_{\text{ses}})/|\mathcal{K}|$ if $|\mathcal{K}| \geq Q_{\text{H}} + N_{\text{ses}}$. Combining all differences in the games sequence $\mathbf{G}_0$ - $\mathbf{G}_{10}$, we have $\text{Adv}_{\text{aSigKEM}}^{\text{ind-wFSKCI}}(\mathcal{A})$ as claimed in Theorem 1.

ANALYSIS OF TIME AND MEMORY. Let **Time**(aSigKEM) and **Mem**(aSigKEM) be the required running time and memory for running a complete protocol execution of aSigKEM, respectively. By the constructions of $\mathcal{B}_1$ and $\mathcal{B}_2$, we have

$$\begin{aligned}
\text{Mem}^*(\mathcal{B}_1) &= \text{Mem}(\mathcal{A}) + O(\log N_{\text{ses}}) + O(\ell) \\
&\quad + O(N_{\text{user}} \log |\mathcal{U}|) + O(\log |\text{SEQ}|) + \text{Mem}(\text{aSigKEM}) \\
\text{Mem}^*(\mathcal{B}_2) &= \text{Mem}(\mathcal{A}) + O(\log N_{\text{ses}}) + O(\ell) \\
&\quad + O(N_{\text{user}} \log |\mathcal{U}|) + O(\log |\text{SEQ}|) + \text{Mem}(\text{aSigKEM})
\end{aligned}$$

In these reductions, we not only have to run the AKE protocol aSigKEM N_{ses} times to simulate the message sending oracles, but also have to re-run algorithms of SIG and KEM to simulate other oracles or the auxiliary algorithms in Figure 6. For simplicity, we simply write **Time**(aSigKEM) to denote the required running time of running any algorithm of SIG and KEM, respectively. By the constructions of $\mathcal{B}_1$ and $\mathcal{B}_2$, we have

$$\text{Time}^*(\mathcal{B}_1) = \text{Time}(\mathcal{A}) + O(N_{\text{ses}}) \cdot \text{Time}(\text{aSigKEM})$$

$$+ O(N_{\text{ses}} + Q_H) \cdot \text{Time}(\text{KEM}) + O(N_{\text{ses}}) \cdot \text{Time}(\text{SIG})$$

$$\text{Time}^*(\mathcal{B}_2) = \text{Time}(\mathcal{A}) + O(N_{\text{ses}} + Q_H) \cdot \text{Time}(\text{aSigKEM})$$

References

1. Abdalla, M., Bellare, M., Rogaway, P.: The oracle Diffie-Hellman assumptions and an analysis of DHIES. In: Naccache, D. (ed.) CT-RSA 2001. LNCS, vol. 2020, pp. 143–158. Springer, Berlin, Heidelberg (Apr 2001). https://doi.org/10.1007/3-540-45353-9_12
2. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology* **9**(3), 169–203 (2015). <https://doi.org/doi:10.1515/jmc-2015-0016>, <https://doi.org/10.1515/jmc-2015-0016>
3. Auerbach, B., Cash, D., Fersch, M., Kiltz, E.: Memory-tight reductions. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part I. LNCS, vol. 10401, pp. 101–132. Springer, Cham (Aug 2017). https://doi.org/10.1007/978-3-319-63688-7_4
4. Bader, C., Hofheinz, D., Jager, T., Kiltz, E., Li, Y.: Tightly-secure authenticated key exchange. In: Dodis, Y., Nielsen, J.B. (eds.) TCC 2015, Part I. LNCS, vol. 9014, pp. 629–658. Springer, Berlin, Heidelberg (Mar 2015). https://doi.org/10.1007/978-3-662-46494-6_26
5. Becker, A., Ducas, L., Gama, N., Laarhoven, T.: New directions in nearest neighbor searching with applications to lattice sieving. In: Krauthgamer, R. (ed.) 27th SODA. pp. 10–24. ACM-SIAM (Jan 2016). <https://doi.org/10.1137/1.9781611974331.ch2>
6. Bellare, M., Ristenpart, T.: Simulation without the artificial abort: Simplified proof and improved concrete security for Waters’ IBE scheme. In: Joux, A. (ed.) EUROCRYPT 2009. LNCS, vol. 5479, pp. 407–424. Springer, Berlin, Heidelberg (Apr 2009). https://doi.org/10.1007/978-3-642-01001-9_24
7. Bellare, M., Rogaway, P.: The exact security of digital signatures: How to sign with RSA and Rabin. In: Maurer, U.M. (ed.) EUROCRYPT’96. LNCS, vol. 1070, pp. 399–416. Springer, Berlin, Heidelberg (May 1996). https://doi.org/10.1007/3-540-68339-9_34
8. Bellare, M., Rogaway, P.: The security of triple encryption and a framework for code-based game-playing proofs. In: Vaudenay, S. (ed.) EUROCRYPT 2006. LNCS, vol. 4004, pp. 409–426. Springer, Berlin, Heidelberg (May / Jun 2006). https://doi.org/10.1007/11761679_25
9. Bhattacharyya, R.: Memory-tight reductions for practical key encapsulation mechanisms. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020, Part I. LNCS, vol. 12110, pp. 249–278. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45374-9_9
10. Blum, A., Kalai, A., Wasserman, H.: Noise-tolerant learning, the parity problem, and the statistical query model. *J. ACM* **50**(4), 506–519 (Jul 2003). <https://doi.org/10.1145/792538.792543>, <https://doi.org/10.1145/792538.792543>
11. Chen, Y., Nguyen, P.Q.: BKZ 2.0: Better lattice security estimates. In: Lee, D.H., Wang, X. (eds.) ASIACRYPT 2011. LNCS, vol. 7073, pp. 1–20. Springer, Berlin, Heidelberg (Dec 2011). https://doi.org/10.1007/978-3-642-25385-0_1
12. Diemert, D., Gellert, K., Jager, T., Lyu, L.: Digital signatures with memory-tight security in the multi-challenge setting. In: Tibouchi, M., Wang, H. (eds.) ASIACRYPT 2021, Part IV. LNCS, vol. 13093, pp. 403–433. Springer, Cham (Dec 2021). https://doi.org/10.1007/978-3-030-92068-5_14
13. Diemert, D., Gellert, K., Jager, T., Lyu, L.: More efficient digital signatures with tight multi-user security. In: Garay, J. (ed.) PKC 2021, Part II. LNCS, vol. 12711, pp. 1–31. Springer, Cham (May 2021). https://doi.org/10.1007/978-3-030-75248-4_1
14. Esser, A., Kübler, R., May, A.: LPN decoded. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part II. LNCS, vol. 10402, pp. 486–514. Springer, Cham (Aug 2017). https://doi.org/10.1007/978-3-319-63715-0_17
15. Fersch, M., Kiltz, E., Poettering, B.: On the one-per-message unforgeability of (EC)DSA and its variants. In: Kalai, Y., Reyzin, L. (eds.) TCC 2017, Part II. LNCS, vol. 10678, pp. 519–534. Springer, Cham (Nov 2017). https://doi.org/10.1007/978-3-319-70503-3_17
16. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Strongly secure authenticated key exchange from factoring, codes, and lattices. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) PKC 2012. LNCS, vol. 7293, pp. 467–484. Springer, Berlin, Heidelberg (May 2012). https://doi.org/10.1007/978-3-642-30057-8_28
17. Fujioka, A., Suzuki, K., Xagawa, K., Yoneyama, K.: Practical and post-quantum authenticated key exchange from one-way secure key encapsulation mechanism. In: Chen, K., Xie, Q., Qiu, W., Li, N., Tzeng, W.G. (eds.) ASIACCS 13. pp. 83–94. ACM Press (May 2013). <https://doi.org/10.1145/2484313.2484323>
18. Gay, R., Hofheinz, D., Kiltz, E., Wee, H.: Tightly CCA-secure encryption without pairings. In: Fischlin, M., Coron, J.S. (eds.) EUROCRYPT 2016, Part I. LNCS, vol. 9665, pp. 1–27. Springer, Berlin, Heidelberg (May 2016). https://doi.org/10.1007/978-3-662-49890-3_1
19. Gellert, K., Gjøsteen, K., Jacobsen, H., Jager, T.: On optimal tightness for key exchange with full forward secrecy via key confirmation. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part IV. LNCS, vol. 14084, pp. 297–329. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38551-3_10

20. Ghoshal, A., Ghosal, R., Jaeger, J., Tessaro, S.: Hiding in plain sight: Memory-tight proofs via randomness programming. In: Dunkelman, O., Dziembowski, S. (eds.) EUROCRYPT 2022, Part II. LNCS, vol. 13276, pp. 706–735. Springer, Cham (May / Jun 2022). https://doi.org/10.1007/978-3-031-07085-3_24
21. Ghoshal, A., Jaeger, J., Tessaro, S.: The memory-tightness of authenticated encryption. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020, Part I. LNCS, vol. 12170, pp. 127–156. Springer, Cham (Aug 2020). https://doi.org/10.1007/978-3-030-56784-2_5
22. Ghoshal, A., Tessaro, S.: On the memory-tightness of hashed ElGamal. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020, Part II. LNCS, vol. 12106, pp. 33–62. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45724-2_2
23. Gjøsteen, K., Jager, T.: Practical and tightly-secure digital signatures and authenticated key exchange. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part II. LNCS, vol. 10992, pp. 95–125. Springer, Cham (Aug 2018). https://doi.org/10.1007/978-3-319-96881-0_4
24. Han, S., Liu, S., Gu, D.: Almost tight multi-user security under adaptive corruptions & leakages in the standard model. In: Hazay, C., Stam, M. (eds.) EUROCRYPT 2023, Part III. LNCS, vol. 14006, pp. 132–162. Springer, Cham (Apr 2023). https://doi.org/10.1007/978-3-031-30620-4_5
25. Herold, G., Kirshanova, E.: Improved algorithms for the approximate k -list problem in euclidean norm. In: Fehr, S. (ed.) PKC 2017, Part I. LNCS, vol. 10174, pp. 16–40. Springer, Berlin, Heidelberg (Mar 2017). https://doi.org/10.1007/978-3-662-54365-8_2
26. Herold, G., Kirshanova, E., May, A.: On the asymptotic complexity of solving LWE. DCC **86**(1), 55–83 (2018). <https://doi.org/10.1007/s10623-016-0326-0>
27. Hofheinz, D., Hövelmanns, K., Kiltz, E.: A modular analysis of the Fujisaki-Okamoto transformation. In: Kalai, Y., Reyzin, L. (eds.) TCC 2017, Part I. LNCS, vol. 10677, pp. 341–371. Springer, Cham (Nov 2017). https://doi.org/10.1007/978-3-319-70500-2_12
28. Hofheinz, D., Jager, T.: Tightly secure signatures and public-key encryption. In: Safavi-Naini, R., Canetti, R. (eds.) CRYPTO 2012. LNCS, vol. 7417, pp. 590–607. Springer, Berlin, Heidelberg (Aug 2012). https://doi.org/10.1007/978-3-642-32009-5_35
29. Hövelmanns, K., Kiltz, E., Schäge, S., Unruh, D.: Generic authenticated key exchange in the quantum random oracle model. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020, Part II. LNCS, vol. 12111, pp. 389–422. Springer, Cham (May 2020). https://doi.org/10.1007/978-3-030-45388-6_14
30. Jaeger, J., Kumar, A.: Memory-tight multi-challenge security of public-key encryption. In: Agrawal, S., Lin, D. (eds.) ASIACRYPT 2022, Part III. LNCS, vol. 13793, pp. 454–484. Springer, Cham (Dec 2022). https://doi.org/10.1007/978-3-031-22969-5_16
31. Jager, T., Kiltz, E., Riepel, D., Schäge, S.: Tightly-secure authenticated key exchange, revisited. In: Canteaut, A., Standaert, F.X. (eds.) EUROCRYPT 2021, Part I. LNCS, vol. 12696, pp. 117–146. Springer, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-77870-5_5
32. Jiang, H., Zhang, Z., Chen, L., Wang, H., Ma, Z.: IND-CCA-secure key encapsulation mechanism in the quantum random oracle model, revisited. In: Shacham, H., Boldyreva, A. (eds.) CRYPTO 2018, Part III. LNCS, vol. 10993, pp. 96–125. Springer, Cham (Aug 2018). https://doi.org/10.1007/978-3-319-96878-0_4
33. Lenstra, A.K., Lenstra, H.W., Manasse, M.S., Pollard, J.M.: The number field sieve. In: Lenstra, A.K., Lenstra, H.W. (eds.) The development of the number field sieve. pp. 11–42. Springer Berlin Heidelberg, Berlin, Heidelberg (1993)
34. Liu, X., Liu, S., Gu, D., Weng, J.: Two-pass authenticated key exchange with explicit authentication and tight security. In: Moriai, S., Wang, H. (eds.) ASIACRYPT 2020, Part II. LNCS, vol. 12492, pp. 785–814. Springer, Cham (Dec 2020). https://doi.org/10.1007/978-3-030-64834-3_27
35. Lyubashevsky, V., Ducas, L., Kiltz, E., Lepoint, T., Schwabe, P., Seiler, G., Stehlé, D., Bai, S.: CRYSTALS-DILITHIUM. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
36. Pagh, R.: Low redundancy in static dictionaries with constant query time. SIAM J. Comput. **31**(2), 353–363 (Feb 2002). <https://doi.org/10.1137/S0097539700369909>, <https://doi.org/10.1137/S0097539700369909>
37. Pan, J., Qian, C., Ringerud, M.: Signed (group) Diffie-Hellman key exchange with tight security. Journal of Cryptology **35**(4), 26 (Oct 2022). <https://doi.org/10.1007/s00145-022-09438-y>
38. Pan, J., Riepel, D., Zeng, R.: Key exchange with tight (full) forward secrecy via key confirmation. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part VII. LNCS, vol. 14657, pp. 59–89. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58754-2_3
39. Pan, J., Wagner, B.: Lattice-based signatures with tight adaptive corruptions and more. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) PKC 2022, Part II. LNCS, vol. 13178, pp. 347–378. Springer, Cham (Mar 2022). https://doi.org/10.1007/978-3-030-97131-1_12
40. Pan, J., Wagner, B., Zeng, R.: Lattice-based authenticated key exchange with tight security. In: Handschuh, H., Lysyanskaya, A. (eds.) CRYPTO 2023, Part V. LNCS, vol. 14085, pp. 616–647. Springer, Cham (Aug 2023). https://doi.org/10.1007/978-3-031-38554-4_20

41. Pan, J., Wagner, B., Zeng, R.: Tighter security for generic authenticated key exchange in the QROM. In: Guo, J., Steinfeld, R. (eds.) ASIACRYPT 2023, Part IV. LNCS, vol. 14441, pp. 401–433. Springer, Singapore (Dec 2023). https://doi.org/10.1007/978-981-99-8730-6_13
42. Pollard, J.M.: A monte carlo method for factorization. BIT **15**(3), 331–334 (Sep 1975). <https://doi.org/10.1007/BF01933667>, <https://doi.org/10.1007/BF01933667>
43. Saito, T., Xagawa, K., Yamakawa, T.: Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part III. LNCS, vol. 10822, pp. 520–551. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78372-7_17
44. Schwabe, P., Avanzi, R., Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Seiler, G., Stehlé, D.: CRYSTALS-KYBER. Tech. rep., National Institute of Standards and Technology (2020), available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>
45. Wang, Y., Matsuda, T., Hanaoka, G., Tanaka, K.: Memory lower bounds of reductions revisited. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part I. LNCS, vol. 10820, pp. 61–90. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78381-9_3
46. Xagawa, K.: Signatures with memory-tight security in the quantum random oracle model. In: Joye, M., Leander, G. (eds.) EUROCRYPT 2024, Part VII. LNCS, vol. 14657, pp. 30–58. Springer, Cham (May 2024). https://doi.org/10.1007/978-3-031-58754-2_2

Supporting Material

A Instantiations

We give two lattice-based instantiations of **aSigKEM**. The first instantiation is based on the lattice-based parameter-lossy encryption [41] and the memory-tight signature scheme proposed in [46]. They are based on hardness problems over lattices (LWE and SIS). This instantiation provides a tighter security bound. The second one combines the Kyber KEM [44] and the Dilithium signature scheme [35]. Both schemes are based on hardness problems over module lattices (Module-LWE and Module-SIS), and they are selected algorithms in the NIST PQC Standardization process.

To illustrate our results on memory-tightness of reductions, in this section, we only provide asymptotic bounds. Let ϵ_{LWE} and ϵ_{SIS} denote the computational bounds (i.e., the probabilities of breaking the assumptions) for the LWE and SIS problems, respectively. Similarly, let m_{LWE}^* and m_{SIS}^* represent the memory bounds required to break these problems. For a cryptographic scheme Π , we similarly write ϵ_{Π} and m_{Π}^* to denote the computational bounds of breaking Π and the reduced memory bounds required to break Π , respectively.

INSTANTIATION BASED ON LATTICES. We use two lattice-based schemes to instantiate **aSigKEM**. The first one is the lattice-based parameter-lossy encryption, denoted as PLE_{LWE} (cf. [41, Figure 2]), which is proposed in [41]. By [41, Lemma 3.5, 3.6, and 3.7], we have

$$\epsilon_{\text{PLE}_{\text{LWE}}} \leq O(\lambda) \cdot \epsilon_{\text{LWE}} + \text{negl}(\lambda).$$

Although the authors of [41] do not study the memory-tightness of their reduction and the relation between parameter-lossy encryption and IND-CPA security, we find that: (1) The reduction given in [41, Lemma 3.6] can easily be adapted to memory-tight, since we only need to maintain the parameter, and all key pairs can be simulated by a (pseudo-)random function. (2) Parameter-lossy encryption tightly implies IND-CPA security in the multi-user, multi-challenge setting. Therefore, we can apply Theorem 3 to PLE_{LWE} and get a lattice-based OW-PCA secure scheme (denoted as KEM_{LWE}) such that

$$\epsilon_{\text{KEM}_{\text{LWE}}} \leq O(\lambda) \cdot \epsilon_{\text{LWE}} + \text{negl}(\lambda) \quad \text{and} \quad m_{\text{KEM}_{\text{LWE}}}^* \approx m_{\text{LWE}}^*.$$

Then we discuss how to instantiate the signature part of **aSigKEM**. In [46], Xagawa proposes a lattice-based signature, denoted as SIG_{xag} , with a memory-tight reduction such that

$$\epsilon_{\text{SIG}_{\text{xag}}}^{\text{sEUF-1CMA}} \leq O(1) \cdot \epsilon_{\text{LWE}} + \epsilon_{\text{SIS}} + \text{negl}(\lambda) \quad \text{and} \quad m_{\text{SIG}_{\text{xag}}}^* \approx m_{\text{LWE}}^* \approx m_{\text{SIS}}^*.$$

By applying Lemma 1, we have

$$\epsilon_{\text{SIG}_{\text{xag}}}^{\text{MU}^c\text{-sEUF-1CMA}} \leq O(Q_{\text{pk}}) \cdot (\epsilon_{\text{LWE}} + \epsilon_{\text{SIS}}) + \text{negl}(\lambda) \quad \text{and} \quad m_{\text{SIG}_{\text{xag}}}^* \approx m_{\text{LWE}}^* \approx m_{\text{SIS}}^*,$$

where the ϵ here denotes the advantage against $\text{MU}^c\text{-sEUF-1CMA}$ security (cf. Definition 1).

Finally, by instantiating **aSigKEM** with KEM_{LWE} and SIG_{xag} and Theorem 1, one can obtain a lattice-based AKE protocol (denoted as AKE_{lat}) such that

$$\epsilon_{\text{AKE}_{\text{lat}}} \leq O(N_{\text{user}} \cdot \lambda) \cdot (\epsilon_{\text{LWE}} + \epsilon_{\text{SIS}}) + \text{negl}(\lambda) \\ m_{\text{AKE}_{\text{lat}}}^* + O(N_{\text{user}} \cdot \lambda) \approx m_{\text{LWE}}^* \approx m_{\text{SIS}}^*,$$

where λ is the security parameter and Q_{pk} is the number of challenge public keys. Without loss of generality, we assume that $Q_{\text{pk}} = N_{\text{user}}$. Namely, the adversary does not issue repeated queries to challenge public key oracle and the number of queries equals to the number of users.

INSTANTIATION BASED ON MODULE LATTICES. We discuss how to use Crystal-Kyber and Crystal-Dilithium to instantiate **aSigKEM**. Note that in this instantiation, we view both Crystal-Kyber and Crystal-Dilithium as two secure underlying building blocks and do not consider their underlying bounds of MLWE and MSIS. Let ϵ_{kyber} and m_{kyber}^* be the computational bound and required (reduced) memory for breaking the CPA PKE scheme in Crystal-Kyber. Similarly, let ϵ_{dilit} and m_{dilit}^* be the computational bound and required (reduced) memory for breaking Crystal-Dilithium.

We start by considering the IND-CPA PKE scheme `KYBER.CPAPKE`. By [44, Section 4.3.1], `KYBER.CPAPKE` has IND-CPA security in the single-user, single-challenge setting. By using the generic construction in Figure 13 and applying Corollary 1, we transform `KYBER.CPAPKE` into a KEM scheme, denoted as $\text{KEM}_{\text{kyber}}^{\text{ow}}$, that has OW-PCA security in the single-user, single-challenge setting. By applying Lemma 4, we lift its OW-PCA security to the multi-user, multi-challenge setting such that

$$\epsilon_{\text{KEM}_{\text{kyber}}^{\text{ow}}} \leq O(Q_{\text{pk}} \cdot Q_{\text{CT}}) \cdot \epsilon_{\text{kyber}} \quad \text{and} \quad m_{\text{KEM}_{\text{kyber}}^{\text{ow}}}^* \approx m_{\text{kyber}}^*.$$

where Q_{pk} and Q_{CT} denotes the numbers of KEM public keys and KEM ciphertexts, respectively.

Then, we consider the signature scheme `Dilithium` in [35]. As discussed in [35, Section 6], `Dilithium` achieves the sEUF-CMA security in the single-user setting, which trivially implies the sEUF-1CMA security (cf. Definition 1). By applying Lemma 1, one can show that `Dilithium` achieves the $\text{MU}^c\text{-sEUF-1CMA}$ security such that

$$\epsilon_{\text{Dilithium}}^{\text{MU}^c\text{-sEUF-1CMA}} \leq O(Q_{\text{pk}}) \cdot \epsilon_{\text{dilit}} \quad \text{and} \quad m_{\text{Sign}_{\text{dilithium}}^{\text{mu}}}^* \approx m_{\text{dilit}}^*.$$

Finally, we use $\text{KEM}_{\text{kyber}}^{\text{ow}}$ and $\text{Sign}_{\text{dilithium}}$ to instantiate `aSigKEM` and denote the resulting AKE protocol as AKE_{mlat} . By applying Theorem 1, we have

$$\begin{aligned} \epsilon_{\text{AKE}_{\text{mlat}}} &\leq O(N_{\text{user}} \cdot N_{\text{ses}}) \cdot \epsilon_{\text{kyber}} + O(N_{\text{user}}) \cdot \epsilon_{\text{dilit}}, \\ m_{\text{AKE}_{\text{mlat}}}^* &\approx m_{\text{kyber}}^*, \quad \text{and} \quad m_{\text{AKE}_{\text{mlat}}}^* + O(N_{\text{user}} \cdot \lambda) \approx m_{\text{dilit}}^*. \end{aligned}$$

Although our MLWE-based AKE protocol does not have a fully tight reduction, its memory-tightness only has a loss factor proportional to the number of user (and the size of the security parameter) and thus has a better memory-efficient reduction than the previous AKE constructions that based on module lattices (e.g., [16,17]).

B More Detailed Explanations of Our New AKE Model

In this section, we discuss several motivations behind the definitions of our AKE model `ind-wFS-KCI`. IN Supp. Mat. C, we also compare our model with prior models.

REPEATED QUERIES TO PK. When $\mathcal{A}$ queries $\text{PK}(u \in \mathcal{U})$, the oracle returns pk_u . If $\mathcal{A}$ queries PK repeatedly on the same user identifier, then the same public key is returned. This method of modelling public key generation is proposed by [20,30]. It allows security reductions to simulate all public key in a memory-tight manner and does not change the adversary's capabilities.

ON THE DEFINITION OF SESSION IDENTIFIERS. We first give a brief review on two previous definitions of session identifiers in [16,31,40]. In [16], session identifiers are defined as protocol messages. For instance, if u sent M_u to v and received M_v , then this session is identified as (u, v, M_u, M_v) . While in [31,40], sessions identifiers are defined as an internal counter controlled by the simulator. Every query to oracles SESSION_I and SESSION_R will increment this counter and use it as a session identifier. This counter can be viewed as a sequence number, with each session assigned a unique value. In Remark 4, we discuss why this approach is unlikely to be memory-tight.

Our definition of session identifiers is a combination of these two approaches, namely, it consists of counter numbers and the protocol messages. For an initiator session, its identifier is of the format

$$\text{sID} := (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v), \quad (\text{Initiator session})$$

where seq_1 and seq_2 are the initial and final sequence numbers, u and v are identities of initiator and responder, and M_u and M_v are protocol transcripts. Correspondingly, the identifier of an responder session is defined as

$$\text{sID} := (\text{seq}, u, v, M_u, M_v), \quad (\text{Responder session})$$

where seq is the sequence number. Since a responder session produces a session key when it receives the protocol message, here we just need one sequence number to define the session identifier of a responder session.

Now we explain why our definition includes both sequence numbers and protocol messages:

- On the one hand, if we only use sequence numbers (more concretely, a reduction-controlled counter or random number) to define session identifiers (as [31,40] did), then we may have to record session keys to simulate REV and TEST. This is because such sequence numbers cannot record the received messages of session without using lists if such messages are generated by the adversary and not reduction-controlled.
- On the other hand, if we only use protocol messages to define session identifiers (as [16] did), then we may have to remember all established sessions identifiers. This is because the adversary can use protocol messages to forge a session identifier.

For example, the initiator session identifiers in our model include two sequence numbers. The first one, seq_1 , is used to identify an incomplete initiator session (since there is no responder protocol messages yet). The second one, seq_2 , is used to identify the complete session along with seq_1 . Such definition helps us to exclude the case that the adversary tries querying any initiator session that does not exist. For example, if we only use seq_1 to identify this session, then we have to remember the history that if such session has been finished. Note that our definition requires that if the adversary sends repeating queries to DER_I , then the oracle outputs same seq_2 .

Furthermore, including protocol messages in the session identifier makes it possible to recover a session key in a memory-tight way without recording. Intuitively, this definition forces the adversary to remember all protocol messages, effectively shifting the memory usage of recording these messages onto the adversary.

MODELLING ATTACKS AND SECURITY. Oracles SESSION_I , SESSION_R , and DER_I are used to model replay attacks and man-in-the-middle (MiM) attacks by allowing adversaries to fully control the transmission of all protocol messages. We also allow the adversary to reveal any established session key by querying the oracle REV.

To capture key-compromise impersonation (KCI) attacks and weak forward-secrecy (wFS), we additionally allow an adversary to corrupt the long-term secret key of any user by querying COR. Roughly speaking, to launch a KCI attack, $\mathcal{A}$ firstly corrupts a target user and then impersonate another user to this target user. To launch a wFS attack against an passive session, an adversary can honestly forwards all protocol messages of this session and then corrupts the long-term secret keys of this session’s participants. All corrupted users are recorded in the set $\mathcal{S}_{\text{cor}}$. Together with oracles SESSION_I , SESSION_R , and DER_I , an adversary can easily perform KCI or wFS attacks.

We define *freshness* and *validity* to exclude all trivial attacks:

- Revealing a session key via REV makes the attack against this session trivial. All sessions revealed by querying REV are recorded in the set $\mathcal{S}_{\text{rev}}$. Moreover, if a session is revealed, then all matching sessions of this session is considered revealed (cf. Definition 5). In this model, we use an auxiliary procedure **Fresh** to check whether a session is fresh.
- For a responder session $\text{sID} = (\text{seq}, u, v, M_u, M_v)$, if the session initiator u is corrupted and (u, M_u) is not generated from SESSION_I , then we consider this session is trivially broken by $\mathcal{A}$. This is because, if $\mathcal{A}$ corrupted u and sends its protocol messages impersonating u , then there is no secret information in this session for security.
- For an initiator session $\text{sID} = (\text{seq}_1, \text{seq}_2, u, v, M_u, M_v)$, if the session responder v is corrupted before the initiator u receives M_v and (u, v, M_u, M_v) is not in $\mathcal{S}_{\text{msg}}$ (which means that sID does not have a matching session), then we consider this session is trivially broken.

The models in [31,40,41] use a table to list all valid attacks. Here, we use an auxiliary procedure **Valid** to check whether a session is valid. In the security proof, this allows us to be precise about the memory complexity of the reduction and to exclude unnecessary use of memory. The formal definitions of freshness and validity are given below, and the procedures for determining them are shown in Figure 3. A more detailed comparison with prior models can be found in Supp. Mat. C.

Remark 2 (Multiple (partially) matching sessions). In our model, if an adversary can construct a session with more than one matching sessions, then we consider such a session insecure. This allows us to exclude those protocols that can establish more than one matching sessions (which is not supposed to happen for a “reasonable” AKE protocol). Such an approach of defining AKE security is also used in [31,40,38].

Different from some previous AKE models (e.g., the model in [41]), our model does not consider multiple “partially” matching sessions. Here partially matching means that (for two-message AKE) a responder session has more than one initiator sessions that have the same initiator messages with this responder session. While previous models (e.g., [31,40,38]) consider such a session insecure, our model does not consider this, because, for a well-designed two-message AKE protocol, the protocol messages

should have enough randomness such that the adversary cannot predict these messages (i.e., predict the protocol messages output by Init_I). If the output of initiator algorithm Init_I has enough randomness, then the probability that a session partially matches more one other sessions is negligible. Therefore, for simplicity, our model does not consider multiple “partially” matching sessions.

Remark 3 (Reuse incomplete initiator sessions). Different from the previous models, our AKE model allows the adversary to reuse incomplete initiator sessions. For example, the adversary can first query SESSION_I to get a session sequence number seq_1 , and the query DER_I on the same seq_1 but different responder message. The oracle DER_I does not check whether the seq_1 has been used. Since our model uses protocol messages to define session, as long as the adversary sends different responder message, these queries (with same seq but different M_v) still produce different sessions. If the adversary issues repeated queries to DER_I , by lines 34 to 39, the oracle will respond the same session identity and produce the same session key (namely, repeated queries do not help the adversary).

We make this modification for the following reasons:

- If we record whether an incomplete initiator session (i.e., seq_1) has been used, then the memory-usage of this information is linear on the number of sessions (which is very large). From the perspective of constructing reduction, this memory usage seems unavoidable, because an adversary can always activate a initiator session, and the reduction has to keep the information since it does not know if the adversary will finish it later. The adversary can leave the session incomplete and force the reduction to store it during the game.
- To address this, we allow the session initiator continually responds new response, regardless of whether the session sequence number seq_1 has been used. This, on one hand, is a stronger property. For example, in our protocol, the KEM scheme should be OW-PCA secure, not OW-PCA secure with one PCO query (and similarly, the IND-one-time-CCA security is also not sufficient). On the other hand, we may understand this in this way: To achieve memory-tight reduction, the AKE protocol itself should have the ability to resist such reuse-incomplete-session attacks and our new model captures this property. If a protocol does not provide security against these attacks, then one may conclude that the memory of recording sessions provides advantage to help the adversary break the underlying building block of the protocol.

The technical details can be found in the proof of Theorem 2 and Remark 4.

C Relation between Our Model and the Previous Ones

In this section, we compare our new model ind-wFS-KCI with the two-message AKE models in [41] and [19]. Both models consider standard AKE security properties such as man-in-the-middle attacks, weak forward secrecy, key impersonation attacks, etc.. Similar to our ind-wFS-KCI , the models in [41] and [19] do not consider session state reveal and ephemeral secret reveal.

For convenience, we first unify these two models into one model using our notations, denoted as $\text{ind-wFS-KCI}'$. In this model, each session identifier sID is an positive integer. Despite the lists and variables in Section 3.2, each sID in $\text{ind-wFS-KCI}'$ is also associated to the following variables which are used to record participants and exchange messages:

- $\text{Type}[\text{sID}]$ stores the type of the session. If sID is an initiator session (resp., responder session), then $\text{Type}[\text{sID}] = I$ (resp., $\text{Type}[\text{sID}] = R$).
- $\text{Init}[\text{sID}]$ stores the user identifier of the session initiator of sID .
- $\text{Imsg}[\text{sID}]$ stores the user identifier of the session initiator of sID .
- $\text{Resp}[\text{sID}]$ stores the first protocol message (initiator message) of sID .
- $\text{Rmsg}[\text{sID}]$ stores the final protocol message (responder message) of sID .

The unified AKE game and its formal definition are presented in Figure 10 and Definition 7.

DIFFERENCE ON SYNTAX. We first highlight several differences between $\text{ind-wFS-KCI}'$ and ind-wFS-KCI in terms of syntax. More specifically,

1. In $\text{ind-wFS-KCI}'$, a session identifier is an positive integer produced by a counter cnt which increments every time a session is produced, rather than the combination of random sequence numbers and protocol messages.

Game ind-wFS-KCI'_{II}^A <pre> 00 cnt := 0 01 b ← {0, 1}, par ← Setup 02 O₁ := (SESSION_I, DER_I, SESSION_R) 03 O₂ := (PK, COR, REV, TEST) 04 b' ← A^{O₁, O₂}(par) 05 for sID* ∈ S_{test} : 06 if Fresh(sID*) = 0 : return b 07 if Valid(sID*) = 0 : return b 08 return [b' = b] Oracle SESSION_I(u, v) 09 cnt := cnt + 1, sID := cnt 10 (Init[sID], Resp[sID]) := (u, v) 11 (M_u, st) ← Init_I(pk_u, sk_u, pk_v) 12 Type[sID] := I, Msg[sID] := M_u 13 L_{st}[sID] := (u, v, M_u, st) 14 S_{msg} := S_{msg} ∪ {(u, M_u)} 15 return (sID, M_u) Oracle SESSION_R(u, v, M_u) 16 cnt := cnt + 1, sID := cnt 17 (Init[sID], Resp[sID]) := (u, v) 18 (M_v, K) ← Der_R(pk_v, sk_v, pk_u, M_u) 19 if K = ⊥ : return ⊥ 20 if (u, M_u) ∈ S_{msg} 21 S_{msg} := S_{msg} ∪ {(u, v, M_u, M_v)} 22 Type[sID] := R 23 Msg[sID] := M_u, Rmsg[sID] := M_v 24 S_{sid} := S_{sid} ∪ {sID} 25 L_{SK}[sID] := K 26 return (sID, M_v) Oracle DER_I(sID, M_v) 27 if sID ∈ S_{sid} // No session reuse 28 return ⊥ 29 S_{sid} := S_{sid} ∪ {sID} 30 (u, v, st, M_u) := L_{st}[sID] 31 if st = ⊥ : return ⊥ 32 K := Der_I(pk_u, sk_u, pk_v, st, M_v) 33 if K = ⊥ : return ⊥ 34 L_{SK}[sID] := K 35 return sID Oracle PK(u ∈ U) 36 if L_{key}[u] = ⊥ 37 L_{key}[u] := (pk, sk) ← KG 38 return pk </pre>	Oracle COR(u) <pre> 39 S_{cor} := S_{cor} ∪ {u} 40 (pk_u, sk_u) := L_{key}[u] 41 return sk_u Oracle REV(sID) 42 S_{rev} := S_{rev} ∪ {sID} 43 return L_{SK}[sID] Oracle TEST(sID) 44 if L_{SK}[sID] = ⊥ : return ⊥ 45 S_{test} := S_{test} ∪ {sID} 46 if L_{test}[sID] = ⊥ : 47 SK₀ := L_{SK}[sID], SK₁ ← SK 48 L_{test}[sID] := SK_b 49 return L_{test}[sID] Alg ComputeMS(sID) 50 (u, v) := (Init[sID], Resp[sID]) 51 (M_u, M_v) := (Msg[sID], Rmsg[sID]) 52 M(sID) := ∅ 53 for sID' ∈ S_{sid} \ {sID} 54 (u', v') := (Init[sID'], Resp[sID']) 55 (M_{u'}, M_{v'}) := (Msg[sID'], Rmsg[sID']) 56 if Type[sID] ≠ Type[sID'] 57 ∧ (u, v, M_u, M_v) = (u', v', M_{u'}, M_{v'}) 58 M(sID) := M(sID) ∪ {sID'} 59 return M(sID) Alg Fresh(sID) 60 M(sID) := ComputeMS(sID) 61 if M(sID) > 1 : return 1 62 if sID ∈ S_{rev} : return 0 63 for sID' ∈ M(sID): 64 if sID' ∈ S_{rev} : return 0 65 return 1 Alg Valid(sID) 66 M(sID) := ComputeMS(sID) 67 if M(sID) > 1 : return 1 68 (u, v) := (Init[sID], Resp[sID]) 69 (M_u, M_v) := (Msg[sID], Rmsg[sID]) 70 if Type[sID] = R // Responder session 71 if u ∉ S_{cor} ∨ (u, M_u) ∈ S_{msg} 72 return 1 73 if Type[sID] = I // Initiator session 74 if v ∉ S_{cor} ∨ (u, v, M_u, M_v) ∈ S_{msg} 75 return 1 76 return 0 </pre>
--	--

Fig. 10. Security game ind-wFS-KCI' in Definition 7 for unifying the models in [41,19]. In ind-wFS-KCI', $\mathcal{A}$ queries PK(u) to access the public key of user u , and all oracles access the key pair of user u via querying PK first and then finding the key pair from $\mathcal{L}_{\text{key}}[u]$. Conceptually, this approach is equivalent to first generating all key pairs and then providing the adversary with all the public keys, as done in [41,19].

2. Since $\text{ind-wFS-KCI}'$ does not use random sequence numbers and protocol messages to define session identifier, we need the variables defined before to record necessary session information.
3. $\text{ind-wFS-KCI}'$ does not allow initiator session reuse, while ind-wFS-KCI permits. This means $\text{ind-wFS-KCI}'$ is *conceptually weaker* than ind-wFS-KCI .

FRESHNESS AND VALIDITY. The *Fresh* and *Valid* procedures in Figure 10 capture the prior freshness and validity definitions in [41] and [19], except with the partially matching session definition defined in [41]. More specifically, the *Fresh* and *Valid* procedures in Figure 10 are equivalent to the wFS key secrecy definition in [19, Definition 3.6], and capture all possible attacks listed in [41, Table 1]. As noted in Remark 2, we do not capture multiple partially matching sessions. Note that the model in [19] also does not capture it.

Definition 7. Let $\Pi := (\text{Setup}, \text{KG}, \text{Init}_I, \text{Der}_R, \text{Der}_I)$ be an AKE protocol and consider the game $\text{ind-wFS-KCI}'$ defined in Figure 10. We say AKE is secure under the model $\text{ind-wFS-KCI}'$ if, for all PPT adversaries $\mathcal{A}$, the following advantage is negligible on the security parameter λ :

$$\text{Adv}_{\Pi}^{\text{ind-wFS-KCI}'}(\mathcal{A}) := \left| \Pr \left[\text{ind-wFS-KCI}'_{\Pi}^{\mathcal{A}} \Rightarrow 1 \right] - 1/2 \right|$$

Now we are ready to show that ind-wFS-KCI is *not weaker* than $\text{ind-wFS-KCI}'$, namely, if an AKE protocol Π is secure in the ind-wFS-KCI model, then it is also secure in the $\text{ind-wFS-KCI}'$ model. We prove this via contradiction: For any adversary $\mathcal{A}$, if $\mathcal{A}$ breaks Π in the ind-wFS-KCI model, then we can construct another adversary to break Π in the $\text{ind-wFS-KCI}'$ model. The formal statement is given in Theorem 2.

Theorem 2 ($\text{ind-wFS-KCI} \implies \text{ind-wFS-KCI}'$). *Let Π be an two-message AKE protocol. For any adversary $\mathcal{A}'$ that breaks the $\text{ind-wFS-KCI}'$ security of Π , there exists an adversary $\mathcal{A}$ that breaks the ind-wFS-KCI security of Π .*

More specifically, if $\mathcal{A}'$ issues $Q_{\text{pk}}, Q_{\text{rev}}, Q_{\text{cor}}, Q_{\text{test}}, Q_{\text{H}}$, and Q_{send} queries to the oracles PK (including direct and indirect queries, i.e., queries from reductions), REV, COR, TEST, H, and $(\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R)$ in $\text{ind-wFS-KCI}'$, respectively, then there exists an adversary $\mathcal{A}$ such that

$$\begin{aligned} \text{Adv}_{\Pi}^{\text{ind-wFS-KCI}}(\mathcal{A}) &= \text{Adv}_{\Pi}^{\text{ind-wFS-KCI}'}(\mathcal{A}'), \\ \text{Time}(\mathcal{A}) &\approx \text{Time}(\mathcal{A}'), \end{aligned}$$

where $\mathcal{A}$ issues $Q_{\text{pk}}, Q_{\text{rev}}, Q_{\text{cor}}, Q_{\text{test}}$, and Q_{send} queries to the oracles PK, REV, COR, TEST, and $(\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R)$ in ind-wFS-KCI , respectively.

Theorem 2 states the relation between ind-wFS-KCI and $\text{ind-wFS-KCI}'$ in terms of time complexity and winning advantage. The reduction in Theorem 2 is not memory-tight (and is also unlikely to be memory-tight, as explained in Remark 4), but the goal of Theorem 2 is to show ind-wFS-KCI captures the security goals in $\text{ind-wFS-KCI}'$ and thus the security established by our model is not weaker than the previous ones.

Proof (Theorem 2). We construct a straightforward adversary $\mathcal{A}$ that simply forwards all queries from $\mathcal{A}'$ to the ind-wFS-KCI game (via adapting the format of session identifiers). The construction of $\mathcal{A}$ is presented in Figure 11. It simulates $\text{ind-wFS-KCI}'$ in the following way:

- $\mathcal{A}$ uses a counter cnt to simulate the session identifiers sID' in $\text{ind-wFS-KCI}'$ and uses an internal (i.e., not accessible by $\mathcal{A}'$) list $\mathcal{L}_{\text{seq-sid}}$ to maintain the session identifiers sID' in $\text{ind-wFS-KCI}'$ and the session identifiers sID in ind-wFS-KCI . When $\mathcal{A}$ issues a query about sID' , $\mathcal{A}$ finds the corresponding sID from $\mathcal{L}_{\text{seq-sid}}$ and forwards the query to ind-wFS-KCI .
- The freshness and validity definitions of $\text{ind-wFS-KCI}'$ are equivalent to those in ind-wFS-KCI , so $\mathcal{A}$ only needs to find the corresponding session identifiers and forwards all queries to ind-wFS-KCI . Since all session-related information (protocol messages, revealed sessions, corrupted parties, etc) has been handled in ind-wFS-KCI , when simulating $\text{ind-wFS-KCI}'$, $\mathcal{A}$ does not need to store this information.

Reduction $\mathcal{A}^{O_1, O_2}(\text{par})$ 00 $\text{cnt} := 0, \mathcal{L}_{\text{seq-sid}} := \emptyset$ // To associate the session identifiers of ind-wFS-KCI and ind-wFS-KCI'. 01 $O_1' := (\text{SESSION}_I', \text{DER}_I', \text{SESSION}_R')$ 02 $O_2' := (\text{PK}', \text{COR}', \text{REV}', \text{TEST}')$ 03 $b' \leftarrow \mathcal{A}^{O_1', O_2'}(\text{par})$ 04 return b' Oracle $\text{SESSION}_I'(u, v)$ 05 $\text{cnt} := \text{cnt} + 1, \text{slD}' := \text{cnt}$ 06 $(\text{seq}_1, M_u) \leftarrow \text{SESSION}_I(u, v)$ 07 $\mathcal{L}_{\text{seq-sid}} := \mathcal{L}_{\text{seq-sid}} \cup \{(u, v, \text{seq}_1), \text{slD}'\}$ 08 return (slD', M_u) Oracle $\text{DER}_I'(\text{slD}', M_v)$ 09 Find $((u, v, \text{seq}_1), \text{slD}') \in \mathcal{L}_{\text{seq-sid}}$ 10 if $(u, v, \text{seq}_1) = \perp$: return $\perp$ // No records 11 else Remove $((u, v, \text{seq}_1), \text{slD}')$ from $\mathcal{L}_{\text{seq-sid}}$ // Ensure that the session will not be reused 12 $\text{slD} \leftarrow \text{DER}_I(u, v, \text{seq}_1, M_v)$ 13 if $\text{slD} = \perp$: return $\perp$ 14 $\mathcal{L}_{\text{seq-sid}} := \mathcal{L}_{\text{seq-sid}} \cup \{(\text{slD}, \text{slD}')\}$ 15 return slD'	Oracle $\text{PK}'(u)$ 16 return $\text{PK}(u)$ Oracle $\text{COR}'(u)$ 17 return $\text{COR}(u)$ Oracle $\text{REV}'(\text{slD}')$ 18 Find slD s.t. $(\text{slD}, \text{slD}') \in \mathcal{L}_{\text{seq-sid}}$ 19 return $\text{REV}(\text{slD})$ Oracle $\text{TEST}'(\text{slD}')$ 20 Find slD s.t. $(\text{slD}, \text{slD}') \in \mathcal{L}_{\text{seq-sid}}$ 21 return $\text{TEST}(\text{slD})$ Oracle $\text{SESSION}_R'(u, v, M_u)$ 22 $\text{cnt} := \text{cnt} + 1, \text{slD}' := \text{cnt}$ 23 $\text{slD} \leftarrow \text{SESSION}_R(u, v, M_u)$ 24 if $\text{slD} = \perp$: return $\perp$ 25 $(\text{seq}, u, v, M_u, M_v) := \text{slD}$ // The format of the re- sponder sid, cf. line 24 26 $\mathcal{L}_{\text{seq-sid}} := \mathcal{L}_{\text{seq-sid}} \cup \{(\text{slD}, \text{slD}')\}$ 27 return (slD', M_v)
---	---

Fig. 11. Reduction $\mathcal{A}$ for proving Theorem 2. $\mathcal{A}$ plays ind-wFS-KCI and simulates ind-wFS-KCI' for the adversary $\mathcal{A}'$. It has access to oracles $O_1 := (\text{SESSION}_I, \text{DER}_I, \text{SESSION}_R)$ and $O_2 := (\text{PK}, \text{COR}, \text{REV}, \text{TEST})$. Since the oracles of ind-wFS-KCI' share the same names with ind-wFS-KCI, we use “'” to mark all oracles in ind-wFS-KCI', and use slD' to distinguish from the session identifiers slD in ind-wFS-KCI. The highlighted code presents how $\mathcal{A}$ uses the oracles from ind-wFS-KCI to simulate ind-wFS-KCI'.

It is straightforward to show that $\mathcal{A}$ perfectly simulates ind-wFS-KCI' for $\mathcal{A}'$, so the challenge bit used in ind-wFS-KCI is also the one (implicitly) used in ind-wFS-KCI'. Therefore, we have

$$\text{Adv}_{\Pi}^{\text{ind-wFS-KCI}}(\mathcal{A}) = \text{Adv}_{\Pi}^{\text{ind-wFS-KCI}'}(\mathcal{A}') \text{ and } \text{Time}(\mathcal{A}) \approx \text{Time}(\mathcal{A}'),$$

where the equation on time complexity is from the fact that $\mathcal{A}$ can search slD in $\mathcal{L}_{\text{seq-sid}}$ via some efficient search algorithms (e.g., binary search in ordered list using slD' as index). This concludes the proof of Theorem 2.

Remark 4 (Why prior models are unlikely to be memory-efficient). The proof of Theorem 2 also reveals the intuition on why it is hard to construct memory-tight reductions under the ind-wFS-KCI' model. In ind-wFS-KCI', the session identifiers are defined as successive positive numbers. Since this structure is known, the AKE adversary can easily predict the session identifiers, so we cannot embed session information into them (and then use the technique in Section 4). This is why ind-wFS-KCI uses a random sequence number as a part of a session identifier.

Moreover, in ind-wFS-KCI', the reduction needs to check whether an initiator session has been used, which is also essentially an dictionary problem discussed in Remark 1. This is why our new model ind-wFS-KCI allows initiator session reuse, otherwise it will incur an inherent loss $\Omega(N_{\text{ses}})$ in memory.

D Deferred Proofs and Constructions

D.1 Generic Construction from IND-CPA PKE to OW-PCA KEM

Before showing our generic construction, we first recall the syntax of public key encryption and its security definition.

Definition 8 (PKE Correctness). Let $\text{PKE} := (\text{Setup}, \text{KG}, \text{Enc}, \text{Dec})$ be a PKE scheme. We say PKE is ρ -correct if, for all adversaries (that can be unbounded) $\mathcal{A}$, we have

$$\Pr[\text{Dec}(\text{sk}, \text{Enc}(\text{pk}, m)) = m : m \leftarrow \mathcal{A}(\text{pk}, \text{sk})] \geq \rho,$$

where the probability is taken over $\text{par} \leftarrow \text{Setup}$, $(\text{pk}, \text{sk}) \leftarrow \text{KG}$, and the randomness of Enc . When using this definition in security proof, we may let $\delta := 1 - \rho$ and use δ to bound the probability of decryption error.

Definition 9 (IND-CPA Security of PKE). Let $\text{PKE} := (\text{Setup}, \text{KG}, \text{Enc}, \text{Dec})$ be a PKE scheme. Consider the game IND-CPA defined in Figure 12. We say PKE is IND-CPA secure if, for all PPT adversaries $\mathcal{A}$, the following advantage is negligible on the security parameter:

$$\text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) := \left| \Pr \left[\text{IND-CPA}_{\text{PKE},0}^{\mathcal{A}} \Rightarrow 1 \right] - \Pr \left[\text{IND-CPA}_{\text{PKE},1}^{\mathcal{A}} \Rightarrow 1 \right] \right|$$

Game $\text{IND-CPA}_{\text{PKE},b}$	Oracle $\text{CHALPK}(u)$	Oracle $\text{CHALCT}(u, i, m)$
00 $\text{par} \leftarrow \text{Setup}$	03 if $\text{pk}[u] = \perp$:	06 if $\text{c}[u, i] \neq \perp$: return $\text{c}[u, i]$
01 $b' \leftarrow \mathcal{A}^{\text{CHALPK}, \text{CHALCT}}(\text{par})$	04 $(\text{pk}[u], \text{sk}[u]) \leftarrow \text{KG}$	07 $m_0 := m, m_1 := 0^{ m }$
02 return b'	05 return $\text{pk}[u]$	08 $c \leftarrow \text{Enc}(\text{CHALPK}(u), m_b)$
		09 $\text{c}[u, i] := c$
		10 return $\text{c}[u, i]$

Fig. 12. Security game for defining IND-CPA security. We implicitly assume that all user identifiers are belong to $\mathcal{U}$.

CONSTRUCTION. Let $\text{PKE} := (\text{Setup}, \text{KG}, \text{Enc}, \text{Dec})$ be a PKE scheme with message space $\mathcal{M}$, randomness space $\mathcal{R}$, and ciphertext space $\mathcal{C}$. Let ℓ and l be some integers. Let $\text{G} : \mathcal{M} \rightarrow \mathcal{R}$ and $\text{H} : \{0, 1\}^\ell \times \{0, 1\}^l \times \mathcal{C} \rightarrow \mathcal{K}$ be two hash functions (that will be modeled as random oracles in the security proof). We construct a generic construction in Figure 13.

Alg KG	Alg Encaps((pk, α))	Alg Decaps((sk, s, α), c)
00 $(\text{pk}, \text{sk}) \leftarrow \text{PKE.KG}$	05 $m \leftarrow \mathcal{M}$	10 $(\text{sk}, \text{s}, \alpha) := \text{sk}_{\text{kem}}$
01 $\alpha \leftarrow \{0, 1\}^\ell, \text{s} \leftarrow \{0, 1\}^l$	06 $r := \text{G}(m)$	11 $m' := \text{Dec}(\text{sk}, c)$
02 $\text{pk}_{\text{kem}} := (\text{pk}, \alpha)$	07 $c := \text{Enc}(\text{pk}, m; r)$	12 if $m' = \perp \vee \text{Enc}(\text{pk}, m; \text{G}(m))$
03 $\text{sk}_{\text{kem}} := (\text{sk}, \text{s}, \alpha)$	08 $k := \text{H}(\alpha, m, c)$	13 $k := \text{H}'(\alpha, \text{s}, c)$
04 return $(\text{pk}_{\text{kem}}, \text{sk}_{\text{kem}})$	09 return (c, k)	14 else $k := \text{H}(\alpha, m, c)$
		15 return k

Fig. 13. A generic KEM construction KEM based on a PKE scheme PKE. The setup algorithm KEM.Setup is the same as the setup algorithm of PKE except that the parameter of KEM also includes ℓ and l .

Theorem 3. Let KEM be the KEM construction presented in Figure 13. If the PKE scheme PKE is $(1 - \delta)$ -correct, then for any adversary $\mathcal{A}$ against the OW-PCA security of KEM, there exists an adversary $\mathcal{B}$ such that

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{OW-mucPCA}}(\mathcal{B}) &\leq \text{Adv}_{\text{PKE}}^{\text{IND-mucCPA}}(\mathcal{A}) + N' \cdot \delta + \frac{(N' + 1)(1 + Q_{\text{PCO}} + Q_{\text{H}'}) + N'^2}{2^l} \\ &\quad \frac{1 + Q_{\text{PCO}} + Q_{\text{H}} + (Q_{\text{ENC}} + Q_{\text{H}})^2}{|\mathcal{K}|} + \frac{|\mathcal{U}| \cdot (Q_{\text{G}} + Q_{\text{H}}) + (Q_{\text{CT}} + 1)^2}{|\mathcal{M}|} \\ \text{Time}^*(\mathcal{B}) &= \text{Time}^*(\mathcal{A}) + \text{O}(Q) + \text{O}(Q_{\text{H}}) \cdot \text{Time}(\text{PKE}) \\ \text{Mem}^*(\mathcal{B}) &= \text{Mem}^*(\mathcal{A}) + \log |\mathcal{U}| + \log |\mathcal{I}| + \text{O}(\text{Mem}(\text{PKE})), \end{aligned}$$

where $\text{Time}(\text{PKE})$ and $\text{Mem}(\text{PKE})$ are the running time and memory usage of one execution of PKE, respectively, and $\mathcal{A}$ issues at most Q_{pk} (including direct or indirect queries), $Q_{\text{CT}}, Q_{\text{G}}, Q_{\text{H}}$, and $Q_{\text{H}'}$ queries to oracles $\text{CHALPK}, \text{CHALCT}, \text{G}, \text{H}$, and H' , respectively. Q is the total number of queries issued by $\mathcal{A}$. $N' \leq Q_{\text{pk}}$ is the number of non-repeated (including direct and indirect) queries to CHALPK .

We combine techniques from IND-CCA proof in the QROM [32,43] and memory-tight PKE constructions in [30] to prove Theorem 3.

COMPARISON WITH THE RESULT IN [30, SECTION 5.2] Jaeger et al. [30] proves that the T transformation [27] has a memory-tight reduction. Since the T transformation converts an IND-CPA PKE into an OW-PCA PKE, one may argue that using this result immediately gives us a OW-PCA KEM. Unfortunately, in

Game G_0-G_7		Oracle $\text{PCO}(u, c, k)$	
00 $f_{\text{kg}} \leftarrow \text{Func}(\mathcal{U}, \mathcal{R}_{\text{ekg}})$	// G_1 - G_7	29 $\text{CHALPK}(u)$	// G_1 - G_3
01 $f_{\text{prfk}} \leftarrow \text{Func}(\mathcal{U}, \{0, 1\}^l)$	// G_1 - G_7	30 $k' := \text{Decaps}((\text{sk}_u, s_u, \alpha_u), c)$	// G_1 - G_3
02 $f_g \leftarrow \text{Func}(\mathcal{M}, \mathcal{R}_{\text{enc}})$	// G_1 - G_7	31 if $k' = k$: return 1	// G_1 - G_3
03 $f_h \leftarrow \text{Func}(\mathcal{M} \times \mathcal{C}, \mathcal{K})$	// G_1 - G_7	32 $k'_0 = f_{\text{Hkey}}(\alpha_u, c)$	// G_4 - G_7
04 $f_{\tilde{h}} \leftarrow \text{Func}(\{0, 1\}^l \times \mathcal{C}, \mathcal{K})$	// G_1 - G_7	33 $k'_1 = H'(\alpha_u, s_u, c)$	// G_4 - G_7
05 $(h_{\text{msg}}, h_{\text{msg}}^{-1}) \leftarrow \text{InjF}(\mathcal{U} \times \mathcal{I}, \mathcal{M})$	// G_1 - G_7	34 if $k = k'_0 \vee k = k'_1$: return 1	// G_4 - G_7
06 $(h_{\text{tag}}, h_{\text{tag}}^{-1}) \leftarrow \text{InjF}(\mathcal{U}, \{0, 1\}^\ell)$	// G_1 - G_7	35 return 0	
07 $f_{\text{Hkey}} \leftarrow \text{Func}(\mathcal{U} \times \mathcal{C}, \mathcal{K})$	// G_2 - G_7	Oracle $G(m)$	
08 $f_{\text{msg}} \leftarrow \text{Func}(\mathcal{U} \times \mathcal{I}, \mathcal{R})$	// G_5 - G_6	36 if $\mathcal{L}_G[m] = \perp$	// G_0
09 par $\leftarrow \text{Setup}$		37 $\mathcal{L}_G[m] := r \leftarrow \mathcal{R}$	// G_0
10 $O := (\text{CHALPK}, \text{CHALCT}, \text{PCO})$		38 return $\mathcal{L}_G[m]$	// G_0
11 $(u^*, i^*, k^*) \leftarrow \mathcal{A}^O(\text{par})$		39 if $\perp \neq (u, i) := h_{\text{msg}}^{-1}(m)$	// G_5 - G_7
12 return $\text{PCO}(u^*, \text{CHALCT}(u^*, i^*), k^*)$		40 return $f_{\text{msg}}(u, i)$	// G_5
Oracle $\text{CHALPK}(u)$		41 abort and output 1	// G_6 - G_7
13 if $\text{pk}[u] = \perp$	// G_0	42 return $f_g(m)$	// G_1 - G_7
14 $\alpha_u \leftarrow \{0, 1\}^\ell$	// G_0	Oracle $H(\alpha, m, c)$	
15 $(\text{pk}[u], \text{sk}[u]) := (\text{pk}_u, \text{sk}_u) \leftarrow \text{KG}$	// G_0	43 if $\mathcal{L}_H[m, c] = \perp$	// G_0
16 $(\text{pk}_u, \text{sk}_u) := \text{PKE.KG}(\cdot; f_{\text{kg}}(u))$	// G_1 - G_7	44 $\mathcal{L}_H[m, c] := k \leftarrow \mathcal{K}$	// G_0
17 $\alpha_u := h_{\text{tag}}(u), s_u := f_{\text{prfk}}(u)$	// G_1 - G_7	45 return $\mathcal{L}_H[m, c]$	// G_0
18 return (pk_u, α_u)		46 if $h_{\text{msg}}^{-1}(m) \neq \perp$	// G_6 - G_7
Oracle $\text{CHALCT}(u, i)$		47 abort and output 1	// G_6 - G_7
19 if $c[u, i] \neq \perp$: return $c[u, i]$	// G_0	48 if $\perp \neq u := h_{\text{tag}}^{-1}(\alpha)$	// G_2 - G_7
20 $m \leftarrow \mathcal{M}$	// G_0	49 if $c = \text{Enc}(\text{pk}_u, m; G(m))$	// G_2 - G_7
21 $m := h_{\text{msg}}(u, i)$	// G_1 - G_5	50 return $f_{\text{Hkey}}(u, c)$	// G_2 - G_7
22 $c := \text{Enc}(\text{pk}_u, m; G(m))$	// G_0 - G_4	51 return $f_h(\alpha, m, c)$	// G_1 - G_7
23 $c := \text{Enc}(\text{pk}_u, m; f_{\text{msg}}(u, i))$	// G_5 - G_6	Oracle $H'(\alpha, s, c)$	
24 $c := \text{Enc}(\text{pk}_u, 0^{ m }; f_{\text{msg}}(u, i))$	// G_7	52 if $\mathcal{L}_{H'}[\alpha, s, c] = \perp$	// G_0
25 $k := H(\alpha_u, m, c)$	// G_0 - G_2	53 $\mathcal{L}_{H'}[\alpha, s, c] := k \leftarrow \mathcal{K}$	// G_0
26 $k := f_{\text{Hkey}}(u, c)$	// G_3 - G_7	54 return $\mathcal{L}_{H'}[\alpha, s, c]$	// G_0
27 $c[u, i] := c$	// G_0	55 return $f_h(\alpha, s, c)$	// G_1 - G_7
28 return c			

Fig. 14. Security game for defining OW-PCA security. We implicitly assume that all user identifiers are belong to $\mathcal{U}$.

their proof, the security bound includes not only the term $\frac{|\mathcal{U}| \cdot Q_{\text{ro}}}{|\mathcal{M}|}$ but also an additive term $|\mathcal{U}| \cdot \delta_{\text{KEM}}$. In Crystals-Kyber512 [44], its size of $\mathcal{M}$ is 2^{256} , but its δ_{KEM} is only around 2^{-139} (cf. [44, Table 1]). When we apply their result in the AKE context, a PKE user will correspond to an AKE session. Specifically, in the AKE proof, the $|\mathcal{U}|$ of KEM is N in Theorem 1. N is larger than the number of sessions and can be more than 2^{128} . This can make Jaeger et al.'s result insufficient. This is why we propose another OW-PCA KEM construction with memory-tight proof. In our proof, the bound is $|N'| \cdot \delta_{\text{KEM}}$ and for PPT adversaries, we have $|N'| \ll |\mathcal{U}|$. That is, the number of unique users is much smaller than the size of user naming space.

Proof (Theorem 3). We use the game sequence in Figure 14 to prove Theorem 3. We assume that there is no collision among all α 's and among the outputs of random oracles G and H . This assumption adds $\frac{N'^2}{2^\ell} + \frac{(Q_{\text{CT}} + Q_{\text{H}})^2}{|\mathcal{K}|}$ to our final bound. Here N' is the number of distinct α during the game. Except for that, we have

$$\Pr \left[\text{OW-PCA}_{\text{KEM}}^A \Rightarrow 1 \right] = \Pr \left[G_0^A \Rightarrow 1 \right]$$

Game G_1 (Use RFs/RPs to derive randomness). In this game, we use random functions/permutations to derive all randomness used in CHALPK , CHALCT , G , H , and H' . Specifically,

- Random oracles G , H , and H' are simulated (without recording) by functions $f_g, f_h, f_{\tilde{h}}$, respectively.
- We use random functions f_{prfk} and f_{kg} to simulate the implicit-rejection secret key s and the key pair (pk, sk) in oracle CHALPK .
- We use random permutation h_{msg} to generate the messages in CHALCT (cf. line 21) and use random permutation h_{tag} to generate random strings of challenge public keys (cf. line 17).

All changes except for the generation of messages are conceptual. For the generation of messages in CHALCT, since we use a random permutation to replace uniformly random sampling, by the switching lemma, we have

$$|\Pr[\mathbf{G}_0^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1]| \leq \frac{(Q_{\text{CT}} + 1)^2}{|\mathcal{M}|}$$

Game $\mathbf{G}_2$ and $\mathbf{G}_3$ (Compute KEM keys without messages). In game $\mathbf{G}_2$, we set $H(\alpha, m, c) := f_{\text{Hkey}}(u, c)$ if $u = h_{\text{tag}}^{-1}(\alpha)$ and $c = \text{Enc}(\text{pk}_u, m; G(m))$, where f_{Hkey} is a random function (cf. lines 48 to 50). Then, in game $\mathbf{G}_3$, we compute the challenge KEM key k in CHALCT as $k := f_{\text{Hkey}}(u, c)$.

We first bound the probability difference caused by the changes introduced in $\mathbf{G}_2$. In $\mathbf{G}_2$, we only program those queries $H(\alpha, m, c)$ that α is the random string of some challenge public key (i.e., $\exists u \in \mathcal{U}$ such that $\alpha = h_{\text{tag}}(u)$), thus we can rewrite the random oracle H in the following form:

$$H(\alpha, m, c) = \begin{cases} f_{\text{Hkey}}(u, c), \text{ where } u = h_{\text{tag}}^{-1}(\alpha) & \text{if } c = \text{Enc}(\text{pk}_u, m; G(m)), \\ f_H(\alpha, m, c), & \text{otherwise.} \end{cases}$$

Since h_{tag} is an injection, the only way to find a collision of H is by finding some m and m' such that $\text{Enc}(\text{pk}_u, m; G(m)) = c = \text{Enc}(\text{pk}_u, m'; G(m'))$. Therefore, to prove that H in $\mathbf{G}_2$ still behaves as a random oracle, it suffices to show that the function $\text{Enc}(\text{pk}_u, \cdot; G(\cdot))$ is injective for any pk_u (that generated by CHALPK).

We use the the correctness definition in Definition 8 to show that, if H is a classical¹ random oracle, then $\text{Enc}(\text{pk}, \cdot; G(\cdot))$ is injective. Suppose that $\mathcal{A}$ queried H on some m and m' where $\text{Enc}(\text{pk}, m; G(m)) = \text{Enc}(\text{pk}, m'; G(m'))$. Since H is a classical random oracle, we can extract m^* (either m or m') from the RO queries and thus find a message-ciphertext pair (m^*, c) such that $c = \text{Enc}(\text{pk}, m^*; G(m^*))$ and $\text{Dec}(\text{sk}, c) \neq m^*$. Note that in $\mathbf{G}_2$, G is a random oracle that can be accessed in a black-box way, thus $c := \text{Enc}(\text{pk}, m^*; G(m^*))$ is equivalent to $c \leftarrow \text{Enc}(\text{pk}, m^*)$. Therefore, if $\mathcal{A}$ queried H on some m and m' where $\text{Enc}(\text{pk}, m; G(m)) = \text{Enc}(\text{pk}, m'; G(m'))$, then it actually breaks the correctness of PKE. We can use the memory-tight guessing technique in Lemma 1 to ensure that repeated CHALPK queries always return the same pk. By this technique, there are at most N' non-repeated challenge public keys during the game, by a union bound, we have

$$|\Pr[\mathbf{G}_1^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1]| \leq N' \cdot \delta$$

In game $\mathbf{G}_2$, we have $k := f_{\text{Hkey}}(\alpha_u, m, c) = f_{\text{Hkey}}(u, c)$ because $H(\alpha, m, c)$ is set as $f_{\text{Hkey}}(u, c)$ if $u = h_{\text{tag}}^{-1}(\alpha)$ and $c = \text{Enc}(\text{pk}_u, m; G(m))$. Therefore, the change introduced in $\mathbf{G}_3$ is conceptual, thus we have

$$\Pr[\mathbf{G}_2^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1].$$

Game $\mathbf{G}_4$ (Simulate PCO without sk). In this game, we no longer use secret keys to simulate the PCO oracle. Instead, to determine whether a KEM key k is the decapsulated key of a ciphertext c with respect to pk_u , we only check if $k = f_{\text{Hkey}}(\alpha_u, c)$ or $k = H'(\alpha_u, s_u, c)$ (cf. lines 32 to 34).

We first analyze the output distributions of PCO in $\mathbf{G}_3$ and $\mathbf{G}_4$. Suppose that $\mathcal{A}$ queries PCO on (u, c, k) .

- If c causes implicit rejection, then the output distributions of PCO in $\mathbf{G}_3$ and $\mathbf{G}_4$ are identical. This is because, if c causes implicit rejection, then both $\mathbf{G}_3$ and $\mathbf{G}_4$ check whether k equals $H'(\alpha_u, s_u, c)$.
- If c does not cause implicit rejection, then we have $\text{Decaps}((\text{sk}_u, s_u, \alpha), c) = H(\alpha_u, m, c)$. By the changes made in $\mathbf{G}_2$, we also have $H(\alpha_u, m, c) = f_{\text{Hkey}}(u, c)$. In this case, we have $\llbracket k = f_{\text{Hkey}}(u, c) \rrbracket = \llbracket k = \text{Decaps}((\text{sk}_u, s_u, \alpha), c) \rrbracket$.

Therefore, the change introduced in $\mathbf{G}_4$ is conceptual, thus we have

$$\Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1].$$

¹ We emphasize *classical random oracle* because this approach may not be applied in the quantum random oracle model

Game $\mathcal{B}^{\text{CHALPK}, \text{CHALCT}}(\text{par}_{\text{PKE}})$ 00 $f_{\text{kg}} \leftarrow \text{Func}(\mathcal{U}, \mathcal{R}_{\text{ekg}})$ 01 $f_{\text{prfk}} \leftarrow \text{Func}(\mathcal{U}, \{0, 1\}^l)$ 02 $f_{\text{g}} \leftarrow \text{Func}(\mathcal{M}, \mathcal{R}_{\text{enc}})$ 03 $f_{\text{h}} \leftarrow \text{Func}(\mathcal{M} \times \mathcal{C}, \mathcal{K})$ 04 $f_{\text{h}} \leftarrow \text{Func}(\{0, 1\}^l \times \mathcal{C}, \mathcal{K})$ 05 $(h_{\text{msg}}, h_{\text{msg}}^{-1}) \leftarrow \text{InjF}(\mathcal{U} \times \mathcal{I}, \mathcal{M})$ 06 $(h_{\text{tag}}, h_{\text{tag}}^{-1}) \leftarrow \text{InjF}(\mathcal{U}, \{0, 1\}^\ell)$ 07 $f_{\text{hkey}} \leftarrow \text{Func}(\mathcal{U} \times \mathcal{C}, \mathcal{K})$ 08 par := par _{PKE} 09 $O := (\text{CHALPK}, \text{CHALCT}, \text{PCO})$ 10 $(u^*, i^*, k^*) \leftarrow \mathcal{A}^O(\text{par})$ 11 $c^* := \text{CHALCT}(u^*, i^*)$ 12 $b' := \text{PCO}(u^*, c^*, k^*)$ 13 output b'	Oracle $\text{CHALPK}(u)$ 14 $\text{pk}_u := \text{CHALPK}(u)$ 15 $\alpha_u := h_{\text{tag}}(u)$ 16 $s_u := f_{\text{prfk}}(u)$ 17 return (pk_u, α_u) Oracle $\text{CHALCT}(u, i)$ 18 $m := h_{\text{msg}}(u, i)$ 19 $c \leftarrow \text{CHALCT}(u, i, m)$ 20 $k := f_{\text{hkey}}(u, c)$ 21 return c Oracle $\text{PCO}(u, c, k)$ 22 $k'_0 := f_{\text{hkey}}(\alpha_u, c)$ 23 $k'_1 := \text{H}'(\alpha_u, s_u, c)$ 24 if $k = k'_0 \vee k'_1$ 25 return 1 26 else return 0	Oracle $\text{G}(m)$ 27 if $\perp \neq (u, i) := h_{\text{msg}}^{-1}(m)$ 28 abort and output 1 29 return $f_{\text{g}}(m)$ Oracle $\text{H}(\alpha, m, c)$ 30 if $h_{\text{msg}}^{-1}(m) \neq \perp$ 31 abort and output 1 32 if $\perp \neq u := h_{\text{tag}}^{-1}(\alpha)$ 33 if $c = \text{Enc}(\text{pk}_u, m; \text{G}(m))$ 34 return $f_{\text{hkey}}(u, c)$ 35 return $f_{\text{h}}(\alpha, m, c)$ Oracle $\text{H}'(\alpha, s, c)$ 36 return $f_{\text{h}}(\alpha, s, c)$
--	---	---

Fig. 15. Reduction $\mathcal{B}$ in bounding $\mathbf{G}_6$ and $\mathbf{G}_7$ in the proof of Theorem 3. $\mathcal{B}$ simulates $\mathbf{G}_6$ or $\mathbf{G}_7$ for $\mathcal{A}$ to break the IND-CPA security of PKE.

Game $\mathbf{G}_5$, $\mathbf{G}_6$, and $\mathbf{G}_7$ (Decouple messages and randomness). We use the game sequence $\mathbf{G}_5 - \mathbf{G}_7$ to decouple the message m and randomness used in generating challenge ciphertext. Namely, in $\mathbf{G}_4$, challenge ciphertexts are generated as $c := \text{Enc}(\text{pk}, m; \text{G}(m))$, while in $\mathbf{G}_7$, challenge ciphertexts are generated as $c \leftarrow \text{Enc}(\text{pk}, m)$. This is an intermediate step for preparing the reduction to the IND-CPA security, since in IND-CPA, we cannot control the randomness of challenge ciphertexts.

Game $\mathbf{G}_5$: If $\mathcal{A}$ queries G on m that $h_{\text{msg}}^{-1}(m) \neq \perp$, then we return $f_{\text{msg}}(u, i)$, where f_{msg} is a random function and $(u, i) := h_{\text{msg}}^{-1}(m)$ (cf. lines 39 to 40). Moreover, $f_{\text{msg}}(u, i)$ is used as the randomness for query $\text{CHALCT}(u, i)$ (cf. line 23).

We claim that this change does not affect the winning advantage of $\mathcal{A}$. Firstly, h_{msg} is an injection, so the output distribution of G does not change. Then, since $\text{G}(m) = f_{\text{msg}}(u, i)$ if $h_{\text{msg}}^{-1}(m) \neq \perp$, we have $\text{Enc}(\text{pk}, h_{\text{msg}}(u, i); \text{G}(m)) = \text{Enc}(\text{pk}, h_{\text{msg}}(u, i); f_{\text{msg}}(u, i))$. Therefore, the change made in $\mathbf{G}_5$ is conceptual and we have

$$\Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1] = \Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1].$$

Game $\mathbf{G}_6$: If $\mathcal{A}$ queries G on m or H on $(\cdot, m, \cdot)$, where $h_{\text{msg}}^{-1}(m) \neq \perp$, then we abort the game and return 1 to indicate that the adversary wins this game (cf. line 41 and lines 46 to 47). This change only increases the winning advantage (i.e., make the game outputs 1) of $\mathcal{A}$, thus we have

$$\Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1] \leq \Pr[\mathbf{G}_6^{\mathcal{A}} \Rightarrow 1].$$

In $\mathbf{G}_6$, the random function f_{msg} can be accessed in a black-box way, which is essential for the IND-CPA reduction in the next game.

Game $\mathbf{G}_7$: The challenge ciphertext in query $\text{CHALCT}(u, i)$ is generated using the message $0^{|m|}$ instead of m , where $m = h_{\text{msg}}(u, i)$.

We construct a reduction from the IND-CPA security of PKE. Reduction $\mathcal{B}$ is shown in Figure 15. If $\mathcal{B}$ plays $\text{IND-CPA}_{\text{PKE}, 0}$ (resp., $\text{IND-CPA}_{\text{PKE}, 1}$), then the challenge ciphertext is the encryption of the challenge ciphertext m , and thus $\mathcal{B}$ perfectly simulates $\mathbf{G}_6$ (resp. $\mathbf{G}_7$) for $\mathcal{A}$. Therefore, we have

$$|\Pr[\mathbf{G}_6^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_7^{\mathcal{A}} \Rightarrow 1]| \leq \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{B}),$$

and the running time and memory usage of $\mathcal{B}$ is

$$\begin{aligned} \text{Time}^*(\mathcal{B}) &= \text{Time}^*(\mathcal{A}) + O(Q) + O(Q_{\text{h}}) \cdot \text{Time}(\text{PKE}) \\ \text{Mem}^*(\mathcal{B}) &= \text{Mem}^*(\mathcal{A}) + O(\text{Mem}(\text{PKE})), \end{aligned}$$

where Q is the total number of queries issued by $\mathcal{A}$.

BOUND THE WINNING PROBABILITY OF $\mathcal{A}$. In $\mathbf{G}_7$, there are two ways for $\mathcal{A}$ to win the game:

- Case 1: $\mathcal{A}$ triggers the abort conditions in $\mathbf{G}$ and $\mathbf{H}$. That is, $\mathcal{A}$ queries $\mathbf{G}$ on m or $\mathbf{H}$ on $(\cdot, m, \cdot)$ where $h_{\text{msg}}^{-1}(m) \neq \perp$.

In $\mathbf{G}_7$, all challenge ciphertexts are encryption of $0^{|m|}$ and all messages in $\mathcal{M}$ are fix-length, so we have that CHALCT does not reveal any information about h_{msg} (and its inverse). Since h_{msg} is a random permutation and $\mathcal{A}$ queries $\mathbf{G}$ and $\mathbf{H}$ at most $(Q_{\mathbf{G}} + Q_{\mathbf{H}})$ times (including all indirect queries), the probability that $\mathcal{A}$ finds a message m such that $h_{\text{msg}}^{-1}(m) \neq \perp$ is $\frac{|\mathcal{U}| \cdot (Q_{\mathbf{G}} + Q_{\mathbf{H}})}{|\mathcal{M}|}$.

- Case 2: $\mathcal{A}$ successfully finds a (u^*, i^*, k^*) such that $\text{PCO}(u^*, c^*, k^*)$, where $c^* := \text{CHALCT}(u^*, i^*)$. To make PCO outputs 1, the adversary needs to find k^* such that either $k^* = f_{\mathbf{H}}(u^*, c^*)$ or $k^* = f_{\mathbf{H}}(\alpha_{u^*}, s_{u^*}, c^*)$. Since $f_{\mathbf{H}}$ is a random function that cannot be accessed by $\mathcal{A}$, the only way for $\mathcal{A}$ to learn information about $f_{\mathbf{H}}$ is by querying PCO and $\mathbf{H}$. Since there are $(1 + Q_{\text{PCO}} + Q_{\mathbf{H}})$ queries to $\mathbf{H}$, the probability that $\mathcal{A}$ successfully “predicts” the output of $f_{\mathbf{H}}$ is bounded by $\frac{1 + Q_{\text{PCO}} + Q_{\mathbf{H}}}{|\mathcal{K}|}$. Moreover, for any $u \in \mathcal{U}$, s_u is uniformly random and independent of the view of the adversary. Since there are at most $(1 + Q_{\text{PCO}} + Q_{\mathbf{H}'})$ queries to $\mathbf{H}'$ during the game, the probability that $\mathcal{A}$ successfully “predicts” the output of $\mathbf{H}'$ is bounded by $\frac{1 + Q_{\text{PCO}} + Q_{\mathbf{H}'}}{2^l}$.

By combining the probability bound of above two cases, we have

$$\Pr[\mathbf{G}_7^{\mathcal{A}} \Rightarrow 1] \leq \frac{|\mathcal{U}| \cdot (Q_{\mathbf{G}} + Q_{\mathbf{H}})}{|\mathcal{M}|} + \frac{1 + Q_{\text{PCO}} + Q_{\mathbf{H}}}{|\mathcal{K}|} + \frac{(N' + 1) \cdot (1 + Q_{\text{PCO}} + Q_{\mathbf{H}})}{2^l}$$

In conclusion, by combining all probability differences, we have

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{OW-PCA}}(\mathcal{A}) &= \Pr[\text{OW-PCA}_{\text{KEM}}^{\mathcal{A}} \Rightarrow 1] \\ &\leq \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) + N' \cdot \delta + \frac{(Q_{\text{CT}} + 1)^2 + |\mathcal{U}| \cdot (Q_{\mathbf{G}} + Q_{\mathbf{H}})}{|\mathcal{M}|} \\ &\quad + \frac{1 + Q_{\text{PCO}} + Q_{\mathbf{H}} + (Q_{\text{CT}} + Q_{\mathbf{H}})^2}{|\mathcal{K}|} + \frac{(N' + 1)(1 + Q_{\text{PCO}} + Q_{\mathbf{H}'}) + N'^2}{2^l}. \end{aligned}$$

This concludes the proof of Theorem 3.

FROM SINGLE-USER, SINGLE CHALLENGE PRIMITIVES. In Theorem 3, if we consider the single-user, single-challenge setting, then there are only one challenge public key and one challenge ciphertext, so we have Corollary 1 (by setting $Q_{\text{pk}} := 1$ and $Q_{\text{CT}} := 1$). While the bound given in Corollary 1 is somewhat loose because in the single-user, single-challenge setting, we do not need to maintain the namespaces of users and challenge ciphertexts, it still shows that IND-CPA-secure PKE can be tightly transformed into a OW-PCA-secure KEM by the generic construction in Figure 13 in the single-user, single-challenge setting.

Corollary 1. *Let KEM be the KEM construction presented in Figure 13. If the PKE scheme PKE is $(1 - \delta)$ -correct, then for any adversary $\mathcal{A}$ against the OW-PCA security of KEM in the single-user, single-challenge setting, there exists an adversary $\mathcal{B}$ against KEM in the single-user, single-challenge setting such that*

$$\begin{aligned} \text{Adv}_{\text{KEM}}^{\text{OW-PCA}}(\mathcal{B}) &\leq \text{Adv}_{\text{PKE}}^{\text{IND-CPA}}(\mathcal{A}) + \delta + 2^{-\ell} + \frac{4 + Q_{\text{PCO}} + Q_{\mathbf{H}'}}{2^l} \\ &\quad + \frac{1 + Q_{\text{PCO}} + Q_{\mathbf{H}} + (1 + Q_{\mathbf{H}})^2}{|\mathcal{K}|} + \frac{|\mathcal{U}| \cdot (Q_{\mathbf{G}} + Q_{\mathbf{H}}) + 4}{|\mathcal{M}|} \\ \text{Time}^*(\mathcal{B}) &= \text{Time}^*(\mathcal{A}) + O(Q) + O(Q_{\mathbf{H}}) \cdot \text{Time}(\text{PKE}) \\ \text{Mem}^*(\mathcal{B}) &= \text{Mem}^*(\mathcal{A}) + \log |\mathcal{U}| + \log |\mathcal{I}| + O(\text{Mem}(\text{PKE})), \end{aligned}$$

where $\text{Time}(\text{PKE})$ and $\text{Mem}(\text{PKE})$ are the running time and memory usage of one execution of PKE, respectively, and $\mathcal{A}$ issues at most $Q_{\mathbf{G}}$, $Q_{\mathbf{H}}$, and $Q_{\mathbf{H}'}$ queries to oracles $\mathbf{G}$, $\mathbf{H}$, and $\mathbf{H}'$, respectively. Q is the total number of queries issued by $\mathcal{A}$.

In Lemma 4, we further show that OW-PCA security in the single-user, single-challenge setting implies the same security in the multi-user, multi-challenge setting. Although the reduction is not tight in terms of the winning advantage, it preserves the memory-tightness. Since the proof of Lemma 4 is quite simple, we only give the construction of the reduction in Figure 16.

Game $\mathcal{B}^{\text{PCO}'}(\text{par}_{\text{KEM}}, \text{pk}^*, c^*)$	Oracle $\text{CHALPK}(u)$	Oracle $\text{CHALCT}(u, i)$
00 $f_1 \leftarrow \text{Func}(\mathcal{U}, [Q_{\text{pk}}])$	10 $(\text{pk}, \text{sk}) := \text{GetKey}(u)$	18 if $f_1(u^*) = U^* \wedge f_2(i^*) = I^*$
01 $f_2 \leftarrow \text{Func}(\mathcal{U}, [Q_{\text{CT}}])$	11 return (pk, sk)	19 $c := c^*$
02 $U^* \leftarrow [Q_{\text{pk}}], I^* \leftarrow [Q_{\text{CT}}]$		20 else
03 $\text{par} := \text{par}_{\text{PKE}}$	Alg $\text{GetKey}(u)$	21 $(\text{pk}, \text{sk}) := \text{GetKey}(u)$
04 $O := (\text{CHALPK}, \text{CHALCT}, \text{PCO})$	12 if $f_1(u) = U^*$	22 $(c, k) := \text{Encaps}(\text{pk}; f_e(u, i))$
05 $(u^*, i^*, k^*) \leftarrow \mathcal{A}^O(\text{par})$	13 $\text{pk} := \text{pk}^*, \text{sk} := \perp$	23 return c
06 if $f_1(u^*) = U^* \wedge f_2(i^*) = I^*$	14 else	Oracle $\text{PCO}(u, c, k)$
$\wedge \text{PCO}'(c^*, k^*) = 1$	15 $r := f_k(u)$	24 if $f_1(u) = U^*$
07 return k^*	16 $(\text{pk}, \text{sk}) := \text{KG}(\cdot; r)$	25 return $\text{PCO}'(c, k)$
08 else	17 return (pk, sk)	26 $(\text{pk}, \text{sk}) := \text{GetKey}(u)$
09 return $k \leftarrow \mathcal{K}$		27 return $\llbracket k = \text{Decaps}(\text{sk}, c) \rrbracket$

Fig. 16. Reduction $\mathcal{B}$ for proving Lemma 4.

Lemma 4. *Let KEM be a KEM scheme. For any adversary $\mathcal{A}$ against the OW-PCA security of KEM in the single-user, single-challenge setting, there exists an adversary $\mathcal{B}$ (cf. Figure 16) against the OW-PCA security of KEM in the multi-user, multi-challenge setting such that*

$$\begin{aligned}
\text{Adv}_{\text{KEM}}^{\text{OW-mucPCA}}(\mathcal{B}) &\leq Q_{\text{pk}} \cdot Q_{\text{CT}} \cdot \text{Adv}_{\text{KEM}}^{\text{OW-sucPCA}}(\mathcal{B}) \\
\text{Time}^*(\mathcal{B}) &\approx \text{Time}^*(\mathcal{A}) + O(Q) \cdot \text{Time}(\text{KEM}) \\
\text{Mem}^*(\mathcal{B}) &\approx \text{Mem}^*(\mathcal{A}) + \log |\mathcal{U}| + \log |\mathcal{I}| + O(\text{Mem}(\text{KEM})),
\end{aligned}$$

where $\text{Time}^*(\text{KEM})$ and $\text{Mem}^*(\text{KEM})$ are the (reduced) running time and memory usage of one execution of KEM, respectively. $\mathcal{A}$ issues at most Q_{pk} , Q_{CT} , and Q_{PCO} queries to oracles CHALPK, CHALCT, and PCO, respectively. $Q = Q_{\text{pk}} + Q_{\text{PCO}} + Q_{\text{CT}}$ is the total number of queries issued by $\mathcal{A}$.

D.2 Postponed proof in Section 4

PROOF OF LEMMA 2. We first prove that, in $\mathbf{G}_4$, different sequence numbers seq_1 generated from SESSION_I always correspond to different protocol messages. Let $\text{seq}_1, \text{seq}'_1$ be arbitrary sequence numbers output by SESSION_I . Suppose that $(u, v, \text{cnt}) = h_{\text{seq}}^{-1}(\text{seq}_1)$ and $(u', v', \text{cnt}') = h_{\text{seq}}^{-1}(\text{seq}'_1)$. Each query to SESSION_I increments the counter, so we always have $\text{cnt} \neq \text{cnt}'$. Since h_{ncc} is injective, this also means that $h_{\text{ncc}}(u, v, \text{cnt}) \neq h_{\text{ncc}}(u', v', \text{cnt}')$, namely, the α values of seq_1 and seq'_1 are different. Thus, if seq_1 comes from SESSION_I , then the game restores the correct (u, v, cnt) , which means that $\mathcal{A}$ cannot get any distinguishing advantage from querying $\text{DER}_I(*, \text{seq}_1, *)$ if seq_1 is generated from SESSION_I .

Therefore, to distinguish $\mathbf{G}_4$ from $\mathbf{G}_3$, $\mathcal{A}$ has to query $\text{DER}_I(u, v, \text{seq}_1, *)$, where seq_1 is not from SESSION_I , such that $(u', v', \text{cnt}') = h_{\text{seq}}^{-1}(\text{seq}_1)$, $(u, v) = (u', v')$, and $\text{cnt} \geq \text{cnt}'$. Since h_{seq} is a random injection and $\mathcal{A}$ queries SESSION_I at most N_{ses} times, we always have $\text{cnt} \leq N_{\text{ses}}$ and there are at most N_{ses} points (i.e., those seq_1 's generated by SESSION_I) of $h_{\text{seq}}(\mathcal{U}^2 \times [N])$ are learned by $\mathcal{A}$. The probability that $\mathcal{A}$ finds an seq_1 such that $(u, v, \text{cnt}') = h_{\text{seq}}^{-1}(\text{seq}_1)$, $(u, v) = (u', v')$, and $\text{cnt} \geq \text{cnt}'$ is less than $\frac{N_{\text{ses}}}{|\mathcal{U}|^2 N - N_{\text{ses}}}$. By setting $|\mathcal{U}|^2 N \geq 2N_{\text{ses}}$ (which is reasonable as we can control N to be larger than $2N_{\text{ses}}$), we have

$$\frac{N_{\text{ses}}}{|\mathcal{U}|^2 N - N_{\text{ses}}} = \frac{2N_{\text{ses}} - N_{\text{ses}}}{|\mathcal{U}|^2 N - N_{\text{ses}}} \leq \frac{2N_{\text{ses}}}{|\mathcal{U}|^2 N}$$

Therefore, since $\mathcal{A}$ queries DER_I at most N_{ses} times, by a union bound, we have

$$|\Pr[\mathbf{G}_3^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_4^{\mathcal{A}} \Rightarrow 1]| \leq N_{\text{ses}} \cdot \frac{2N_{\text{ses}}}{|\mathcal{U}|^2 N} = \frac{2N_{\text{ses}}^2}{|\mathcal{U}|^2 N}$$

PROOF OF LEMMA 3. We first prove that $\mathbf{H}$ behaves as a random oracle except with a negligible probability. On input $(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$, we first let $(u_1, v_1, \text{cnt}_1) := h_{\text{ncc}}^{-1}(\alpha)$ and $(u_2, v_2, \text{cnt}_2) := h_{\text{ncc}}^{-1}(\beta)$. Since $\mathbf{H}$ in $\mathbf{G}_5$ and $\mathbf{G}_6$ proceed identically if either (u_1, v_1, cnt_1) is $\perp$ or (u_2, v_2, cnt_2) is $\perp$, we only consider the case that both are not $\perp$. We further let $(\text{epk}', \text{esk}') := \text{KEM.KG}(\cdot; f_{\text{ekg}}(\text{cnt}_1))$. Then, we can re-write $\mathbf{H}$ in the following form: $\mathbf{H}(\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k) = f_{\mathbf{H}} \circ f(u, v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, k)$, where

$$f(\text{Trans}, k) = \begin{cases} (\text{Trans}, f_k(\text{epk}, c)), & \text{if } \text{epk} = \text{epk}' \wedge k = \text{KEM.Decaps}(\text{esk}', c), \\ (\text{Trans}, f_k(\text{epk}, c)), & \text{else if } (c, k) = \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}_2)), \\ (\text{Trans}, k), & \text{otherwise.} \end{cases}$$

Here $\text{Trans} = (\text{pk}_u, \text{pk}_v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2)$.

The function f behaves like an injective function unless (1) $f_k(\text{epk}, c)$ in the first case equals to the k in the third case, or (2) there exist some $(\text{epk}, \text{esk}, c, k)$ produced by SESSION_I and Der_R such that $(c, k) = \text{KEM.Encaps}(\text{epk}; f_{\text{enc}}(\text{epk}, \text{cnt}_2))$ but $k \neq \text{KEM.Decaps}(\text{esk}, c)$. Since $\mathcal{A}$ issues at most Q_H queries to H and f_k is a random function, we have that f is injective except with the probability $(Q_H + N_{\text{ses}})^2/|\mathcal{K}| + N_{\text{ses}} \cdot \delta_{\text{KEM}}$, where N_{ses} is the total number of sessions during the game and δ_{KEM} is the decryption error bound of KEM.

Then we argue that, as long as H behaves as a random oracle, then these modifications do not change the distributions of outputs of DER_I , SESSION_R , REV , and TEST . Let $\text{sid} = (*, u, v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2)$ be a session identifier. If sid is valid, i.e., generated from DER_I or SESSION_R , then at least one if-condition in lines 55 and 58 hold and thus the session key of sid is computed as $f_H(u, v, \alpha, \beta, \text{epk}, c, \sigma_1, \sigma_2, f_k(\text{epk}, c))$, which means that the distributions of oracles DER_I and SESSION_R do not change. For oracles REV and TEST , since we use CheckSid to exclude all sid that does not exist, and the outputs of REV and TEST are consistent to the outputs of DER_I and SESSION_R , their distributions do not change as well.

If f is injective and f_H is a random function (and their domains and codomains match), then $H = f_H \circ f$ also behaves as a random function. Therefore, we have

$$|\Pr[\mathbf{G}_5^{\mathcal{A}} \Rightarrow 1] - \Pr[\mathbf{G}_6^{\mathcal{A}} \Rightarrow 1]| \leq \frac{(Q_H + N_{\text{ses}})^2}{|\mathcal{K}|} + N_{\text{ses}} \cdot \delta_{\text{KEM}}.$$